

KV CACHING

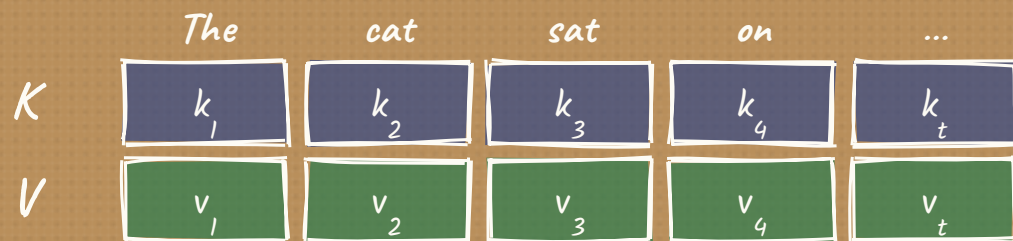
from first principles → production inference

Notebook complete edition · Parts 1–22

Covers math · memory · GPUs · vLLM-style serving · code · interviews

Name

Started 23 Sep 2026



one page of the cache · one layer

A · Foundations B · Memory C · Serving
D · Optimization E · Hands-on F · Production

compute once · read forever

Before you start

This notebook takes you from "what is a KV cache?" to reasoning about it the way an inference engineer does: the math, the tensor shapes, the GPU memory, the serving systems built around it, code you can run on your own RTX 4050, and the interview and system-design questions that test all of it.

It is written to be **read slowly**. Every part follows one rule: **intuition first, terminology second, numbers always**. Every optimization is introduced through the bottleneck that motivated it.

→ Colour code (used on every page)

Ink	Meaning
black / blue pen	the explanation itself
orange = Q · blue = K · green = V	query, key and value always use these colours
red pen	pitfalls, wrong mental models, "careful!"
grey pencil	side notes and advanced asides. Safe to skip on a first read.
yellow sticky	interview tip: how to say it out loud
green box	the one insight to copy into your handwritten notes

→ The pattern behind every optimization

Problem → Naive approach → Bottleneck → Optimization → Trade-off	
Problem	What are we trying to do? (e.g. serve many chats at once)
Naive	The obvious first attempt (e.g. reserve max-length memory per chat)
Bottleneck	The specific resource that runs out first (e.g. memory is reserved but unused)
Optimization	The idea that attacks exactly that resource (e.g. PagedAttention)
Trade-off	What you pay for it (e.g. indirection in the kernel, a block table to manage)

When you meet a new technique in the wild, fill in these five rows. If you can't name the bottleneck, you don't yet understand the technique.

→ How each part ends

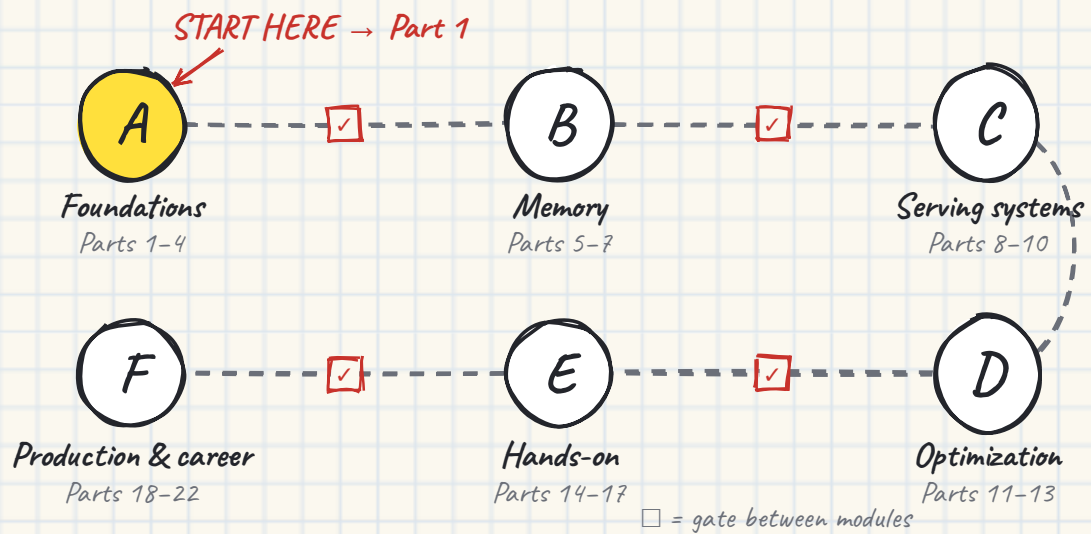
A **checkpoint**: 3 recall questions, 2 reasoning questions, 1 practical engineering question, 1 calculation or coding exercise, and the single insight for your handwritten notes. **Answer in your own notebook before opening the answer key** at the back. The questions are the real curriculum; the pages just prepare you for them.

Contents

1	Build the Intuition: Q, K, V; causality; with vs without cache; the notebook model	6
2	Attention Math: deriving the cache from $Q = XW$, tensor shapes through prefill and decode	17
3	Prefill vs Decode: compute-bound vs bandwidth-bound, TTFT, ITL, throughput vs latency	23
4	Complexity: what gets cheaper and what still grows (no misleading Big-O)	29
5	KV Cache Memory: deriving the formula; 7B / 13B / 70B at 2K–128K; FP32 → INT4	34
6	MHA, MQA and GQA: query heads vs KV heads, and how much memory sharing saves	39
7	KV in GPU Memory: HBM, bandwidth, layouts, contiguous allocation and fragmentation	44
8	PagedAttention: blocks, block tables, allocation, freeing and sharing, the OS-paging analogy	49
9	Continuous Batching: static vs dynamic vs iteration-level scheduling; max concurrent sequences	54
10	Prefix Caching: reusing KV across requests; hashing, prefix matching, eviction, invalidation	58
11	KV Cache Quantization: FP16 → FP8 → INT8 → 4-bit and below; savings, overheads, accuracy	63
12	Long Context: 8K → 1M tokens; memory, attention compute, bandwidth and concurrency	67
13	Advanced Techniques: windows, eviction, compression, offloading, disaggregation, speculation, chunked prefill, KV-aware scheduling	71
14	Implement It From Scratch: a tiny PyTorch decoder, generation with and without a cache, benchmarks	77
15	Inspect a Real Model: use_cache, past_key_values and DynamicCache shapes in Hugging Face	84
16	KV Cache Lab: a portfolio project on your RTX 4050: architecture, experiments, metrics, graphs, README	89
17	Profiling: PyTorch profiler, CUDA memory stats, Nsight, roofline reasoning, diagnosis drills	99
18	Production Engines: how the scheduler, KV manager and kernels interact in vLLM, TensorRT-LLM, SGLang, TGI	105
19	Failure Modes: eleven KV-cache problems: symptom, cause, diagnosis, fix	109
20	Interview Preparation: 30 questions across five levels, plus how a senior interviewer grades answers	115
21	System Design: an LLM service for thousands of users, redesigned under six rounds of new constraints	119
22	First-Principles Check: 13 questions to answer without notes, with model answers and reteach pointers	124
A	Appendix A · Answer key: model answers for every checkpoint in Parts 1–21	128
B	Appendix B · Interview answers: model answers to all 30 Part 20 questions, with the senior-level extras	136
C	Appendix C · Formula sheet, glossary and the papers behind each idea	141

The roadmap

22 parts in 6 modules. Each module ends with a **gate**: something you should be able to do without notes before moving on.



Module	You'll be able to...	Gate (without notes)
A. Foundations 1 Intuition · 2 Attention math · 3 Prefill vs decode · 4 Complexity	derive the KV cache from the attention equation; explain what gets cheaper and what doesn't	trace tensor shapes through prefill and decode
B. Memory 5 Memory math · 6 MHA/MQA/GQA · 7 KV in GPU memory	estimate KV memory for any model on paper; reason about HBM and bandwidth	compute max concurrent requests for a given model and GPU
C. Serving systems 8 PagedAttention · 9 Continuous batching · 10 Prefix caching	explain how a vLLM-style engine allocates, shares and frees KV memory	hand-simulate a block table for 3 requests
D. Optimization 11 Quantization · 12 Long context · 13 Advanced techniques	pick an optimization by first naming the bottleneck it fixes	given a symptom, pick the technique and state its trade-off
E. Hands-on 14 From scratch · 15 HF inspection · 16 KV Cache Lab · 17 Profiling	build, measure and profile KV caching on your RTX 4050	a working benchmark with graphs you can defend
F. Production & career 18 Engines · 19 Failure modes · 20 Interviews · 21 System design · 22 First-principles check	discuss inference infrastructure at a senior level	pass the Part 22 check with no gaps

Notation used throughout

Symbol	Meaning	Example (Llama-2-7B)
L	number of transformer layers	32
H_q	number of query (attention) heads per layer	32
H_{kv}	number of key/value heads per layer ($= H_q$ for MHA)	32 (Llama-3-8B: 8)
d_h	head dimension	128
d	model (hidden) width $= H_q \cdot d_h$	4096
T, t	sequence length / current position	prompt 4,000, then 4,001, ...
B	batch size (sequences processed together)	1 ... 256
b	bytes per stored element	FP16/BF16 = 2, FP8/INT8 = 1
P	parameter count	6.74 B

Units: GB = 10^9 bytes (used for bandwidth and GPU marketing numbers). GiB = 2^{30} bytes (used for memory we compute). KiB = 1,024 bytes. The two differ by about 7% at this scale, which matters in capacity planning.

Your hardware, for the hands-on parts

RTX 4050 (laptop) · 6 GB GDDR6

About **192 GB/s** of memory bandwidth (a 96-bit bus). Compare an A100 80GB at ~2,000 GB/s or an H100 SXM at ~3,350 GB/s. Six GB fits a 0.5B–1.5B model in FP16 with a meaningful KV budget, and every effect in this book (cache vs no cache, bandwidth-bound decode, memory limits on batch × context, prefix reuse) shows up clearly at that scale. Parts 14–17 are designed around it.

How to study this

Read a section → close the notebook → redraw its main diagram from memory → only then move on. If you can't redraw it, you don't own it yet. One part per session is a good pace.

PART 1 · Module A: Foundations

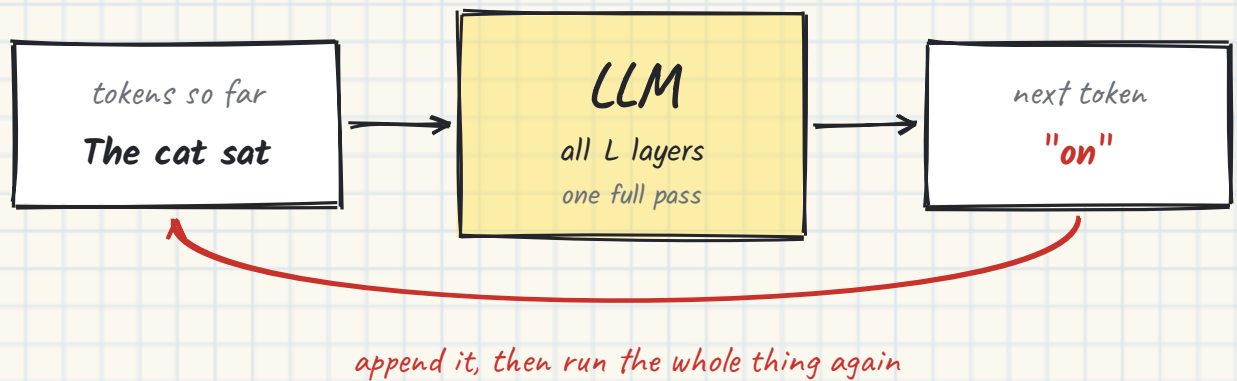
Build the Intuition

Goal of this part

By the end you can explain, **from causality alone**, why a KV cache exists, exactly what it stores, why Q isn't in it, and what it saves versus what it doesn't. No code yet.

1.0 The problem: generation is a loop

A decoder-only LLM produces text **one token at a time**. Each prediction is a full trip through every layer, and the new token is appended to the input for the next trip.



the autoregressive loop: one full forward pass per generated token

Two facts make this expensive:

1. To predict token $t+1$, the model has to take **all** of tokens $1...t$ into account, at **every layer**.
2. From one lap to the next, the input changes by exactly **one token**. Almost everything is the same as last lap.

"What did lap t compute that lap $t+1$ can reuse exactly?"

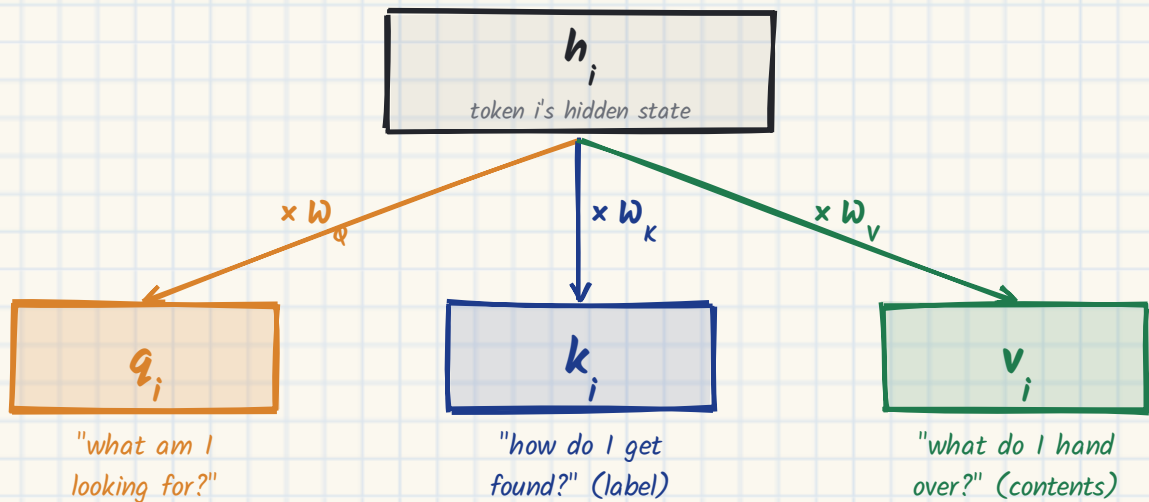
That question is Part 1. The answer turns out to be **the keys and values of every past token at every layer**, and the reason is a single property of decoder models: causality.

"Token" = a sub-word piece, e.g. "The", "cat", "sat". A 4,000-token prompt is roughly 3,000 English words.

1.1 What Q , K and V actually are

Forget the names for a moment. At every layer, each token i carries a hidden vector h_i , a running summary of "what this token means in context so far". Self-attention is how a token **pulls information from other tokens**. To make that possible, each token produces **three**

different views of itself through three learned weight matrices:



→ The filing-cabinet analogy

- **Query q :** the question this token is asking right now.
- **Key k :** the label on the token's filing card, used only to decide relevance.
- **Value v :** the contents of the card, which is what actually gets mixed into the answer.

→ Why keep K and V separate?

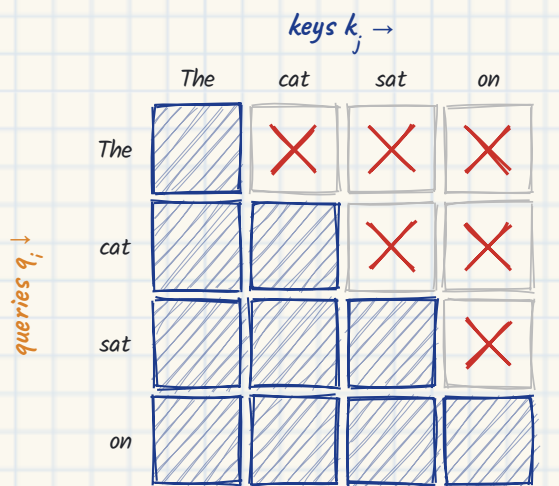
Because "how to find me" and "what I contribute" are different jobs. The token "sat" might be easy to find for any query looking for a verb (its **key**), while what it hands over is tense, the subject it agrees with, and so on (its **value**). One vector couldn't do both jobs well.

Real models run many heads per layer. Each head has its own slice of W_Q , W_K , W_V so each head has its own q , k , v . Everything in Part 1 applies per head. How many K/V heads a model keeps is the story of Part 6.

1.2 Inside self-attention, for one token

- ① **Score:** $s_{ij} = q_i \cdot k_j / \sqrt{d_h}$ for every $j \leq i$ (how relevant is j to my question?)
- ② **Normalize:** $a_{ij} = \text{softmax}_j(s_{ij})$ (weights ≥ 0 that sum to 1)
- ③ **Mix:** $\text{out}_i = \sum_j a_{ij} \cdot v_j$ (a weighted blend of the contents)

Read the ingredients list. To compute out_i , token i needs its own q_i , plus the k_j and v_j of every token $j = 1 \dots i$ (including itself).



lower triangle = allowed · $\times$ = masked ($-\infty$)

the causal mask: each query row sees only keys at
or before itself

→ The causal rule

In a decoder, token i **never** looks at tokens after it. The scores for $j > i$ are set to $-\infty$ before the softmax, so their weight is exactly 0.

This single rule is the foundation of everything in this book. It's why the KV cache is correct, not a lucky approximation (proof in 1.2c).

Why divide by $\sqrt{d_h}$? A dot product of two random d_h -dimensional vectors grows roughly like $\sqrt{d_h}$. Dividing keeps scores in a range where softmax doesn't collapse to a one-hot.

1.2b Worked example with real numbers

Tiny head dimension $d_h = 2$ ($\sqrt{2} \approx 1.414$). The sentence so far is "The cat sat". We compute the attention output for **sat** at one layer, one head.

token j	k_j	v_j
The	[1, 0]	[1, 0]
cat	[0, 1]	[0, 2]
sat	[1, 1]	[1, 1]

Query of the current token:

$$q_{\text{sat}} = [2, 1]$$

j	$q \cdot k_j$	$\div \sqrt{2} = s$	e^s	weight $a = e^s / \Sigma$
The	$2 \cdot 1 + 1 \cdot 0 = 2$	1.414	4.113	0.284
cat	$2 \cdot 0 + 1 \cdot 1 = 1$	0.707	2.028	0.140
sat	$2 \cdot 1 + 1 \cdot 1 = 3$	2.121	8.342	0.576
Σ			14.484	1.000 ✓

$$\text{out}_{\text{sat}} = 0.284 \cdot [1, 0] + 0.140 \cdot [0, 2] + 0.576 \cdot [1, 1] \approx [0.86, 0.86]$$

→ Now "on" arrives. What changes?

"on" brings a new query q_{on} and adds one new row ($k_{\text{on}}, v_{\text{on}}$). The three rows above are **bit-for-bit identical**: they came from $h_{\text{The}}, h_{\text{cat}}, h_{\text{sat}}$ and nothing about those tokens changed. Only these are new: 1 row of K and V (computed once, then kept), and 1 query, 4 scores, 4 weights, 1 output (used once, then discarded).

Try it (answer in the answer key)

Let $q_{on} = [1, 2]$, $k_{on} = [1, -1]$, $v_{on} = [2, 0]$. Reuse the three old rows as they are. Compute the four attention weights and out_{on} . Notice how much of the table you didn't rebuild.

1.2c Why past K and V can never change

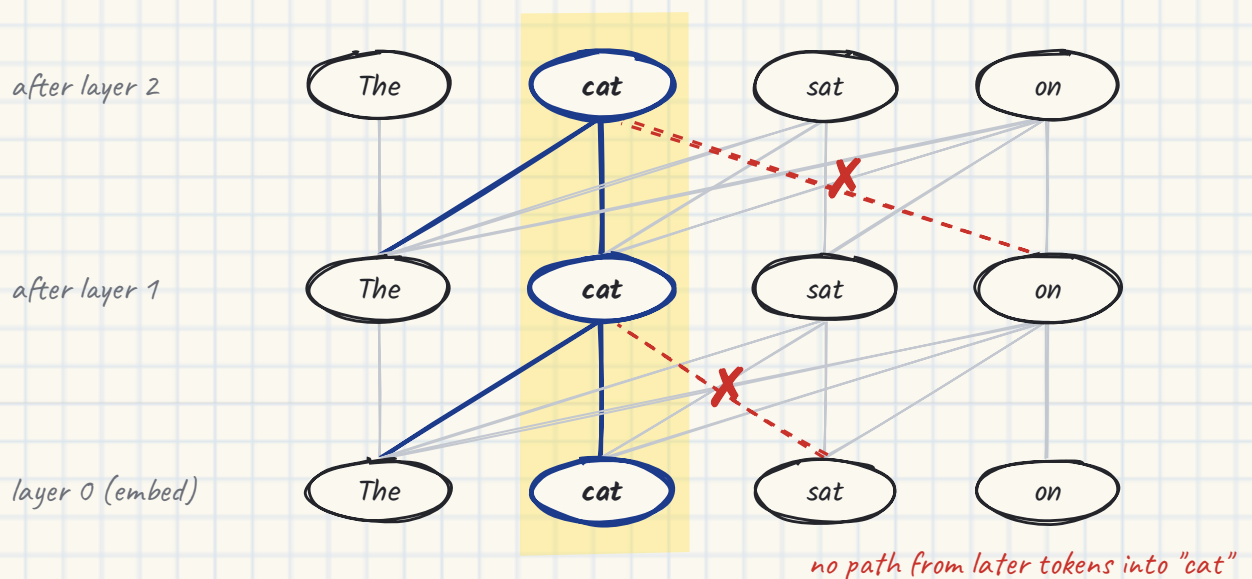
This is the argument an interviewer wants to hear. It's a short **induction over layers**.

Layer 0: $h_i^0 = \text{embedding}(x_i)$ (+ position i). Depends only on token i .

Layer ℓ : $h_i^\ell = \text{MLP}(\text{attention over } h_1^{\ell-1} \dots h_i^{\ell-1})$ (+ residuals, norms). By the causal rule, it uses only positions $\leq i$.

So by induction: h_i^ℓ depends only on tokens $x_1 \dots x_i$ at every layer ℓ .

Since $k_i^\ell = h_i^{\ell-1} \omega_K$ and $v_i^\ell = h_i^{\ell-1} \omega_V$, they too depend only on $x_1 \dots x_i$. Tokens that arrive later cannot reach back and change them.



information flows only upward and to the right. Nothing from "sat" or "on" reaches the "cat" column.

Conclusion

Once token i has been processed, its K and V at every layer are **final**. Compute them once, store them, read them forever. That's the whole justification for a KV cache, and it makes the cache **exact**, not an approximation.

Fine print: this assumes fixed weights and fixed positions. With RoPE, k_i is rotated by its own position i before it's stored. That position never changes, so the cached k is still final.

Hold this thought for checkpoint Q4: which step of the induction breaks if attention could look forward?

1.3 Why generation keeps needing the past

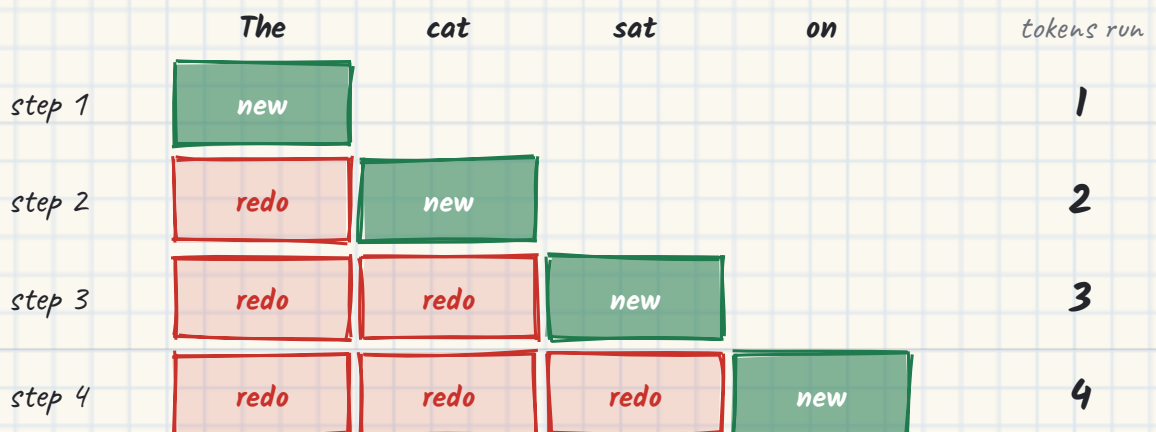
To predict token $t+1$ the model runs the **newest** token through all layers, and **at every layer** that token's query must attend over the keys and values of **all previous tokens**. The past never "finishes". Every new token wants to consult it again.

1.4 "The cat sat" without a KV cache

Feed tokens one at a time, predicting the next each time. Two-layer model. (Part 3 shows that a real prompt goes in as one parallel pass. Token-by-token here is for clarity.)

```

STEP 1 input [The] → predicts "cat"
        L1: q,k,v for The ; attend The→{The} ; MLP
        L2: q,k,v for The ; attend The→{The} ; MLP
STEP 2 input [The, cat] → predicts "sat"
        L1: q,k,v for The, cat ← "The" recomputed
        L2: q,k,v for The, cat ← "The" recomputed
        + attention rows AND full MLPs for both tokens
STEP 3 input [The, cat, sat] → predicts "on"
        every layer: q,k,v for all 3 ← 2 of 3 are repeats
STEP 4 input [The, cat, sat, on] → predicts "the"
        every layer: q,k,v for all 4 ← 3 of 4 are repeats
  
```



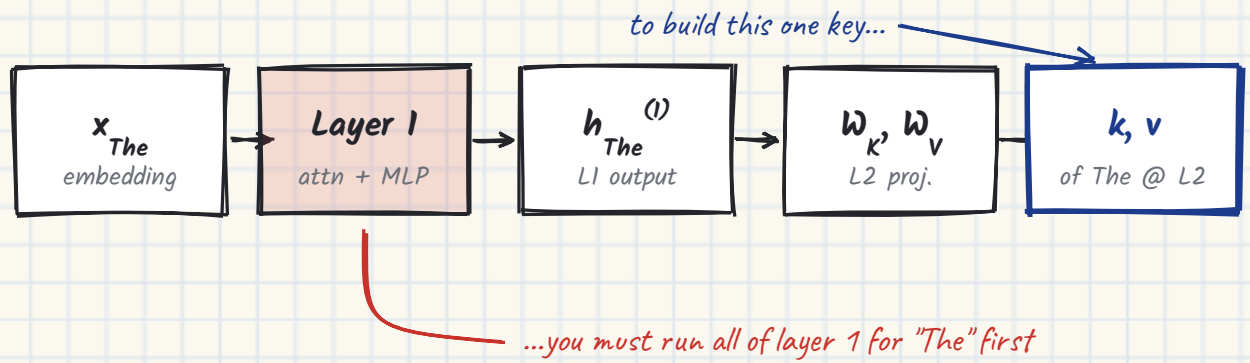
total = 10 token passes, of which 6 are pure repeats

each row is one generation step. Red = work already done in an earlier step.

→ The subtle point people miss

"Without a cache, why not compute just the last row of attention at each layer and skip the rest?"

Because that last row needs k and v for every old token at that layer, and you can't get those without running the layers below for those tokens:



no cache $\Rightarrow$ this chain reruns for **EVERY** old token, **EVERY** step

- To get k_{The} at layer 2 you need h_{The} after layer 1.
- h_{The} after layer 1 needs layer 1's **attention** and **MLP** for "The".
- In a 32-layer model, that chain is 31 layers deep for the top layer.

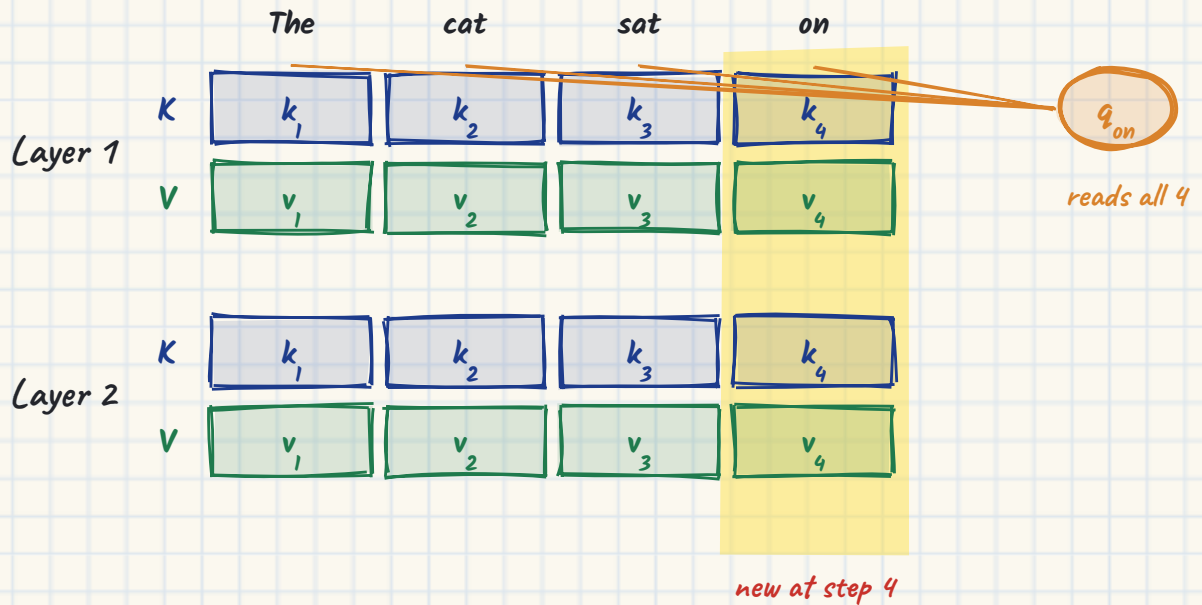
So "no cache" costs more than it sounds

You don't just recompute K and V. You drag **every old token through the entire forward pass** (projections, attention, MLP, norms) at every layer, on every step. The MLP alone holds about $\frac{2}{3}$ of a transformer's parameters, so this is most of the model's compute, repeated.

1.5 The same walkthrough with a KV cache

```

STEP 1 input: The
  L1: q,k,v(The) → store k,v      cache[L1] = {The}
  L2: q,k,v(The) → store k,v      cache[L2] = {The}
STEP 2 input: cat      ← only the NEW token goes in
  L1: q,k,v(cat); append k,v → cache[L1] = {The, cat}
      q_cat attends over cache[L1] (2 keys)
  L2: same              → cache[L2] = {The, cat}
STEP 3 input: sat
  L1: append → {The, cat, sat} ; q_sat vs 3 keys
STEP 4 input: on
  L1: append → {The, cat, sat, on} ; q_on vs 4 keys
  
```



the cache lives per layer. One column is new each step, but the new query still reads every column.

Work per step, with a cache	Amount	Grows with t ?
q, k, v projections, MLP, norms	for 1 token, every layer	no: constant
writes to cache	1 k + 1 v per layer (per KV head)	no
attention scores + weighted sum	t keys and values read per layer	yes: linear in t

1.6 Side-by-side accounting

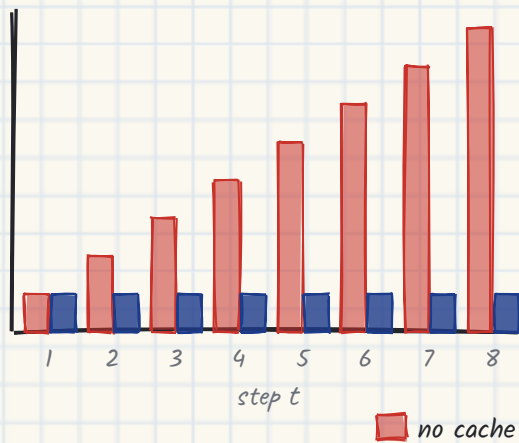
Step t	Tokens through all layers no cache	with cache	Attention scores* no cache	with cache
1	1	1	1	1
2	2	1	3	2
3	3	1	6	3
4	4	1	10	4
Total	10	4	20	10

* per layer, per head. With causal masking, step t without a cache computes a full triangle of $1+2+\dots+t = t(t+1)/2$ scores. With a cache it computes one row of t scores.

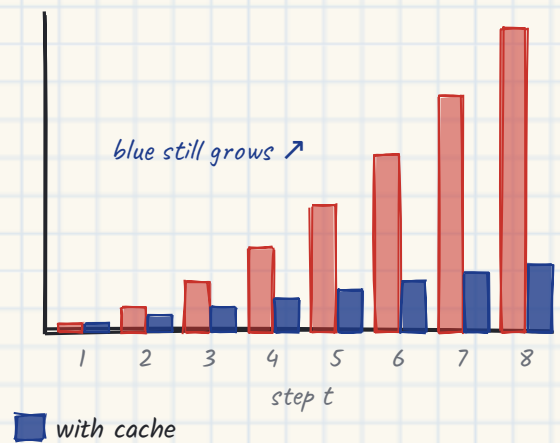
For T steps	No cache	With cache
token forward passes	$\sum t = T(T+1)/2$ (quadratic)	T (linear)
attention scores	$\sum t(t+1)/2 = T(T+1)(T+2)/6$ (cubic)	$T(T+1)/2$ (still quadratic!)
$T = 1,000$	500,500 passes · 167,167,000 scores	1,000 passes · 500,500 scores

That's a ~500× cut in projection + MLP work at 1,000 tokens, and the gap widens as T grows.

token passes per step



attention scores per step

**What did NOT disappear**

At step 4, the new query still reads all 4 keys and values. The cache removes recomputation. It does not remove re-reading. Re-reading a growing cache on every step is the seed of Parts 3, 4, 7 and 12 (memory bandwidth, long context).

1.7 Why cache K and V but not Q?

Ask one question of every tensor: "Who will need this in the future?"

Tensor (old token j)	Who uses it	When	Can it change later?	Cache?
q_j	only token j itself	only at step j	n/a (never read again)	No
k_j	every future token's query	every later step	no (causality)	Yes
v_j	every future token's output	every later step	no (causality)	Yes
scores s_{ij} , weights a_{ij}	only row i	only at step i	new query $\Rightarrow$ new row	No
out $_j$, MLP output	next layer, for token j	same step	n/a	No

A query is a one-time question. Keys and values are a permanent public record.

Reused each step (read from cache)

K and V of every past token, at every layer and every KV head.

Computed fresh (new token only)

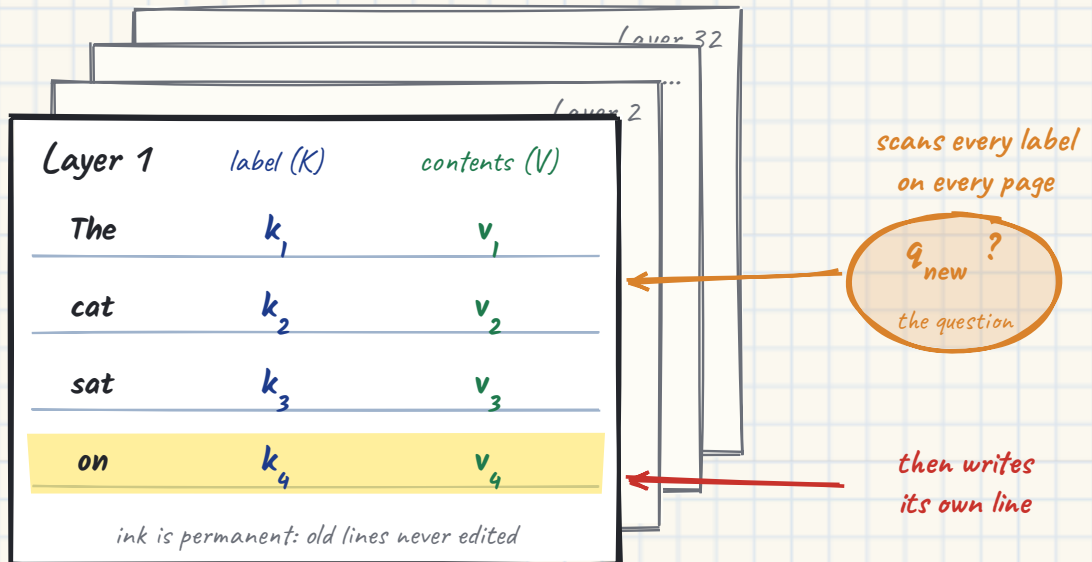
embedding; q , k , v ; scores + softmax over all t keys; weighted sum; MLP; norms; logits.

Advanced aside. Why not cache the layer inputs h_i and re-project K and V each step? You could. With multi-head attention, $K+V$ is $2\times$ the size of h , so this halves memory, but it adds back 2 matmuls per old token per step (compute that grows with t). With GQA (Part 6), $K+V$ is usually smaller than h , so caching K and V directly wins on both counts.

1.8 The mental model to keep

A notebook, one page per layer

Every token that arrives writes **one line on every page**: a label (K) and its contents (V). The ink is permanent, because causality guarantees old lines never need editing. The newest token carries a question (Q). On each page it **scans all labels**, reads a weighted blend of the contents, then writes its own line. **Without a notebook**, you'd rewrite the whole notebook from scratch every time someone new arrives, just to ask one question.



Notebook fact	Consequence	Part
grows by one line per page per token	KV memory grows linearly with context, $\times$ layers	(5)
every question reads every line	decode is limited by memory reads (bandwidth), not arithmetic	(3)
every conversation has its own notebook	GPU memory caps how many users you can serve at once	(9)

→ Wrong mental models to cross out

~~X "The KV cache stores previous outputs / tokens."~~

✓ It stores per-layer, per-head **K and V activation vectors**. Not tokens, not logits, not hidden states.

~~X "It caches the attention scores."~~

✓ Scores involve the **new** query, so they're new every step.

~~X "It makes each decoding step constant time."~~

✓ Projections and MLP become constant per step. Attention still **scans all t past tokens**.

~~X "It's shared across users."~~

✓ Within one generation it's **private**. Reuse across requests is **prefix caching** (Part 10).

~~X "It's an approximation that trades accuracy for speed."~~

✓ It's **exact**: the same math, reorganized. Outputs match the no-cache run, up to floating-point summation order.

~~X "The cache is small next to the model."~~

✓ It grows with $\text{tokens} \times \text{layers} \times \text{KV heads} \times \text{head_dim} \times 2$, per sequence. At long contexts and big batches it can exceed the weights (Part 5).

Why does it exist?

Causal attention makes every past token's K and V final, so we compute them once and read them on every later step instead of re-running the whole prefix.

Why not Q?

A query is used only by its own token at its own step. No future computation ever reads an old query.

Checkpoint · Part 1

Recall

- 1 What three vectors does each token produce at each layer, and what role does each play?
- 2 What exactly is stored in the KV cache, and at what granularity (per token? per layer? something else)?
- 3 What still grows with sequence length even when the cache is used?

Reasoning

- 4 Suppose attention were **bidirectional** (like BERT), so token 1 could see token 4. Would KV caching still be valid during generation? Why or why not?

- 5 Without a cache, why can't we save work by computing only the final token's attention row at every layer?

Practical engineering

- 6 A colleague says: "Our chatbot is slow on long conversations even with KV caching enabled, so caching must be broken." Give one reason it could be slow even if caching works perfectly.

Calculation

- 7 A 32-layer model generates 200 tokens after a single starting token. Count total token forward passes through the full stack (a) without a cache and (b) with a cache. Then count the total number of k/v vectors written to the cache across all layers by the end (one head, for simplicity).

Handwritten note

Causality makes past K and V immutable, so we compute them once and read them forever. The cache trades recomputation for memory and memory reads.

What do you think happens next?

We fed "The cat sat" one token at a time. But a real prompt is known all at once. How should the model handle a 4,000-token prompt, and what does that do to the cache before generation starts? (Part 2–3.)

PART 2 · Module A: Foundations

Attention Math

Goal of this part

Derive the KV cache from the attention equation itself, so it stops being a "trick" and becomes an algebraic consequence. Then follow every tensor's shape through prefill, the first decode step, and later decode steps.

2.1 Start from the equations

For one layer and one head, with the sequence stacked as rows of a matrix X :

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_h} + M) V$$

Symbol	Shape	Meaning
X	$[T, d]$	one row per token: the layer's input hidden states
W_Q, W_K, W_V	$[d, d_h]$ per head	learned weights, the same for every token
Q, K, V	$[T, d_h]$	one query, key and value row per token
QK^T	$[T, T]$	entry $(i, j) = q_i \cdot k_j$: how much token i cares about token j
M (causal mask)	$[T, T]$	0 where $j \leq i$, $-\infty$ where $j > i$
output	$[T, d_h]$	one output row per token

2.2 Fact 1: projections act row by row

Matrix multiplication by a weight matrix treats every row independently: row i of XW is $x_i W$. Nothing in row i depends on any other row. So when a new token arrives and X gains a row, K and V simply gain a row:

$$X_{1:t+1} = [X_{1:t}; x_{t+1}] \implies K_{1:t+1} = [K_{1:t}; x_{t+1}W_K]$$

$$V_{1:t+1} = [V_{1:t}; x_{t+1}W_V]$$

The old rows $K_{1:t}, V_{1:t}$ are untouched. Appending to the cache is literally appending a row to a matrix.

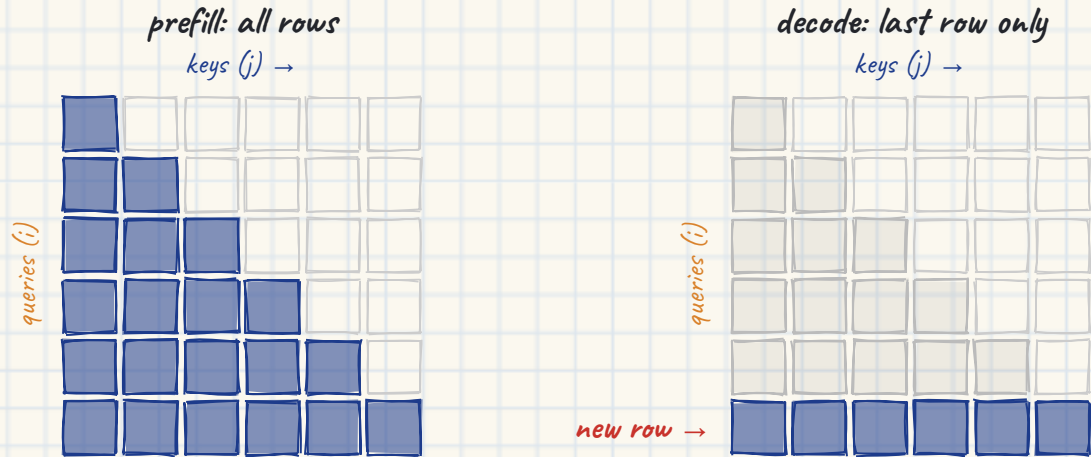
This needs $X_{1:t}$ itself not to change when token $t+1$ arrives. At layer 1 that's obvious (embeddings). At deeper layers it's the causality induction from Part 1 §1.2c.

2.3 Fact 2: each output row needs only its own query

Look at row i of the attention output. With the causal mask, the softmax for row i only sees columns $j \leq i$:

$$\text{out}_i = \text{softmax}(q_i K_{1:i}^T / \sqrt{d_h}) \cdot V_{1:i}$$

Row i needs q_i (one row of Q) and $K_{1:i}$, $V_{1:i}$ (all rows up to i). It never needs any other row of Q .



$T(T+1)/2$ scores in one parallel pass

grey rows: done on earlier steps, never recomputed

left: prefill computes the whole masked triangle. right: a decode step computes only the new bottom row.

2.4 Putting it together: generating x_{t+1}

To predict the next token we need the **last row of the last layer only** (the LM head turns it into logits). Working down the layers:

1. The last row at layer L needs $q_t^{(L)}$ and $K_{1:t}^{(L)}$, $V_{1:t}^{(L)}$.
2. Rows $1 \dots t-1$ of $K^{(L)}$, $V^{(L)}$ were computed on earlier steps and haven't changed (Fact 1 + causality) → **read them from the cache**.
3. Row t of $K^{(L)}$, $V^{(L)}$ and $q_t^{(L)}$ need only $h_t^{(L-1)}$, the **new token's** hidden state from the layer below.
4. Recurse: at every layer we need only the new token's row, plus that layer's cache.

The derivation in one line

Row-wise projections + a causal mask $\implies$ for each new token, at each layer: compute **one** q , k , v row; append k , v to the cache; attend q over the whole cache. Every other quantity is either reused (old K , V) or never needed again (old Q).

→ Exactly what must be calculated for x_{t+1} (per layer)

Computation	Uses	Status
-------------	------	--------

$h_t \rightarrow q_t, k_t, v_t$	3 small matmuls on ONE row	new, cheap
append k_t, v_t to cache	one write per layer (per KV head)	new, tiny
scores $q_t \cdot K_{1:t}^T$	reads all t cached keys	grows with t
softmax, then weights $\cdot V_{1:t}$	reads all t cached values	grows with t
output projection, residual, norm, MLP	ONE row	new, fixed cost
$K_{1:t-1}, V_{1:t-1}$	—	reused from cache

2.5 Adding heads and batch: the real tensor shapes

Real layers have H heads and process B sequences at once. The hidden size splits across heads: $d = H_q \cdot d_h$. The standard reshape:

```
# x: [B, T, d]
q = x @ W_Q
q = q.view(B, T, H_q, d_h)
q = q.transpose(1, 2)
k = (x @ W_K).view(B, T, H_kv, d_h).transpose(1, 2)
v = (x @ W_V).view(B, T, H_kv, d_h).transpose(1, 2)
scores = q @ k.transpose(-2, -1) / d_h**0.5
out = softmax(scores + mask) @ v
out = out.transpose(1, 2).reshape(B, T, H_q * d_h)
```

one hidden vector per token
 # [B, T, H_q * d_h]
 # [B, T, H_q, d_h] ← "batch, seq, heads, head_dim"
 # [B, H_q, T, d_h] ← layout used for the matmuls
 # [B, H_kv, T, d_h]
 # [B, H_kv, T, d_h]
 # [B, H, T_q, T_k]
 # [B, H, T_q, d_h]
 # back to [B, T, d], then @ W_O

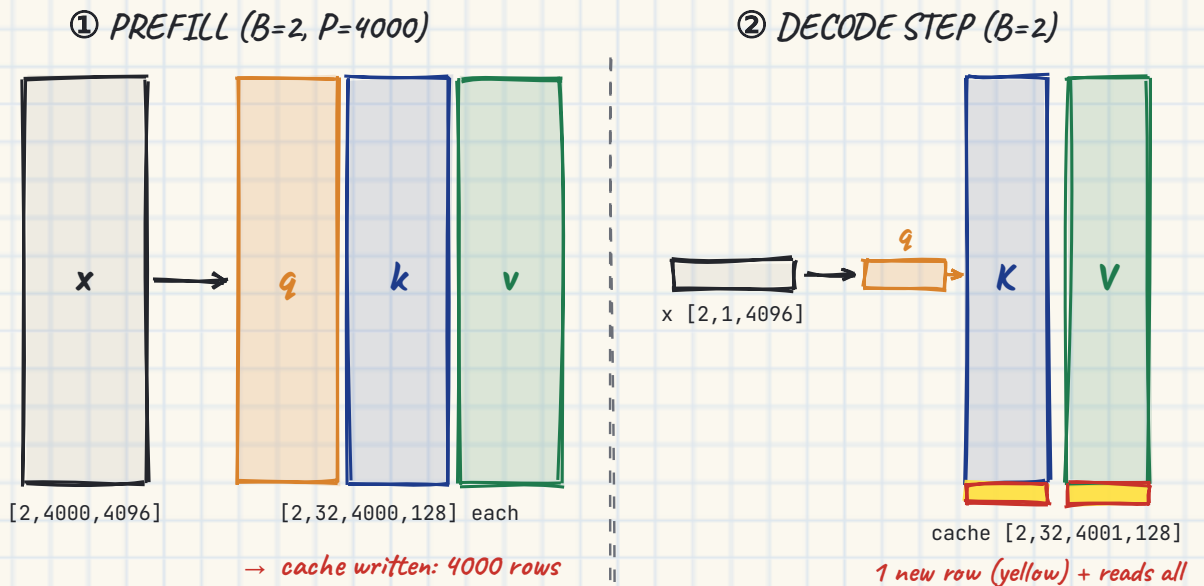
Why transpose to $[B, H, T, d_h]$? So that (B, H) become "batch" dimensions and each head's attention is an independent $[T, d_h] \times [d_h, T]$ matmul. The cache is usually stored in this layout too (Hugging Face does). Production engines use a paged layout instead (Part 8). For now $H_{kv} = H_q$. When they differ (GQA, Part 6), each K/V head serves a group of query heads.

2.6 Shapes through the three phases

Running example: **Llama-2-7B** ($L = 32, H = 32, d_h = 128, d = 4096$, vocab 32,000), batch $B = 2$, prompt $P = 4,000$ tokens.

Tensor (one layer)	① Prefill	② First decode step	③ Decode step n
input ids	[2, 4000]	[2, 1]	[2, 1]
hidden x	[2, 4000, 4096]	[2, 1, 4096]	[2, 1, 4096]
q	[2, 32, 4000, 128]	[2, 32, 1, 128]	[2, 32, 1, 128]
k, v (new)	[2, 32, 4000, 128]	[2, 32, 1, 128]	[2, 32, 1, 128]
K, V cache after write	[2, 32, 4000, 128]	[2, 32, 4001, 128]	[2, 32, 4000+n, 128]
scores	[2, 32, 4000, 4000]	[2, 32, 1, 4001]	[2, 32, 1, 4000+n]
attention out	[2, 32, 4000, 128]	[2, 32, 1, 128]	[2, 32, 1, 128]
logits (last layer)	[2, 4000, 32000]*	[2, 1, 32000]	[2, 1, 32000]

* Only the last row is needed to pick the first token. Good implementations compute logits for the last position only (in Hugging Face: logits_to_keep=1). The full [2, 4000, 32000] tensor is 256 M elements, 512 MB in FP16. Part 16 shows this is a real OOM source on a 6 GB card.



the same layer in the two regimes: a fat matrix of tokens (prefill) versus one row per sequence (decode)

→ Two numbers worth remembering

- **Prefill scores** $[2, 32, 4000, 4000] = 1.02$ billion elements, about **2 GB in FP16 for ONE layer**. That's why real kernels (FlashAttention) never materialize this matrix: they stream K/V tiles through on-chip memory and keep a running softmax.
- **Decode scores** $[2, 32, 1, 4001] \approx 256$ K elements. Tiny. But producing them reads the whole cache: $2 \times 32 \times 4001 \times 128 \times 2$ (K and V) $\times 2$ bytes $\approx$ **131 MB per layer**, 4.2 GB across 32 layers, for this batch of 2.

2.7 The shape change that explains everything after this

Prefill = matrix $\times$ matrix

$[B \cdot P, d] \times [d, d]$: each weight is loaded once and used for $B \cdot P = 8,000$ rows. Lots of arithmetic per byte loaded. **Compute-bound**. (Part 3)

Decode = (skinny) matrix $\times$ matrix

$[B, d] \times [d, d]$: each weight is loaded once and used for only $B = 2$ rows. Almost no arithmetic per byte. **Memory-bound**. (Part 3)

The KV cache turned decode from "re-run a 4,000-row GEMM" into "a 2-row GEMV-like op plus a scan of the cache". That's a huge win in FLOPs, and it hands the bottleneck to the memory system.

→ Wrong mental models to cross out

~~✗ "Decode computes the full attention matrix but with the cache."~~

✓ Decode computes one row of the matrix per sequence per head: shape $[B, H, I, t]$.

~~X "The cache stores the attention matrix QK^T ."~~

✓ It stores K and V ($[B, H_{kv}, t, d_h]$ each, per layer). QK^T depends on the new q , so it can't be reused.

~~X "Positions don't matter once cached."~~

✓ The new token's position must equal the current cache length. An off-by-one in positions silently corrupts outputs (Parts 14, 19).

Checkpoint · Part 2

Recall

- 1 What are the shapes of the attention score tensor during prefill and during a decode step?
- 2 Which algebraic property lets you compute the new token's k and v without touching old rows?
- 3 What exactly is stored per layer, and in what shape (Hugging Face layout)?

Reasoning

- 4 Suppose a model normalized its hidden states across the sequence dimension (mixing all tokens) before computing K and V . Would a KV cache still be exact? Why?
- 5 Why does going from prefill to decode turn a matrix-matrix product into (effectively) a matrix-vector product, and why does that matter for hardware?

Practical engineering

- 6 Generated text is fine for the first token, then turns to gibberish. Logging shows the layer-0 K cache has shape $[1, 32, 1, 128]$ at every step. What's the bug?

Calculation

- 7 Llama-3-8B ($L = 32$, $H_q = 32$, $H_{kv} = 8$, $d_h = 128$), batch 4, prompt 1,000. Write the shapes of q , the layer-0 K cache and the scores (a) during prefill and (b) during the 10th decode step (after its cache write).

Handwritten note

Projections work row by row, so a new token appends one row to K and V . Decode computes one row of attention against the whole cache: a thin matmul, not a fat one.

What do you think happens next?

Prefill reuses each weight for thousands of tokens. Decode reuses it for only B . If a GPU can do ~ 150 FLOPs in the time it moves one byte, which phase leaves the math units idle?

PART 3 · Module A: Foundations

Prefill vs Decode

Goal of this part

See inference as two very different workloads sharing one model. Be able to say, with numbers, why prefill saturates the GPU's math units while decode leaves them idle and waits on memory, and how that shapes TTFT, ITL and throughput.

3.1 Two phases, one request

Prefill (a.k.a. prompt processing)

All P prompt tokens go through the model **in one forward pass**. Output: the full KV cache for the prompt at every layer, plus logits for the last position → the **first generated token**.

Decode (a.k.a. generation)

One new token per sequence per step. Each step: feed the last token, append its KV, attend over the cache, sample the next token. Repeat until EOS or max tokens.

→ *Why prefill is parallel*

All prompt tokens are **known in advance**. The causal mask lets us compute every row of the attention triangle at once. Token 3's output doesn't need token 3's output from this layer, only rows 1–3 of K and V, which we can compute for all tokens simultaneously. It's the same trick as training with teacher forcing.

→ *Why decode is sequential*

Token $t+1$ is chosen by **sampling from the logits of token t** . You can't compute step $t+2$ until you know what step $t+1$ produced. That's a true data dependency, and no amount of hardware parallelism can remove it within one sequence. (Speculative decoding, Part 13, cheats by guessing several tokens and verifying them in parallel.)

3.2 Arithmetic intensity: the one idea you need

Every GPU kernel moves bytes from memory and does arithmetic on them. The ratio decides which resource runs out first:

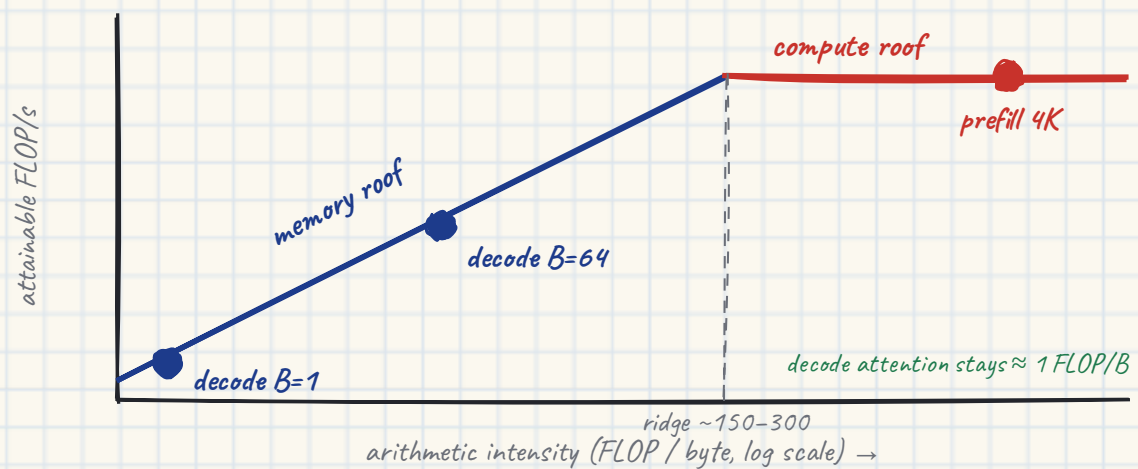
$$\text{arithmetic intensity} = \text{FLOPs performed} \div \text{bytes moved from memory} \\ [\text{FLOP/byte}]$$

The GPU has its own ratio, the **ridge point** = peak FLOP/s ÷ peak bytes/s:

GPU	Dense BF16 peak	Memory bandwidth	Ridge point
A100 80GB SXM	312 TFLOP/s	~2.0 TB/s	≈ 153 FLOP/B
H100 SXM	~989 TFLOP/s	~3.35 TB/s	≈ 295 FLOP/B
RTX 4050 laptop	tens of TFLOP/s	~192 GB/s	also >> 1

Rule

If a kernel's intensity is **below** the ridge point, it's **memory-bandwidth-bound**: math units wait for data. **Above** it, it's **compute-bound**. Modern GPUs need on the order of **100–300 FLOPs per byte** to keep their math units busy.



the roofline: attainable FLOP/s rises with intensity until it hits the compute ceiling

→ Intensity of a linear layer (the bulk of the model)

A weight matrix of shape $[d, d]$ in FP16 is $2d^2$ bytes. Multiplying it by N token rows costs $2 \cdot N \cdot d^2$ FLOPs. So the intensity $\approx N$ FLOP/byte, where N is the number of token rows sharing that weight read.

Situation	Rows N sharing each weight load	Intensity	Verdict (A100, ridge 153)
Prefill, $1 \times 4,000$ -token prompt	4,000	~4,000	compute-bound
Decode, batch 1	1	~1	memory-bound (150x under)
Decode, batch 64	64	~64	still memory-bound
Decode, batch 256	256	~256	crosses the ridge for linear layers

→ Intensity of decode attention (the KV part)

Here batching **doesn't help**. Each sequence has its **own** cache, so the KV bytes you load serve only that sequence's query. Per cached element: one multiply-add for the score, one for the weighted sum → roughly **1 FLOP per byte** for multi-head attention, whatever the batch size. (With GQA, each KV head serves g query heads, so intensity rises to about g . Part 6.)

So decode is memory-bound twice over

Weights are memory-bound at small batch. KV reads are memory-bound at any batch. As contexts get longer and batches get bigger, the KV traffic becomes the dominant cost. This is the root of Parts 5–13.

3.3 How the KV cache changes decode performance

Decode step without cache	Decode step with cache
re-runs a prefill over all t tokens: a big GEMM, compute-bound, cost grows ~linearly (plus quadratic attention) in t	processes 1 token: tiny GEMMs + a cache scan, memory-bound
$\text{time/step} \approx \text{FLOPs}(t) \div \text{compute throughput}$	$\text{time/step} \approx (\text{weight bytes} + \text{KV bytes}(t)) \div \text{bandwidth}$
gets slower fast as the conversation grows	gets slower slowly: only the KV-bytes term grows

$$\text{decode step time} \approx (W + B \cdot t \cdot kv_{\text{tok}}) \div BW$$

W = weight bytes · B = batch · t = context length · kv_{tok} = KV bytes per token · BW = memory bandwidth

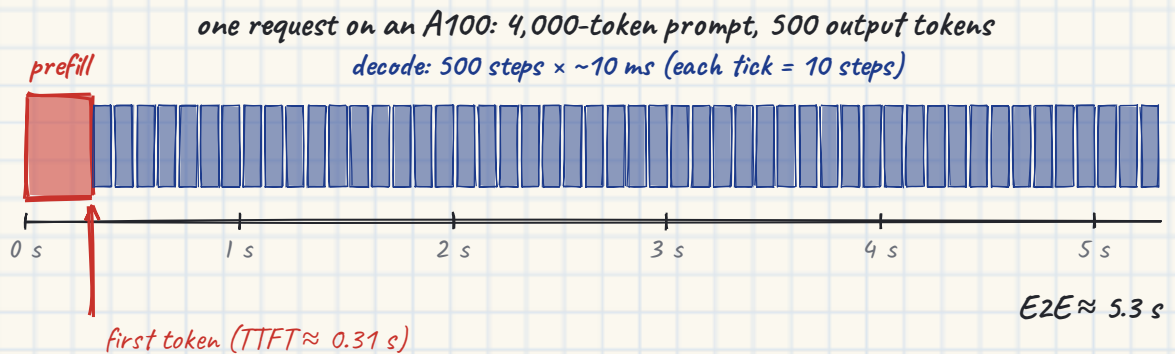
3.4 The latency metrics

Metric	Definition	Dominated by
TTFT · time to first token	request arrives → first token returned. Queueing + tokenization + prefill + one sample.	prompt length, queue depth, compute
ITL / TPOT · inter-token latency	time between consecutive output tokens = one decode step (as seen by the user)	memory bandwidth, batch size, context length
E2E latency	$\approx \text{TTFT} + (N_{\text{out}} - 1) \cdot \text{ITL}$	usually decode, for long outputs
tokens/s per user	$\approx 1 / \text{ITL}$: what a single stream feels like	ITL
throughput (system tokens/s)	total output tokens/s across all sequences = B / ITL	batch size × per-step efficiency

Streaming UIs feel fast when TTFT is low and ITL is below human reading speed (~5–10 tokens/s is readable, 20+ feels instant). Batch jobs (summarizing 10,000 documents) only care about throughput.

3.5 Timeline: 4,000-token prompt → 500 generated tokens

Llama-2-7B, FP16, one A100 80GB, batch 1. Weights $W \approx 13.5$ GB. KV per token = 0.5 MiB (Part 5 derives it).



compute-bound (tensor cores busy) → bandwidth-bound (streaming weights + KV)

Phase	Work	Estimate	What the GPU is doing
Prefill	$\approx 2 \cdot P \cdot \text{params} + \text{attention}$ $= 2 \times 6.74e9 \times 4000 + 4.2e12 \approx 5.8 \times 10^{13}$ FLOPs	at ~60% of 312 TFLOP/s → ≈ 0.31 s (TTFT)	tensor cores saturated with big GEMMs; FlashAttention tiles through a 4000×4000 triangle; writes 4,000 × 0.5 MiB ≈ 2 GiB of KV
Each decode step	read 13.5 GB weights + (4,000...4,500) × 0.5 MiB $\approx 2.1\text{--}2.4$ GB KV $\approx 15.6\text{--}15.8$ GB	at 2.0 TB/s: 7.6–7.8 ms ideal; ~10 ms realistic	streaming weights and cache from HBM; tensor cores mostly idle; "GPU utilization" still reads ~100%
All 500 steps	$\approx 7.9 \times 10^{12}$ FLOPs total	$\approx 4\text{--}5$ s	the same memory-streaming loop, 500 times

The punchline

Prefill does ~88% of the FLOPs but takes ~6% of the time. Decode does ~12% of the FLOPs and takes ~94% of the time. FLOPs are the wrong way to think about decode. Bytes are the right way.

"GPU utilization 100%" in nvidia-smi only means a kernel was running. It says nothing about whether the math units or the memory bus is saturated. During decode the tensor cores may be <5% busy while "utilization" shows 100%. Part 17 shows how to measure properly.

3.6 Throughput vs latency: batching in numbers

Batch B sequences into each decode step. The weights are read **once** per step and shared, but every sequence brings its **own** KV. Using the formula above (A100, 2.04 TB/s, context $\approx 4,250$, ideal bandwidth):

Batch B	Llama-2-7B (MHA): bytes/step	ITL	system tok/s	Llama-3-8B (GQA): ITL	system tok/s
1	15.7 GB	7.7 ms	130	8.1 ms	123
8	31.3 GB	15.4 ms	521	10.1 ms	795
16	49.1 GB	24.1 ms	664	12.2 ms	1,306
32	84.8 GB	41.6 ms	770*	16.6 ms	1,926

* 32 sequences $\times$ 4,500 tokens $\times$ 0.5 MiB $\approx$ 75 GB of KV, which doesn't even fit next to the weights on an 80 GB card. That row is hypothetical. Ideal-bandwidth numbers; real systems land at maybe 60–80% of these.

- **Throughput rises with batch** because the weight read is amortized.
- **Latency per user (ITL) also rises**, because each step must read every sequence's KV.
- For the MHA model the KV term dominates quickly, so gains flatten. For the GQA model (4x less KV per token) batching keeps paying off. **This is why KV size decides serving economics.**

→ Wrong mental models to cross out

~~✗ "The KV cache reduces time to first token."~~

✓ Within one request, prefill creates the cache; it doesn't benefit from it. TTFT improves only when a cache already exists from an earlier request (prefix caching, Part 10).

~~✗ "Decode is slow because attention is $O(n^2)$."~~

✓ Per decode step, attention is $O(t)$. Decode is slow because every step streams all weights plus the whole cache from memory.

~~✗ "A faster GPU (more TFLOPS) speeds up generation proportionally."~~

✓ Decode tracks memory bandwidth, not TFLOPS. Prefill tracks TFLOPS.

Checkpoint · Part 3

Recall

- 1 Define TTFT and ITL. Which phase dominates each?

- 2 What is arithmetic intensity, and what's the approximate intensity of a linear layer during decode with batch B ?
- 3 Why can't decode be parallelized across time steps within one sequence?

Reasoning

- 4 Batching improves the intensity of the linear layers during decode but not of decode attention. Explain why.
- 5 You upgrade from a GPU with 2 TB/s and 300 TFLOP/s to one with 2 TB/s and 600 TFLOP/s. Which metric (TTFT or ITL) improves, and roughly by how much?

Practical engineering

- 6 Your service has great throughput but users complain responses "stutter". Metrics show ITL $p50 = 45$ ms, $p99 = 400$ ms. What phase interaction could cause the spikes? (Hint: new requests arriving.)

Calculation

- 7 A 13B model (FP16 weights = 26 GB, KV = 0.78 MiB/token) runs on a GPU with 3.35 TB/s. Estimate the ideal ITL at batch 1 with 2,000 tokens of context, and at batch 8 with 2,000 tokens each.

Handwritten note

Prefill is a compute problem (FLOPs). Decode is a memory problem (bytes): every step reads all the weights plus every sequence's whole cache.

What do you think happens next?

We keep saying "the cache removes recomputation". In Part 4 we'll count exactly what disappears and what still grows, with no Big-O hand-waving.

PART 4 · Module A: Foundations

Complexity

Goal of this part

Replace "KV caching makes attention linear" (wrong) with a precise account. We separate five costs: attention scores, KV projections (and the rest of the linear layers), sequence growth, memory movement, and total generation cost. For each: what the cache changes, and what still grows with context.

4.1 Set-up and honest cost units

Model with P parameters, L layers, width $d = H_q \cdot d_h$. The prompt has n tokens and we generate T more. Position t means "t tokens of context exist".

Cost for ONE token at position t	FLOPs	Why
all linear layers (Q, K, V, O projections, MLP, LM head)	$\approx 2P$	each parameter does one multiply-add per token
K and V projections alone	$\approx 4 \cdot L \cdot d \cdot H_{kv} d_h$	two $[d \rightarrow H_{kv} d_h]$ matmuls per layer
attention scores $q \cdot K^T$	$\approx 2 \cdot L \cdot d \cdot t$	t dot products of length d_h for every head
weighted sum $a \cdot V$	$\approx 2 \cdot L \cdot d \cdot t$	same size as the scores

So one token at position t costs $\approx 2P + 4Ldt$ FLOPs. For Llama-2-7B at $t = 4,000$: $2P \approx 1.35 \times 10^{10}$, $4Ldt \approx 2.1 \times 10^9$. The linear layers still dominate at this length. The crossover (where attention FLOPs equal linear FLOPs) is $t \approx P/(2Ld) \approx 26K$ tokens for this model.

4.2 Five costs, carefully separated

Cost	Step t , no cache	Step t , with cache	Whole generation (n prompt + T new)
① attention scores (+ weighted sum)	full triangle over t tokens: $\propto t^2$	one row: $\propto t$	no cache: $\sum t^2 \propto (n+T)^3$ · cache: $\sum t \propto (n+T)^2$
② KV projections (and all other linear work)	for all t tokens: $\propto t$	for 1 token: constant	no cache: $\propto (n+T)^2$ · cache: $\propto (n+T)$
③ sequence growth	input grows by 1 per step	cache grows by 1 row per layer per step	both: context reaches $n+T$

④ memory movement	weights once + recomputed activations	weights once + whole cache read : $W + t \cdot kv_{tok}$	cache: $T \cdot W + kv_{tok} \cdot (nT + T^2/2)$
⑤ memory held	transient activations only	cache: $t \cdot kv_{tok}$ persistent	peaks at $(n+T) \cdot kv_{tok}$ per sequence

What people mean by "the cache removes repeated attention computation"

Precisely: at step t , the **scores for old query rows** (rows $1 \dots t-1$ of the triangle) and the **KN projections of old tokens** are no longer recomputed. That turns per-step attention from $\propto t^2$ into $\propto t$, and per-step linear work from $\propto t$ into a constant. It does **not** make attention "linear overall": over a whole generation attention is still quadratic in length.

4.3 Worked totals: 4,000-token prompt + 500 new tokens

Llama-2-7B ($P = 6.74 \times 10^9$, $L = 32$, $d = 4,096$). "No cache" means every step re-runs the full sequence.

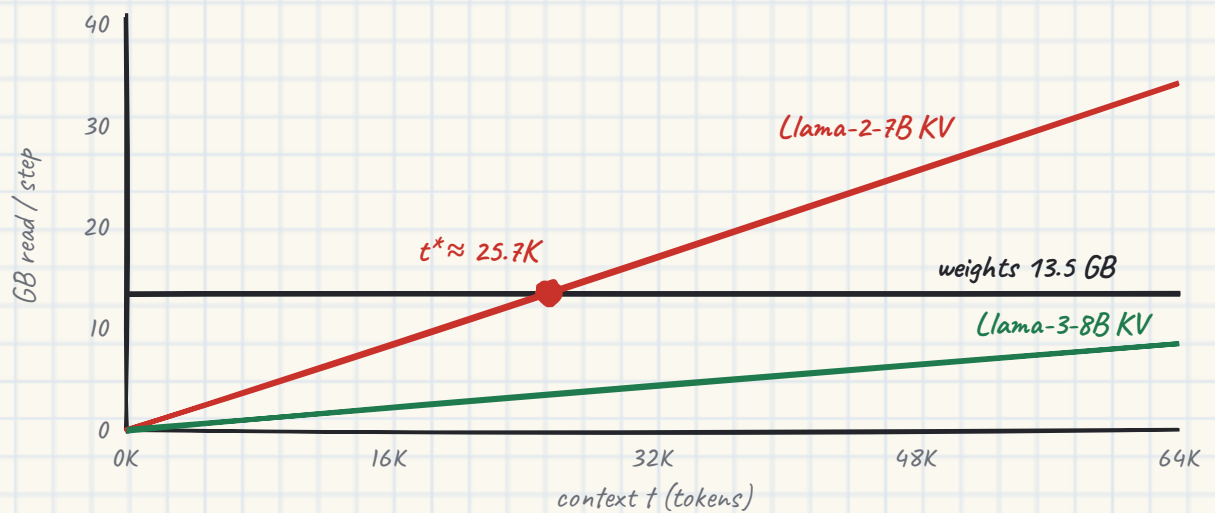
Quantity	No cache	With cache	Ratio
total FLOPs (prefill + 500 steps)	$\approx 3.1 \times 10^{16}$	$\approx 6.6 \times 10^{13}$	$\approx 470\times$
... of which decode linear layers	$\approx 2.8 \times 10^{16}$	6.7×10^{12}	$\sim 4,250\times$
... of which decode attention	$\approx 2.3 \times 10^{15}$	1.1×10^{12}	$\sim 2,000\times$
bytes of weights read during decode	≈ 6.7 TB	≈ 6.7 TB	$1\times$ (!)
bytes of KV read during decode	0 (none stored)	≈ 1.1 TB	new cost
peak KV memory held	0	≈ 2.2 GiB	new cost

Computed from the per-token formulas above: no-cache step t costs $2P \cdot t + 2Ldt^2$; cached step costs $2P + 4Ldt$. Weight bytes read are the same because both approaches still run one forward pass per step: the weights are streamed once per step either way.

Read the last three rows twice

The cache eliminated $\sim 99.8\%$ of the FLOPs, yet decode's **weight traffic didn't change at all**, and we **added** 1.1 TB of KV reads and a 2.2 GiB resident cache. That's why cached decode ends up bandwidth-bound (Part 3) and why memory capacity becomes the serving limit (Parts 5, 9).

4.4 What still grows with context (per decode step)



bytes read per decode step (batch 1) versus context length. The weight line is flat; the KV line grows.

Per step the weights cost a **constant** W bytes while the cache costs $t \cdot kv_{tok}$. They're equal at the **crossover context**:

$$t^* = W \div kv_{tok} \rightarrow \text{Llama-2-7B: } 13.5 \text{ GB} \div 0.5 \text{ MiB} \approx \mathbf{25.7K \text{ tokens}} \quad \cdot \quad \text{Llama-3-8B: } 16.1 \text{ GB} \div 128 \text{ KiB} \approx \mathbf{122.5K \text{ tokens}}$$

Beyond t^* , a single sequence's cache costs more to read than the entire model. With a batch of B sequences the crossover drops to t^*/B . At batch 16, Llama-2-7B's KV already dominates at $\sim 1.6K$ tokens.

4.5 Big-O statements: right and wrong

Misleading shortcut	Precise version
"KV cache makes attention $O(n)$ instead of $O(n^2)$."	Per decode step , attention goes from $O(t^2)$ (recompute the triangle) to $O(t)$. Over a whole sequence, attention is still $O(n^2)$ total with the cache, and $O(n^3)$ without.
"KV cache makes generation $O(n)$."	Linear-layer work becomes $O(n)$ total (was $O(n^2)$). Attention work stays $O(n^2)$ total. For short contexts the linear part dominates, which is why it feels linear.
"Decode is $O(1)$ per token with a cache."	Linear layers are $O(1)$ per token. Attention and memory traffic per token are $O(t)$.
"The cache trades time for nothing."	It trades compute for $O(n)$ memory per sequence and $O(n)$ memory traffic per step.

Interview phrasing

"With a KV cache, each decode step does constant linear-layer work and attention linear in the current context, instead of re-running the whole prefix. Total attention over a generation is still quadratic, and we now pay memory proportional to context, plus a full read of that memory every step, which is why decode becomes bandwidth-bound."

→ A note on prefill complexity

Prefill is unchanged by the cache: it's one pass over n tokens costing $\approx 2P \cdot n + 2Ldn^2$ FLOPs (causal). The quadratic term takes over past the same crossover ($\sim 26K$ tokens for Llama-2-7B), which is why very long prompts are slow to start (Part 12). FlashAttention reduces prefill memory from $O(n^2)$ to $O(n)$, but **not** its FLOPs.

Checkpoint · Part 4

Recall

- 1 Per decode step, how do attention FLOPs and linear-layer FLOPs scale with context t , with and without a cache?
- 2 What is the crossover context t^* , and what's its formula?
- 3 Which cost does the KV cache add?

Reasoning

- 4 Why are the weight bytes read during decode the same with and without a cache, even though FLOPs drop by $\sim 470\times$?
- 5 Does FlashAttention change the asymptotic FLOPs of prefill? What does it change?

Practical engineering

- 6 A teammate benchmarks decode at 512-token contexts, sees flat per-token latency, and concludes "latency doesn't depend on context length". What will happen at 64K, and why didn't the benchmark show it?

Calculation

- 7 For Llama-3-8B (weights 16.1 GB, KV 128 KiB/token), at batch 8, find the context length where KV bytes read per step equal weight bytes read per step.

 Handwritten note

The cache makes per-step linear work constant and per-step attention linear, but total attention stays quadratic, and every step still reads the weights plus the whole growing cache.

What do you think happens next?

We've been saying "0.5 MiB per token" for Llama-2-7B. Where does that number come from? Before turning the page, try to derive it from $L = 32$, $H_{kv} = 32$, $d_h = 128$, FP16.

PART 5 · Module B: Memory

KV Cache Memory

Goal of this part

Derive the KV memory formula from scratch (no memorizing), compute it for real models and context lengths, and understand why a cache of "just activations" can outgrow the model itself.

5.1 Derive it by building up one number at a time

Start with the smallest thing the cache stores and multiply outward. Each step answers "how many of the previous thing are there?"

#	Thing	How many	Llama-2-7B (FP16)
1	one number (element)	b bytes	2 B
2	one key vector for one head	$\times d_h$ elements	$128 \times 2 = 256$ B
3	keys for one token, one layer	$\times H_{kv}$ heads	$32 \times 256 = 8$ KiB
4	keys and values	$\times 2$	16 KiB
5	... for every layer	$\times L$	32×16 KiB = 512 KiB / token
6	... for every token in the sequence	$\times T$	4,096 tokens → 2 GiB
7	... for every sequence in the batch	$\times B$	batch 8 → 16 GiB

The formula (now you've built it)

$$\text{KV bytes} = 2 \times L \times H_{kv} \times d_h \times b \times T \times B$$

the first five factors are a **per-token constant** for a given model and dtype: $kv_{tok} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$

Why H_{kv} and not H_q ? Because only K/V heads are stored. For older models (MHA) they're equal. For GQA/MQA models H_{kv} is smaller, and that's the whole point of Part 6. Also: $d = H_q \cdot d_h$, so for MHA, $kv_{tok} = 2 \cdot L \cdot d \cdot b$ ("two vectors of width d per layer per token").

Try first (solution below)

Mistral-7B: $L = 32$, $H_q = 32$, $H_{kv} = 8$, $d_h = 128$. Compute kv_{tok} in FP16, then the cache for one 32K-token sequence.

Solution. $2 \times 32 \times 8 \times 128 \times 2 = 131,072$ B = **128 KiB/token**. $\times 32,768$ tokens = 4 GiB. Same shape as Llama-3-8B. Four times less than Llama-2-7B, purely because of H_{kv} .

5.2 Per-token cost for real models

Model	L	H_q / H_{kv}	d_h	kv_{tok} (FP16)	weights (FP16)
Llama-2-7B (MHA)	32	32 / 32	128	512 KiB	13.5 GB
Llama-2-13B (MHA)	40	40 / 40	128	800 KiB	26.0 GB
Llama-2/3-70B (GQA)	80	64 / 8	128	320 KiB	~140 GB
... the same 70B if it were MHA	80	64 / 64	128	2,560 KiB	~140 GB
Llama-3-8B / Mistral-7B (GQA)	32	32 / 8	128	128 KiB	16.1 / 14.5 GB
Qwen2.5-1.5B (GQA)	28	12 / 2	128	28 KiB	3.1 GB
Qwen2.5-0.5B (GQA)	24	14 / 2	64	12 KiB	1.0 GB
GPT-2 small (MHA)	12	12 / 12	64	36 KiB	0.25 GB

Configs as published on the models' Hugging Face pages. Always read `config.json` (`num_hidden_layers`, `num_key_value_heads`, `head_dim` or `hidden_size/num_attention_heads`) rather than trusting a table.

5.3 The big table: one sequence, batch 1 (GiB)

Try first

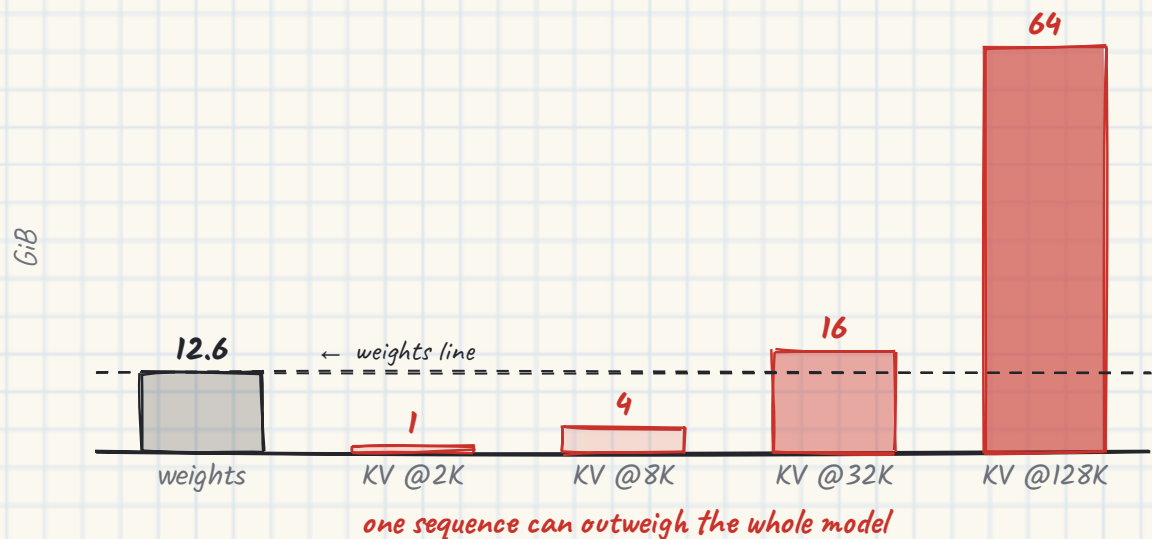
Before reading the table, compute the 13B model at 32K in FP16 in your head. Hint: 800 KiB × 32K.

Model	Precision (b)	2K	8K	32K	128K
7B	FP32 (4)	2.0	8.0	32.0	128.0
Llama-2-7B, MHA	FP16/BF16 (2)	1.0	4.0	16.0	64.0
	FP8 / INT8 (1)	0.5	2.0	8.0	32.0
	INT4 (0.5)	0.25	1.0	4.0	16.0
13B	FP32	3.1	12.5	50.0	200.0
Llama-2-13B, MHA	FP16/BF16	1.6	6.25	25.0	100.0
	FP8 / INT8	0.8	3.1	12.5	50.0
	INT4	0.4	1.6	6.25	25.0
70B	FP32	1.25	5.0	20.0	80.0
Llama-2/3-70B, GQA-8	FP16/BF16	0.6	2.5	10.0	40.0
	FP8 / INT8	0.3	1.25	5.0	20.0
	INT4	0.16	0.6	2.5	10.0

$K = 1,024$ tokens. Llama-2 was trained for 4K contexts, so its 32K/128K columns are architectural "what-ifs" that show the scaling. Nobody stores KV in FP32 in practice; it's there to show the ladder. INT8/INT4 also store small scale factors (e.g. one FP16 scale per group of 32–128 values), adding a few percent.

Notice three things

1. The **70B** model's cache is **smaller per token** than the **7B's**. Architecture (H_{kv}) beats parameter count.
2. 7B at 32K in FP16 (16 GiB) already exceeds the 7B's own weights (12.6 GiB). One sequence.
3. Halving precision halves memory exactly: the formula is linear in b .



Llama-2-7B in FP16: weights (fixed) vs KV cache for ONE sequence at growing context

5.4 Why the cache gets so big when the weights don't change

Weights

Shared. One copy serves every token of every request. Size is fixed: $P \times \text{bytes per parameter}$. Loaded once at startup.

KV cache

Per token, per request. Activation state that must persist for the lifetime of the sequence. Size = $kv_{tok} \times (\text{tokens in flight across all requests})$.

So the question "how big is the KV cache?" really means "how many tokens are in flight?" A server with 64 active chats averaging 4K tokens has 256K tokens in flight: 128 GiB for Llama-2-7B, 32 GiB for Llama-3-8B, all on top of the weights. Weights are the entry fee. The cache is the per-customer cost.

5.5 From memory to "how many users fit?"

$$\text{tokens that fit} = \left(\text{GPU memory} \times \text{utilization} - \text{weights} - \text{activations/workspace} \right) \div kv_{tok}$$

Try first

A100 80 GB, 90% usable, Llama-2-7B FP16 (13.5 GB weights), ~3 GB for activations and workspace. How many 2K-token conversations fit? 8K?

Solution. $0.9 \times 80 - 13.5 - 3 \approx 55.5$ GB for KV. $55.5 \times 10^9 \div 524,288 \approx 105,900$ tokens. At 2K each: ~52 conversations. At 8K: ~13. The same GPU with Llama-3-8B (128 KiB/token) holds ~404K tokens: ~197 at 2K, ~49 at 8K. Part 9 turns this into the scheduler's "max concurrent sequences".

Units trap: GPU marketing uses GB (10^9), PyTorch reports GiB (2^{30}). 80 "GB" $\approx$ 74.5 GiB. Mixing them gives a 7% error in capacity plans, which at the edge of OOM is the difference between working and crashing.

→ Wrong mental models to cross out

~~✗ "Bigger model → bigger KV cache."~~

✓ KV per token depends on $L \times H_{kv} \times d_m$, not parameter count. Llama-2-70B (GQA) stores less per token than Llama-2-7B (MHA).

~~✗ "The KV formula uses the number of attention heads."~~

✓ It uses the number of **key/value** heads.

~~✗ "Memory is only a problem at 128K contexts."~~

✓ Memory is a function of **total tokens in flight**. 64 users $\times$ 2K = 128K tokens too.

Checkpoint · Part 5

Recall

- 1 Write the KV memory formula and explain each factor in one phrase.
- 2 What is kv_{tok} for Llama-2-7B in FP16, and how do you get it?
- 3 Why is the factor 2 in the formula?

Reasoning

- 4 Explain why Llama-2-70B has a smaller per-token cache than Llama-2-13B despite being 5x larger.
- 5 A colleague says "our 7B model only needs a 16 GB GPU". Under what workloads is that true, and when is it false?

Practical engineering

- 6 Your service OOMs only on Mondays at 10 am, never in load tests. The load test uses 512-token prompts. Form a hypothesis and say what you'd measure.

Calculation

- 7 Qwen2.5-1.5B (L 28, H_{kv} 2, d_h 128) in FP16 on your RTX 4050 (6 GB). Assume 90% usable, 3.1 GB weights, 0.8 GB overhead. How many tokens of KV fit? How many 4K-token sequences? What changes with an 8-bit KV cache?

Handwritten note

$KV \text{ bytes} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$ per token, times every token in flight. Weights are the entry fee; the cache is the per-customer cost.

What do you think happens next?

The single biggest lever in the formula was H_{kv} . What would you lose if many query heads shared one KV head?

PART 6 · Module B: Memory

MHA, MQA and GQA

Goal of this part

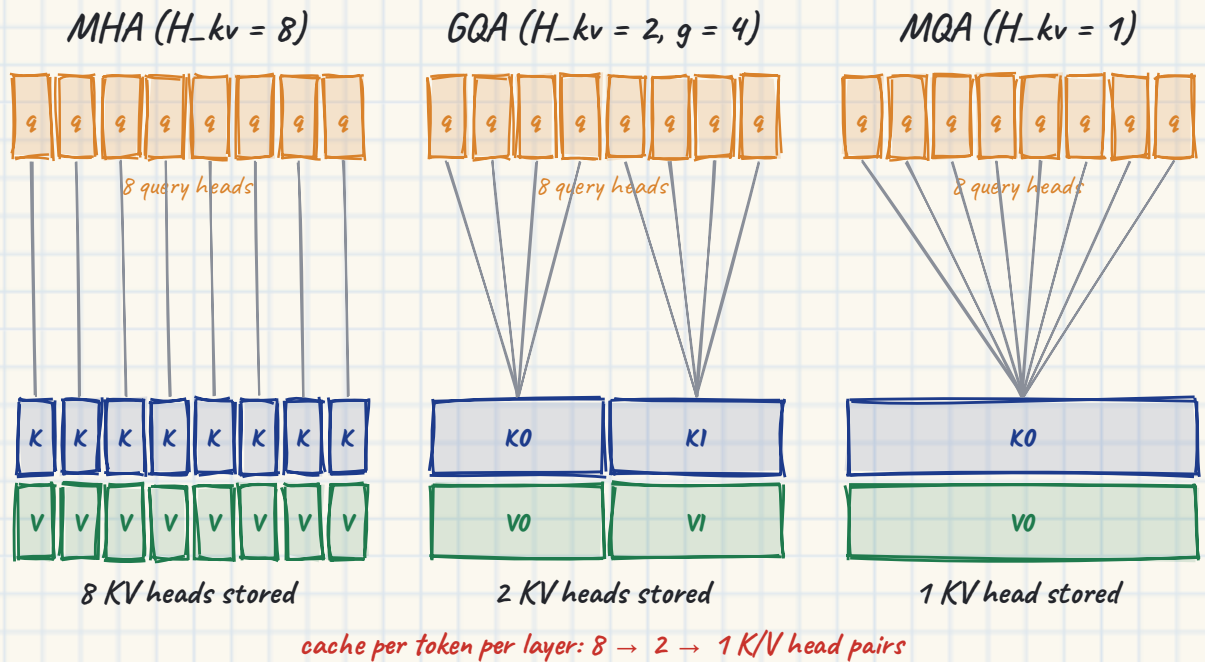
Understand the three ways to arrange attention heads, purely through their effect on the KV cache: how many KV heads are stored, how the shapes change, how much memory and bandwidth you save, and what it costs.

Problem → Naive → Bottleneck → Optimization → Trade-off

Problem	Serve long contexts and big batches; decode reads the whole cache every step.
Naive	Multi-head attention: every query head has its own K and V head.
Bottleneck	KV memory and KV bandwidth scale with H_q , often 32–64 heads' worth per layer per token.
Optimization	Let groups of query heads share one KV head (MQA: all share one; GQA: groups share).
Trade-off	Less expressive attention → possible quality loss (large for MQA, small for GQA), and it must be decided at training time.

6.1 Heads, one more time

A layer with H_q query heads runs H_q attention computations in parallel, each in its own d_h -dimensional subspace. Different heads learn different "questions" (syntax, coreference, position...). The design choice is: **how many distinct KV heads do those questions get to look at?**



8 query heads in each design. Only the blue/green K/V heads are stored in the cache.

Design	num_attention_heads (H_q)	num_key_value_heads (H_{kv})	group size $g = H_q/H_{kv}$	KV vs MHA
MHA multi-head	H	H	1	1x
GQA grouped-query	H	G ($1 < G < H$)	H/G	$1/g$
MQA multi-query	H	1	H	$1/H$

In a Hugging Face config.json these are literally the fields num_attention_heads and num_key_value_heads. If the second is missing, the model is MHA.

6.2 Tensor shapes

```
# Llama-3-8B:  $H_q = 32, H_{kv} = 8, g = 4, d_h = 128$ . Decode step, batch  $B$ , context  $t$ .
q      : [B, 32, 1, 128]      # 32 query heads, as before
K_cache : [B, 8, t, 128]      # only 8 K heads stored ← 4x smaller than MHA
V_cache : [B, 8, t, 128]
# compute: query head  $i$  uses KV head  $i // g$  (heads 0-3 → kv 0, 4-7 → kv 1, ...)
K_exp = K_cache.repeat_interleave(4, dim=1) # [B, 32, t, 128] (naive: materializes a copy!)
scores = q @ K_exp.transpose(-2, -1)        # [B, 32, 1, t]
```

Good kernels never materialize K_{exp} . They index the right KV head ($kv = head // g$) and load each K/V tile *once* for all g query heads of the group. The naive repeat_interleave in reference code throws away the bandwidth saving at runtime, though not the storage saving.

6.3 How much does it save? Calculate.

Try first

A 32-layer model, $H_q = 32$, $d_h = 128$, FP16, context 32K, batch 8. Compute the KV cache for MHA ($H_{kv} 32$), GQA-8, GQA-4 and MQA.

Design	H_{kv}	kv_{tok}	1 seq × 32K	8 seq × 32K	saving
MHA	32	512 KiB	16 GiB	128 GiB	—
GQA-8	8	128 KiB	4 GiB	32 GiB	4×
GQA-4	4	64 KiB	2 GiB	16 GiB	8×
MQA	1	16 KiB	0.5 GiB	4 GiB	32×

The famous real-world case: **Llama-2-70B** uses 64 query heads and 8 KV heads. As MHA it would need **2,560 KiB** per token; with GQA-8 it needs **320 KiB**. At 4K context that's 10 GiB → 1.25 GiB per sequence. Without GQA, a 70B model would be very hard to serve at useful batch sizes.

6.4 Why modern LLMs mostly use GQA

Concern	MQA	GQA
KV memory & bandwidth	best ($1/H$)	great ($1/g$, typically 4–8×
quality	noticeable drops reported; all heads share one "view"	close to MHA in the original GQA paper (Ainslie et al., 2023)
training	reported instabilities	stable; can even be "uptrained" from an MHA checkpoint with a small fraction of the original compute
tensor parallelism	1 KV head can't be split across GPUs: it's replicated on every GPU, wasting memory	with $H_{kv} \geq$ TP degree (e.g. 8 KV heads on 8 GPUs), each GPU owns its own KV heads cleanly
decode attention intensity	~H FLOPs/byte	~g FLOPs/byte (each loaded K/V serves g queries)

GQA sits on the sweet spot: most of MQA's savings, most of MHA's quality, and it cooperates with multi-GPU serving. Llama-2-70B, Llama-3, Mistral, Qwen2/2.5 and many others use it. Falcon-7B and PaLM are well-known MQA models.

Beyond GQA: DeepSeek-V2/V3 use **Multi-head Latent Attention (MLA)**, which caches one small compressed latent vector per token per layer and reconstructs per-head K/V from it. It shrinks the cache further while keeping per-head diversity. See Part 13 (cache compression).

→ **Wrong mental models to cross out**~~X "GQA reduces the number of attention heads."~~✓ It keeps all H_q query heads; it reduces the number of **KN** heads they read from.~~X "GQA makes attention compute $g\times$ cheaper."~~✓ Score FLOPs are unchanged (still H_q heads $\times t$ keys). What drops $g\times$ is **KV memory and KV bytes read**, which is what decode is bound by.~~X "You can switch a model to GQA at inference time."~~

✓ It's an architecture decision. Converting needs retraining or uptraining.

Checkpoint · Part 6**Recall**

- 1 Define MHA, MQA and GQA in terms of H_q and H_{kv} .
- 2 Which config fields tell you a Hugging Face model uses GQA?
- 3 By what factor does GQA with group size g reduce KV memory?

Reasoning

- 4 Why does MQA interact badly with tensor parallelism, while GQA-8 on 8 GPUs doesn't?
- 5 Why does GQA raise the arithmetic intensity of decode attention, and why does that matter?

Practical engineering

- 6 You profile a GQA model and find KV bytes read per step equal to what MHA would read. The model config is correct. Where would you look?

Calculation

- 7 A 48-layer model with $H_q = 48$, $d_h = 128$ serves 16 sequences of 8K tokens in BF16. Compute the KV cache as MHA and as GQA with $H_{kv} = 8$. How many more sequences fit in the memory you saved?

 Handwritten note

The cache stores **KN** heads, not query heads. GQA shares each **KN** head across g query heads: $g\times$ less KV memory and bandwidth, near-MHA quality.

What do you think happens next?

We know how many bytes. Now: where exactly do they live on the GPU, and what happens when 100 users with different, unknown lengths all want some?

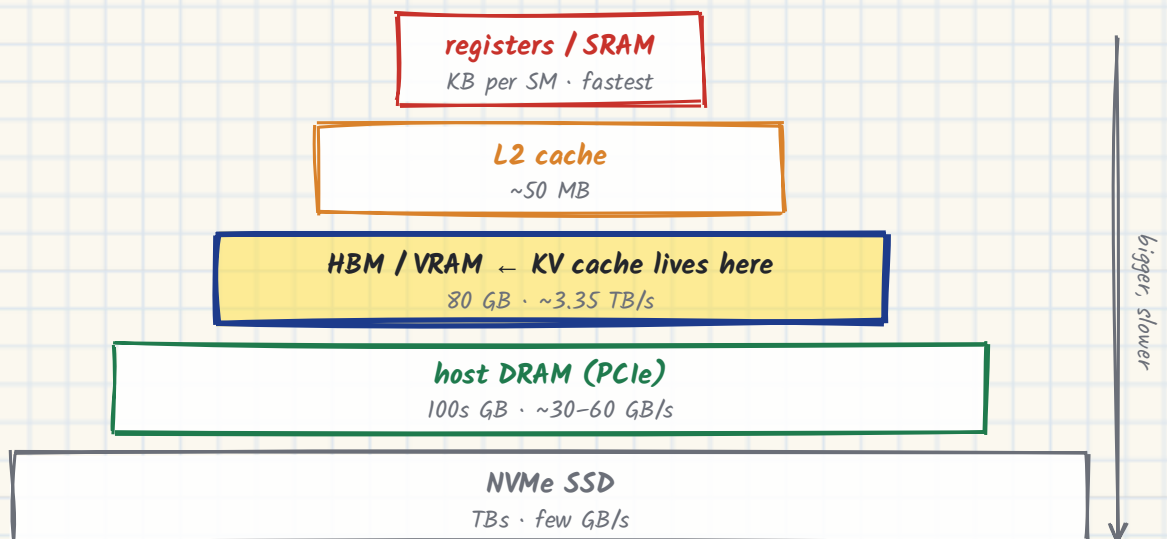
PART 7 · Module B: Memory

KV Cache in GPU Memory

Goal of this part

Drop below the Transformer abstraction. Where do the K/V bytes physically live, how fast can they be read, how are they laid out, and why does serving many users at once turn KV caching into a memory-management problem, the kind operating systems solved decades ago?

7.1 The memory hierarchy: where KV lives



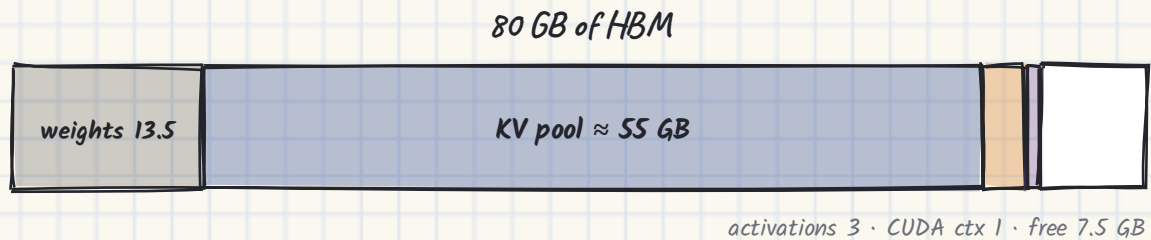
each level down is bigger and slower. The KV cache lives in HBM (VRAM), and in the worst case spills lower.

Level	Size (H100-class)	Bandwidth	KV's relationship to it
registers / shared memory (SRAM)	~256 KB per SM (x132 SMs)	tens of TB/s	attention kernels stage K/V tiles here while computing
L2 cache	~50 MB	several TB/s	far too small to hold a cache; helps a little with reuse
HBM (VRAM)	80 GB	~3.35 TB/s	the KV cache's home. Every decode step streams it
host DRAM over PCIe	hundreds of GB – TBs	~25-32 GB/s (Gen4 x16), ~50-64 (Gen5)	offload tier (Part 13): ~100x slower than HBM

NVMe SSD	TBs	a few GB/s	cold tier for very long or reusable prefixes
----------	-----	------------	--

Your RTX 4050 laptop: 6 GB of GDDR6 (not HBM) at ~192 GB/s, over a 96-bit bus. Same hierarchy, smaller numbers. HBM is stacked DRAM on the GPU package with a very wide bus; GDDR is conventional graphics DRAM around the chip.

7.2 What's in VRAM during serving



an A100 80 GB serving Llama-2-7B in FP16 (approximate): the KV pool is the biggest tenant

- **Weights:** fixed, loaded once (13.5 GB).
- **KV cache pool:** everything left after the rest. Serving engines like vLLM **pre-allocate** it at startup (e.g. 90% of VRAM minus weights and a profiled activation peak), then hand it out themselves.
- **Activations / workspace:** temporary tensors for the current batch (largest during big prefills), attention and GEMM workspaces, logits.
- **CUDA context, kernels, libraries:** a few hundred MB to ~1 GB.
- **Allocator slack:** PyTorch's caching allocator keeps freed blocks reserved. `memory_reserved ≥ memory_allocated` (Part 17).

7.3 Reads and writes per decode step

For each layer, a decode step **writes** one new K and V row per sequence (tiny) and **reads** every cached K and V row of every sequence in the batch (huge). Llama-2-7B, batch 16, 4K context:

Per step	Bytes	Relative
KV writes: 16 seq × 1 token × 512 KiB	8 MiB	≈ 0
KV reads: 16 seq × 4,096 tokens × 512 KiB	32 GiB	2.5× the weights
weight reads	12.6 GiB	shared across the batch

At ~2 TB/s that's ~24 ms per step just moving bytes. **Every byte you don't have to store is also a byte you don't have to read on every step.** That's why memory capacity optimizations (GQA, quantization) are also speed optimizations.

7.4 Tensor layouts

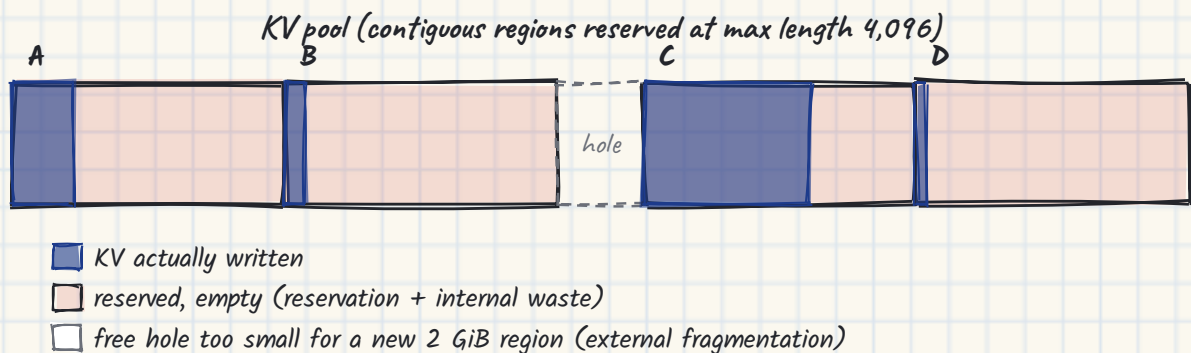
Layout	Shape (per layer)	Pros / cons
--------	-------------------	-------------

Concatenated (HF DynamicCache)	$[B, H_{kv}, t, d_h]$, a new tensor each step	simple. But <code>torch.cat</code> allocates a new, larger tensor and copies the entire old cache every step : $O(t)$ bytes copied per step, $O(T^2)$ total. In a CPU profiling run of Part 14's model, <code>aten::cat</code> was ~19% of runtime.
Static / preallocated	$[B, H_{kv}, T_{max}, d_h]$, allocated once	in-place writes, no copies, CUDA-graph friendly. But it reserves T_{max} for everyone: huge waste when lengths vary.
Paged (vLLM-style)	one pool: $[num_blocks, block_size, H_{kv}, d_h]$ + per-sequence block tables	no copies, near-zero waste, sharing possible. Needs kernels that follow the block table (Part 8).

Inside a row, the d_h values of one head for one token are contiguous ($128 \times 2 B = 256 B$), which matches how GPUs like to read memory: in wide, aligned, coalesced chunks. Layouts are chosen so a warp reads long contiguous runs of K or V.

7.5 Many users, naive allocation: the failure

Naive policy: when a request arrives, reserve one contiguous region big enough for its **maximum** possible length (prompt + `max_new_tokens`), because we don't know when it will stop.



pool is "full" while holding ~24% real data

contiguous max-length reservations: most reserved memory holds nothing

Three kinds of waste appear:

Waste	What it is	Example
Reservation waste	space reserved for tokens not generated yet	a chat reserved for 4,096 that's currently at 700
Internal fragmentation	space reserved for tokens that will never exist	the chat stops at 900 tokens, so 3,196 slots were never used
External fragmentation	free memory exists but in holes too small for a new contiguous region	3 GB free in total, split across gaps of 600 MB, and a new request needs 2 GB contiguous

Try first

Llama-2-7B, 55 GiB KV pool, max length 4,096 (2 GiB per reservation). Requests actually average 600 tokens. How many requests fit, and what fraction of the pool holds real data?

Solution. $55 \div 2 = 27$ requests. Real data: $27 \times 600 \times 0.5 \text{ MiB} \approx 7.9 \text{ GiB} \rightarrow \sim 14\%$ utilization. If memory were allocated exactly as needed, the same pool would hold ~ 187 requests of 600 tokens. The vLLM paper (Kwon et al., 2023) measured existing systems storing actual token states in only about **20–38%** of their KV memory.

7.6 Why this is a systems problem, not a Transformer problem

Property of the workload	Why the model can't solve it
output lengths are unknown in advance	you can't size allocations correctly
requests arrive and finish continuously	allocations churn; holes appear
memory must be shared (same prompt, parallel samples)	needs reference counting, copy-on-write
memory may run out mid-generation	needs preemption, swapping or recompute policies
the bottleneck is bandwidth	layout, precision and kernel design decide speed

Those are exactly the problems an OS virtual-memory system solves for processes. Part 8 borrows its answer.

Checkpoint · Part 7

Recall

- 1 Where does the KV cache live during inference, and roughly how fast can an H100 read it?
- 2 Name the three kinds of KV memory waste under naive contiguous allocation.
- 3 Why does `torch.cat`-based caching get more expensive as generation proceeds?

Reasoning

- 4 Why is a memory-capacity optimization (like FP8 KV) usually also a latency optimization for decode?

- 5 A static preallocated cache avoids copies and fragmentation. Why don't serving engines just use one static buffer per request?

Practical engineering

- 6 `nvidia-smi` shows 78 of 80 GB used, but your engine reports only 40% of KV blocks in use. Is something leaking? What else could explain it?

Calculation

- 7 H100 (3.35 TB/s), Llama-3-8B FP16 (16.1 GB weights, 128 KiB/token), batch 32, context 8K each. Compute KV bytes read per decode step and the ideal step time. What fraction of step time is KV?

Handwritten note

The cache lives in HBM and is re-read every step, so bytes saved are time saved. Unknown lengths plus many users make KV allocation an OS-style memory-management problem.

What do you think happens next?

Operating systems gave every process the illusion of contiguous memory while really using scattered fixed-size pages. What would that look like for KV?

PART 8 · Module C: Serving systems

PagedAttention

Goal of this part

Understand PagedAttention as the answer to Part 7's fragmentation problem: KV memory split into fixed-size blocks, mapped through per-sequence block tables, allocated on demand, freed instantly and shared safely. Be able to hand-simulate a block table.

Problem → Naive → Bottleneck → Optimization → Trade-off

Problem	Hold KV for many concurrent sequences of unknown, varying length.
Naive	One contiguous region per sequence, sized for the maximum length.
Bottleneck	Reservation waste, internal and external fragmentation: often only 20–40% of KV memory holds real tokens. Batch size, and so throughput, is capped by wasted memory.
Optimization	Split KV into fixed-size blocks (pages) of B_s tokens; keep a per-sequence block table mapping logical → physical blocks; allocate one block at a time.
Trade-off	Attention kernels must gather K/V through an indirection; block-table bookkeeping on the CPU; up to B_s-1 slots wasted per sequence.

8.1 The OS analogy

Operating system	PagedAttention	Why it matters
process	sequence (request)	each has its own address space / token history
virtual page	logical block (tokens $i \cdot B_s \dots (i+1) \cdot B_s - 1$)	the sequence thinks its KV is contiguous
physical frame	physical block in the GPU pool	actual location, anywhere in the pool
page table	block table	maps logical → physical, one per sequence
demand paging	allocate a block only when the last one fills	no reservation waste
shared pages + copy-on-write	shared blocks + ref counts + CoW	parallel sampling, beam search, prefixes

swap to disk

swap blocks to CPU (or
recompute)

preemption when the GPU is full

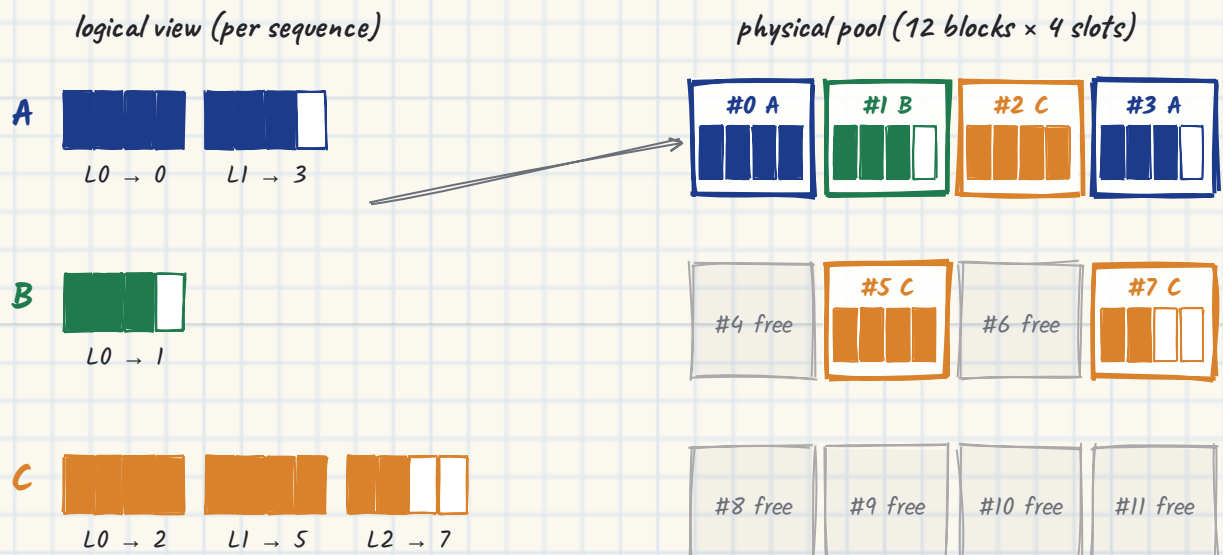
8.2 The pieces

- **Block size B_s** : tokens per block. vLLM's default on GPUs is **16**. One block holds K and V for B_s tokens, for one layer (each layer has its own pool, all indexed by the same block IDs).
- **Physical pool**: allocated once at startup: $[\text{num_blocks}, B_s, H_{kv}, d_n]$ for K and for V, per layer. A block for Llama-2-7B = $16 \times 512 \text{ KiB} = 8 \text{ MiB}$ across all layers.
- **Free list**: IDs of unused physical blocks.
- **Block table per sequence**: [physical id of logical block 0, of block 1, ...] plus the number of filled slots in the last block.
- **Slot mapping**: for each new token, its physical location = $(\text{block_table}[\text{pos} // B_s], \text{pos} \% B_s)$. The model's cache-write kernel scatters the new K/V there.

```
# where does token at position pos of sequence s live?
logical_block = pos // BLOCK_SIZE      # e.g. pos 37 → logical block 2 (for B_s = 16)
offset        = pos % BLOCK_SIZE      # offset 5
physical      = block_table[s][logical_block]
K_pool[layer][physical, offset] = k_new # write during the forward pass
# attention for s: loop over block_table[s], load each block's K/V tile, run online softmax
```

8.3 Worked example: three requests

Block size 4 (small, to fit on paper). Physical pool of 12 blocks, IDs 0–11. Three requests arrive with prompts of **A = 7**, **B = 3**, **C = 10** tokens.



block tables: A → [0, 3] B → [1] C → [2, 5, 7]

logical blocks (left) map to scattered physical blocks (right) through each sequence's block table

Seq	Tokens	Blocks needed = $\lceil n/4 \rceil$	Block table	Slots used in last block
-----	--------	-------------------------------------	-------------	--------------------------

A	7	2	[0, 3]	3 of 4
B	3	1	[1]	3 of 4
C	10	3	[2, 5, 7]	2 of 4

Free list: [4, 6, 8, 9, 10, 11]. Physical order doesn't matter; only the table does.

→ Now let time run

1. **Decode step 1:** A → 8 tokens (fills block 3's last slot, no allocation). B → 4 (fills block 1). C → 11 (block 7 now 3/4).
2. **Decode step 2:** A → 9: block 3 is full → pop 4 from the free list → A = [0, 3, 4]. B → 5: pop 6 → B = [1, 6]. C → 12: fills block 7.
3. **B finishes (EOS):** blocks 1 and 6 go straight back to the free list. No compaction, no holes that matter: any free block fits any sequence.
4. **New request D** (prompt 5) arrives: pops 1 and 6 (or any two free IDs) → D = [1, 6]. Freed memory is reused immediately.

Waste accounting

The only unused slots are in each sequence's **last** block: at most $B_s - 1$ per sequence. With $B_s = 16$ and 1,000-token sequences, that's < 1.5% waste. The vLLM paper reports under ~4% waste, versus 60–80% in earlier systems, and 2–4× higher throughput than FasterTransformer and Orca at similar latency.

8.4 Sharing: parallel sampling and copy-on-write

Request C asks for $n = 2$ samples of the same prompt. Both samples share C's prompt blocks: no copies, just **reference counts**.

```
C.sample1 block table: [2, 5, 7]      refcount(2)=2  refcount(5)=2  refcount(7)=2
C.sample2 block table: [2, 5, 7]
sample1 generates a token → must write into block 7 (partially filled, SHARED)
→ copy-on-write: allocate block 8, copy block 7's 2 filled slots, write there
C.sample1: [2, 5, 8]  C.sample2: [2, 5, 7]  refcount(7)=1  refcount(8)=1
full shared blocks (2, 5) stay shared forever, since nobody writes into a full block
```

Beam search is the same idea with more branching: beams share their common history block by block, and pruning a beam just decrements ref counts.

8.5 Deallocation, preemption, swapping

- **Finish:** decrement ref counts; blocks at 0 return to the free list (or to a prefix cache, Part 10).
- **Out of blocks mid-generation:** the scheduler **preempts** a sequence (typically the most recently arrived). Its blocks are either **swapped** to CPU memory and back later, or simply freed and later **recomputed** by re-running prefill over prompt + generated tokens. Recompute is often cheaper than PCIe transfers for moderate lengths.

8.6 Choosing the block size

Smaller blocks (e.g. 8)	Larger blocks (e.g. 32–128)
less tail waste per sequence	more tail waste
finer-grained prefix sharing	coarser sharing (only full blocks are shareable)
bigger block tables, more gather overhead, shorter contiguous reads	longer contiguous reads, more kernel-friendly

Alternatives exist. **vAttention** keeps a contiguous virtual KV layout and maps physical pages behind it using CUDA's virtual-memory APIs, so unmodified attention kernels still work. Same goal, different layer of the stack.

→ Wrong mental models to cross out

X ~~"PagedAttention is a faster attention algorithm."~~

✓ It's a **memory-management** scheme plus a kernel that can read scattered blocks. The math is the same attention. The gain is capacity → bigger batches → throughput.

X ~~"Pages get moved around to defragment."~~

✓ Nothing moves. Indirection makes physical placement irrelevant, so fragmentation never needs fixing.

Checkpoint · Part 8

Recall

- 1 What are logical blocks, physical blocks and block tables?
- 2 What's the maximum wasted KV space per sequence under paging?
- 3 What happens to a sequence's blocks when it finishes?

Reasoning

- 4 Why does PagedAttention eliminate external fragmentation completely?
- 5 Why is copy-on-write only needed for the last, partially filled shared block?

Practical engineering

- 6 You raise block size from 16 to 128 to speed up the attention kernel. Throughput drops for a chat workload with many short requests. Why?

Calculation

- 7 Block size 16, pool of 20 blocks. Requests: $A = 40$ tokens, $B = 17$, $C = 64$. (a) Block tables if the allocator hands out the lowest free ID first, in order A, B, C. (b) Free blocks left. (c) After 10 decode steps for all three, how many new blocks were allocated?

Handwritten note

PagedAttention = virtual memory for KV: fixed-size blocks + block tables. Allocate on demand, free instantly, share with ref counts. Waste $\leq$ one partial block per sequence.

What do you think happens next?

Memory is now handed out block by block. Could the scheduler also add and remove sequences from the running batch on every single step?

PART 9 · Module C: Serving systems

Continuous Batching

Goal of this part

See how a server keeps the GPU busy with many requests of different lengths, and why the KV cache (not compute) usually decides how many requests can run at once. Leave able to compute "maximum concurrent sequences" for any model and GPU.

Problem → Naive → Bottleneck → Optimization → Trade-off

Problem	Decode is memory-bound at small batch (Part 3); we need big batches to amortize weight reads.
Naive	Static batching : group N requests, run until all finish, then start the next group.
Bottleneck	Finished sequences leave idle slots until the longest one ends; new arrivals wait; padding wastes compute and KV.
Optimization	Continuous batching (iteration-level scheduling): rebuild the batch every step. Finished sequences leave, waiting ones join immediately.
Trade-off	Scheduler runs every iteration (CPU overhead); needs a KV allocator that grows and frees per step (paging); prefills of new requests disturb running decodes.

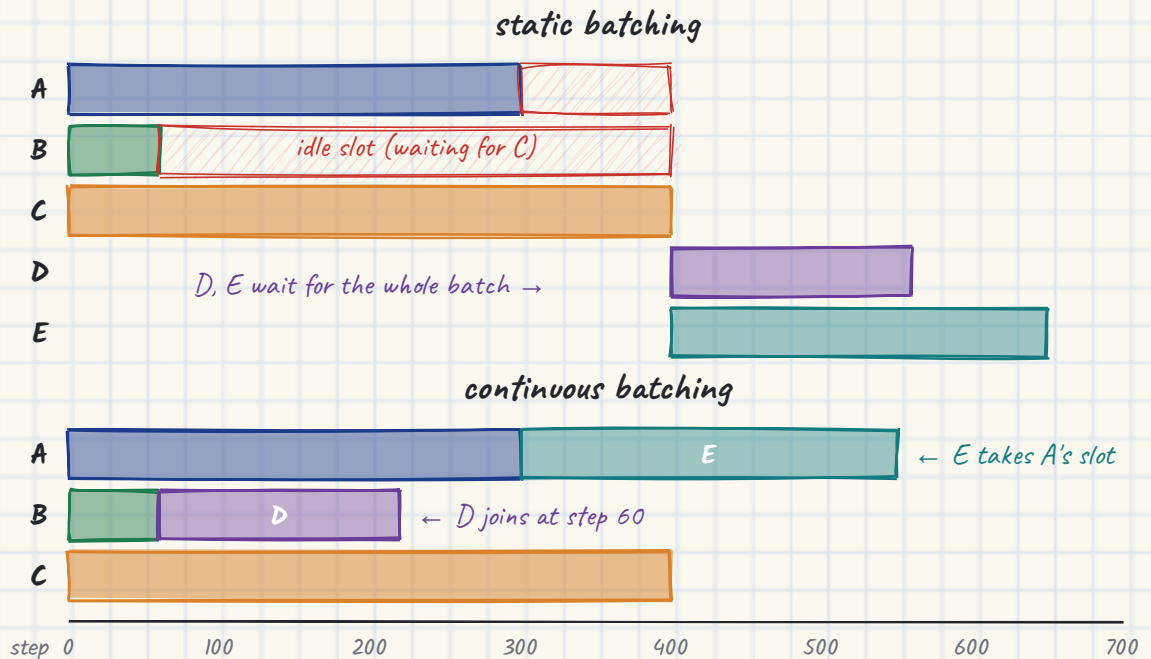
9.1 Three ways to batch

Scheme	How it works	KV consequence
Static	fixed batch; everyone starts together, the batch ends when the longest finishes	finished sequences' KV (or padding) sits idle; memory sized for the worst case
Dynamic	collect requests for up to X ms or N requests, then run as a static batch	better batch formation, same idle-slot problem during generation
Continuous (iteration-level)	scheduling decision at every decode iteration : remove finished, admit new (their prefill joins the step)	KV blocks freed the moment a sequence ends and reused next step; requires paged allocation

Iteration-level scheduling was introduced by Orca (Yu et al., OSDI 2022). vLLM, TGI, TensorRT-LLM ("in-flight batching") and SGLang all build on it.

9.2 Example: $A = 2,000$, $B = 200$, $C = 7,000$ tokens

Treat these as prompt lengths, with outputs of 300 (A), 60 (B) and 400 (C) tokens. Requests D (prompt 500) and E (prompt 1,200) arrive shortly after. Llama-2-7B, 0.5 MiB/token.



static batching idles the slots of finished sequences; continuous batching refills them on the next iteration

→ What the server does, step by step (continuous)

- Iteration 0:** admit A, B, C. Their prefills run (9,200 prompt tokens). KV allocated: $9,200 \times 0.5 \text{ MiB} \approx 4.5 \text{ GiB}$ (~ 576 blocks of 16). Each produces its first token.
- Iterations 1–59:** one decode token for each of A, B, C per step. Each gains a new block every 16 tokens.
- Iteration 60:** B hits EOS → its ~ 17 blocks return to the free list. **D is admitted at the same iteration:** its 500-token prefill runs alongside A and C's decode tokens.
- E joins when the scheduler has blocks and batch budget for it. A leaves at ~ 300 , C at ~ 400 . Nothing waits for the longest request.

The KV-memory view of the same story

C alone holds $7,400 \times 0.5 \text{ MiB} \approx 3.6 \text{ GiB}$ at the end: about $3\times$ what A holds, or ~ 28 B-sized requests. Long prompts don't just take longer; they occupy the shared KV pool, cutting how many other requests can run (see Part 19).

9.3 Maximum concurrent sequences

The batch can only contain sequences whose KV fits. So the scheduler's hard limit is:

$$N_{\max} \approx (M_{\text{GPU}} \cdot u - M_{\text{weights}} - M_{\text{activations}}) \div (kv_{\text{tok}} \cdot \bar{L})$$

u = usable fraction (e.g. 0.9) · $\bar{L}$ = average tokens per sequence at its peak (prompt + generated)

Model on GPU	KV budget	tokens that fit	@ 2K	@ 8K	@ 32K
Llama-2-7B, A100 80GB	~55.5 GB	~106K	~52	~13	3
Llama-3-8B, A100 80GB	~52.9 GB	~404K	~197	~49	~12
Qwen2.5-0.5B, RTX 4050 6GB	~3.6 GB	~294K	~143	~36	~9
Qwen2.5-1.5B, RTX 4050 6GB	~1.5 GB	~53K	~26	~6	1

Assumptions: 90% usable; 3 GB activations on the A100, 0.8 GB overhead on the 4050. On the 4050 you'll probably be limited by compute and bandwidth before reaching those batch sizes, but the memory ceiling is real.

In practice engines also cap the batch with settings like `max_num_seqs` (sequences) and `max_num_batched_tokens` (tokens per iteration), and keep headroom for sequences that are still growing.

9.4 Why KV allocation dominates the scheduler's life

- **Admission:** "Can I admit this request?" means "are there enough free blocks for its prompt, plus headroom?"
- **Growth:** every running sequence needs a new block every B_s tokens. If none is free, someone must be **preempted** (swapped to CPU or dropped and recomputed later).
- **Fairness vs throughput:** admitting one 32K prompt might evict or block ten 2K chats.
- **Prefill interference:** a big new prefill in the same iteration makes that step long, so every running decode sees an ITL spike. Chunked prefill (Part 13) bounds it.

Interview phrasing

"Continuous batching makes the batch a per-iteration decision. That only works if KV memory can grow and shrink per sequence at fine granularity, which is why it goes hand in hand with paged KV allocation. The scheduler's real constraint is free KV blocks, not FLOPs."

→ Wrong mental models to cross out

~~X "Batch size is a fixed configuration number."~~

✓ With continuous batching the running batch changes every step, bounded by free KV memory and configured caps.

~~X "Adding more requests to the batch is free until compute is saturated."~~

✓ Each request adds KV **memory** and KV **reads** per step, so ITL grows with batch even while compute sits idle.

Checkpoint · Part 9

Recall

- 1 Contrast static, dynamic and continuous batching in one sentence each.
- 2 Write the formula for maximum concurrent sequences.
- 3 What happens when a running sequence needs a new block and none are free?

Reasoning

- 4 Why does continuous batching practically require paged (or similarly fine-grained) KV allocation?
- 5 Why can admitting one long-prompt request hurt the latency of requests already running, even if memory is available?

Practical engineering

- 6 Your server's throughput is capped at ~40 concurrent sequences although GPU compute is barely used. What limit are you probably hitting, and name two ways to raise it.

Calculation

- 7 Llama-3-70B with FP8 weights (~70 GB), KV 320 KiB/token in FP16, 90% usable memory, 2 GB activations per GPU. (a) How many 4K-token sequences fit on one H100 80GB? (b) On two H100s with tensor parallelism (weights and KV heads split across both)? (c) What changes with FP8 KV?

Handwritten note

Continuous batching rebuilds the batch every iteration. Its ceiling is free KV blocks:
 $N_{\max} \approx \text{KV budget} \div (k_{\text{v}_{\text{tok}}} \times \text{tokens per sequence}).$

What do you think happens next?

If 100 users send the same 5,000-token system prompt, the pool holds 100 identical copies of its KV. Can blocks be shared across requests?

PART 10 · Module C: Serving systems

Prefix Caching

Goal of this part

Separate two ideas people blur: the KV cache **inside one generation**, and **reusing KV across different requests** that share a prefix. Understand how engines find, share, evict and invalidate reusable prefixes.

Problem → Naive → Bottleneck → Optimization → Trade-off

Problem	Many requests start with identical tokens: system prompts, few-shot examples, a shared document, earlier chat turns.
Naive	Prefill every request from scratch; store a private copy of the same KV for each.
Bottleneck	Repeated prefill compute (TTFT, GPU time) and duplicated KV memory (fewer concurrent sequences).
Optimization	Keep KV blocks of finished or ongoing prefixes; identify them by content hash ; map new requests' block tables onto the existing blocks.
Trade-off	Hashing and lookup overhead; cached blocks compete with live requests for memory (eviction policy); only exact token prefixes match.

10.1 Two different things called "caching"

	KV cache (within one generation)	Prefix cache (across requests)
reuses	this sequence's own past tokens	another request's KV for an identical prefix
saves	recomputation on every decode step	prefill compute and duplicate memory
improves	ITL / throughput (vs no cache)	TTFT, prefill GPU time, memory
lifetime	until the sequence ends	until evicted (can outlive the request)
correctness risk	positions, masks	must match tokens AND everything else that affects KV (model, adapter, position)

10.2 Example: 5,000-token system prompt, 100 users

Llama-2-7B (0.5 MiB/token), block size 16. Each user adds a short question after the shared system prompt.

Try first

How much KV memory does the system prompt take for 100 users without sharing? With sharing? How much prefill compute is saved?

	No prefix caching	With prefix caching
system-prompt KV in memory	$100 \times 2.44 \text{ GiB} = 244 \text{ GiB}$ (impossible on one GPU)	2.44 GiB once
prefill tokens processed for the prefix	$100 \times 5,000 = 500,000$	5,000 (first user only)
prefill FLOPs for the prefix	$\approx 2 \times 6.74e9 \times 500,000 \approx 6.7 \times 10^{15}$	$\approx 6.7 \times 10^{13}$
TTFT for users 2–100	$\sim 0.4 \text{ s}$ (5,000-token prefill)	only the question's tokens: tens of ms

What exactly is reusable? $5,000 \div 16 = 312$ full blocks (4,992 tokens) + 8 leftover tokens. Engines typically share only full blocks, so each user recomputes the last 8 system-prompt tokens along with their question. Negligible.

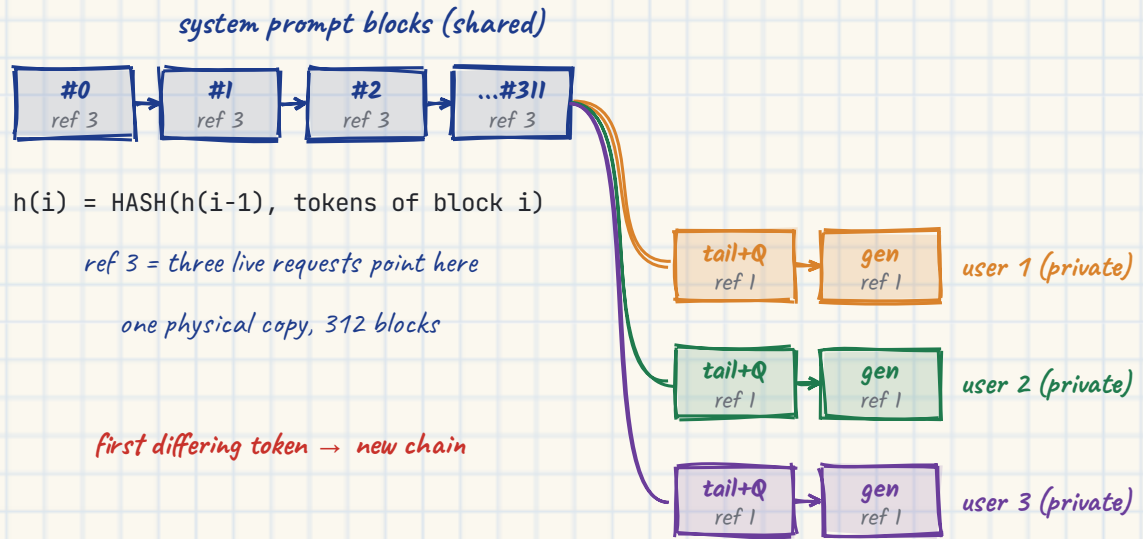
Why are full blocks the sharing unit? A block's content is identified by its tokens. A partial block would change as soon as the owner writes its next token, so sharing it would need copy-on-write on every hit.

10.3 How engines recognise a shared prefix: chained block hashes

Each full block gets a hash that covers its own tokens and everything before it:

```
h(block 0) = HASH( None,      tokens[0:16],  extra_keys )
h(block 1) = HASH( h(block 0), tokens[16:32], extra_keys )
h(block i) = HASH( h(block i-1), tokens[16i : 16i+16], extra_keys )
# extra_keys: anything else that changes the KV: LoRA adapter id, image/multimodal hashes, cache salt...
```

Chaining matters: the same 16 tokens appearing after a different history produce different KV (attention mixed in different earlier tokens, and positions differ), so they must get a different hash.



shared blocks with reference counts: three requests reuse the system-prompt blocks and diverge afterwards

→ Lookup on a new request

1. Tokenize; compute block hashes for all full blocks of the prompt.
2. Walk from block 0: if the hash is in the cache table, reuse that physical block (ref count +1) and continue; stop at the **first miss**.
3. Prefill only the tokens after the matched prefix, attending to the reused blocks as if it had computed them.

→ Lifetime and eviction

- When a request finishes, its blocks' ref counts drop. Blocks at 0 are **not** erased: they stay as *cached, evictable* blocks.
- When memory is needed, evict evictable blocks, usually **LRU**, preferring blocks at the end of a chain (evicting block 5 of a prefix while keeping blocks 0–4 still gives partial hits).
- Live requests always win over cached-but-unused blocks.

vLLM's "automatic prefix caching" works like this with hashed blocks. SGLang's **RadixAttention** keeps prefixes in a radix tree of token sequences and schedules requests to maximize tree hits. API providers' "prompt caching" is the same idea offered as a product: cached input tokens are billed at a discount and skip prefill.

10.4 Invalidation: when a "prefix" isn't the same prefix

Change

any token earlier in the prompt differs (a timestamp, a user name at the top of the system prompt)

Effect

every block after it misses, because the hashes are chained

same text, different tokenization (whitespace, template change)	different tokens → miss
same text at a different position (shifted by one token)	miss: KV depends on absolute positions (RoPE)
different model weights, LoRA adapter, or KV dtype	miss (must be part of the key)
different sampling settings (temperature, top-p)	still a hit: sampling happens after the logits, KV is unaffected
model redeploy / restart	cache is gone (it lives in GPU memory)

Prompt-engineering rule that falls out of this

Put **static content first** (system prompt, tools, documents, few-shot examples) and **dynamic content last** (user message, date, request ID). A timestamp at the top of a system prompt silently destroys prefix caching for every request.

→ Wrong mental models to cross out

~~X "Prefix caching speeds up decode."~~

✓ It removes **prefill** work (TTFT) and duplicate memory. Decode per step is unchanged (it still reads the whole prefix KV each step).

~~X "Similar prompts share cache."~~

✓ Only an **exact** token prefix from position 0 matches. One different early token breaks the chain.

Checkpoint · Part 10

Recall

- 1 Which latency metric does prefix caching improve, and why not the other one?
- 2 Why are block hashes chained to the previous block's hash?
- 3 Name three changes that cause a prefix-cache miss and one that doesn't.

Reasoning

- 4 Two requests contain the same 2,000-token document, but one puts it after a 30-token instruction and the other after a 45-token instruction. Can they share the document's KV? Why?
- 5 Why should evicted-but-cached blocks be taken from the end of prefix chains first?

Practical engineering

- 6 After a "small" prompt-template change, your prefix-cache hit rate drops from 85% to 3% and TTFT doubles. What most likely happened, and how do you confirm it?

Calculation

- 7 Llama-3-8B (128 KiB/token), block size 16. 200 concurrent users share a 3,000-token system prompt, and each has 1,000 private tokens. Compute total KV with and without prefix sharing, and the number of shared full blocks.

Handwritten note

KV cache = reuse within one sequence (saves decode recompute). Prefix cache = reuse across requests (saves prefill and memory). Matching is by chained block hashes over exact tokens from position 0.

What do you think happens next?

We've cut waste and shared duplicates. What if each stored number simply took fewer bytes?

PART 11 · Module D: Optimization

KV Cache Quantization

Goal of this part

Know why and how to store K and V in fewer bits, what each format costs in accuracy and overhead, and how to decide when it's worth it.

Problem → Naive → Bottleneck → Optimization → Trade-off

Problem	Long contexts and big batches: KV memory caps concurrency; KV reads dominate decode time.
Naive	Store K/V in the model's compute precision (FP16/BF16, 2 bytes).
Bottleneck	Bytes per element: b appears linearly in both memory (Part 5) and per-step bandwidth (Part 3).
Optimization	Store K/V in 8 bits (FP8/INT8) or fewer, with scale factors; dequantize on the fly inside the attention kernel.
Trade-off	Rounding error → possible quality loss (keys more sensitive than values); quantize/dequantize work; needs kernel support.

11.1 Why quantizing the cache is especially attractive

- **Memory:** half the bytes → twice the tokens in flight → twice the concurrent sequences or context.
- **Bandwidth:** decode is memory-bound, so reading half the bytes makes the KV part of each step up to $\sim 2\times$ faster.
- **Overhead is "free":** dequantizing costs a few arithmetic ops per element, and decode leaves the math units idle anyway (intensity ≈ 1). The extra math **hides under the memory time**.
- Weights are quantized once, offline. **KV is produced at runtime**, so it must be quantized on the fly, at every cache write.

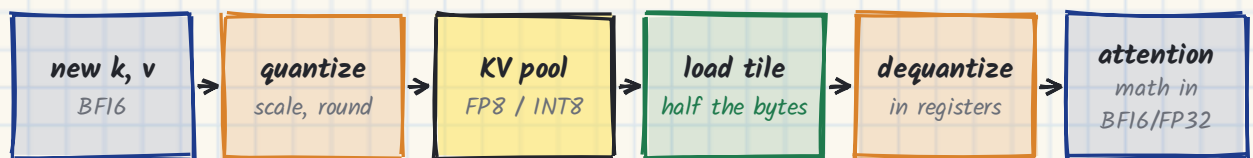
11.2 The formats

Format	Bytes	How values are represented	Typical use for KV
FP16	2	5 exponent + 10 mantissa bits: good precision, max $\sim 65,504$	baseline (common)

BF16	2	8 exponent + 7 mantissa: FP32's range, less precision	baseline (common)
FP8 E4M3	1	4 exponent + 3 mantissa: max ± 448 , finer steps	preferred FP8 flavour for KV; needs a scale so values fit the range
FP8 E5M2	1	5 exponent + 2 mantissa: huge range, coarse steps	works without careful scaling; a bit less accurate
INT8	1 (+ scales)	integers $-128 \dots 127 \times \text{a scale}$ (per token, per head or per group)	widely supported, e.g. llama.cpp q8_0
INT4 / 4-bit	0.5 (+ scales)	16 levels per group $\times \text{scale}$ (+ zero point)	aggressive; needs care
2-bit and mixed	≤ 0.25 (+)	research methods: KIVI, KVQuant, ...	extreme compression for very long contexts

Scale overhead example: INT4 with one FP16 scale + FP16 zero point per group of 64 values = 4 extra bytes per 32 data bytes, so "4-bit" really costs ~ 4.5 bits per value. Always count the metadata.

11.3 How quantized KV flows through a decode step



write path: once per token

read path: every step, every token

HBM traffic halves; extra math uses idle compute

quantize once on write, dequantize tile by tile on read, inside the attention kernel

→ Granularity and why keys are harder

- A single scale for the whole tensor is cheapest but lets one outlier ruin everyone's precision.
- **Keys** often have a few **outlier channels** (dimensions with consistently large magnitude), partly because of how RoPE rotates pairs of dimensions. Research methods like **KIVI** therefore quantize keys **per channel** and values **per token**. **KVQuant** quantizes keys before RoPE is applied, among other tricks.
- Errors in K distort **softmax scores** (which token gets attention). Errors in V only blur the **blend**. That's why keys usually need more bits or finer scales.
- Many methods keep the most recent tokens (and sometimes the first few "sink" tokens) in full precision, since they tend to get the most attention.

11.4 Trade-offs in numbers

Llama-3-8B, 32K context, batch 16 (512K tokens in flight), H100 3.35 TB/s:

KV format	KV memory	KV read / step	≈ KV time / step	Accuracy risk
BF16	64 GiB	68.7 GB	20.5 ms	none (baseline)
FP8	32 GiB	34.4 GB	10.3 ms	usually negligible
INT8 (grouped)	~33 GiB	~35 GB	~10.5 ms	usually small
INT4 (grouped)	~18 GiB	~19 GB	~5.8 ms	task-dependent; test it

Add the weight read (16.1 GB ≈ 4.8 ms/step) and the BF16 step is ~25 ms while FP8 is ~15 ms: ITL drops by ~40%, and the freed 32 GiB can hold another 16 sequences.

11.5 When is KV quantization worth it?

Worth it when

- long contexts or large batches (KV ≫ weights in memory or traffic)
- you're capacity-bound: preemptions, low max batch
- decode ITL matters and KV reads dominate
- your engine has fused FP8/INT8 attention kernels

Think twice when

- contexts are short: weights dominate, little to gain
- tasks need precise long-range retrieval (needle-in-a-haystack, code, math); validate first
- kernels fall back to a slow path
- you're chasing the last point of benchmark accuracy

→ How to turn it on (examples)

```
# vLLM
vllm serve meta-llama/Llama-3.1-8B-Instruct --kv-cache-dtype fp8
# Ollama (q8_0 or q4_0; requires flash attention)
OLLAMA_FLASH_ATTENTION=1 OLLAMA_KV_CACHE_TYPE=q8_0 ollama serve
# llama.cpp server
llama-server -m model.gguf -fa on --cache-type-k q8_0 --cache-type-v q8_0
```

Flag names and spellings change between versions. Check your version's docs. In llama.cpp, quantizing the V cache requires flash attention to be enabled, and the flag's syntax has varied across releases.

Interview phrasing

"Decode is bandwidth-bound, so KV quantization buys both capacity and speed, and the dequantization math is nearly free because compute is idle. FP8 is usually safe. Below 8 bits, keys need special handling because of outlier channels, and you validate on long-context retrieval tasks."

Checkpoint · Part 11

Recall

- 1 What's the difference between FP8 E4M3 and E5M2?
- 2 Why are keys usually harder to quantize than values?
- 3 Where does dequantization happen during decode?

Reasoning

- 4 Why is dequantization overhead nearly free during decode but could matter during a huge prefill?
- 5 Why do many methods keep the most recent tokens in full precision?

Practical engineering

- 6 After enabling 4-bit KV, chat quality looks fine but a "find the clause in this 100-page contract" feature starts missing answers. Explain, and propose a plan.

Calculation

- 7 RTX 4050 (6 GB, 192 GB/s), Qwen2.5-1.5B (3.1 GB weights, 28 KiB/token in FP16), batch $4 \times 8K$ context. Compute KV memory and the ideal decode step time with FP16 KV and with 8-bit KV.

Handwritten note

b is linear in both memory and bandwidth. Quantizing KV to 8 bits halves both, nearly for free, because decode's math units are idle. Keys need more care than values.

What do you think happens next?

At 1M tokens even FP8 KV for one sequence is enormous. What else breaks when context grows 100×?

PART 12 · Module D: Optimization

Long Context

Goal of this part

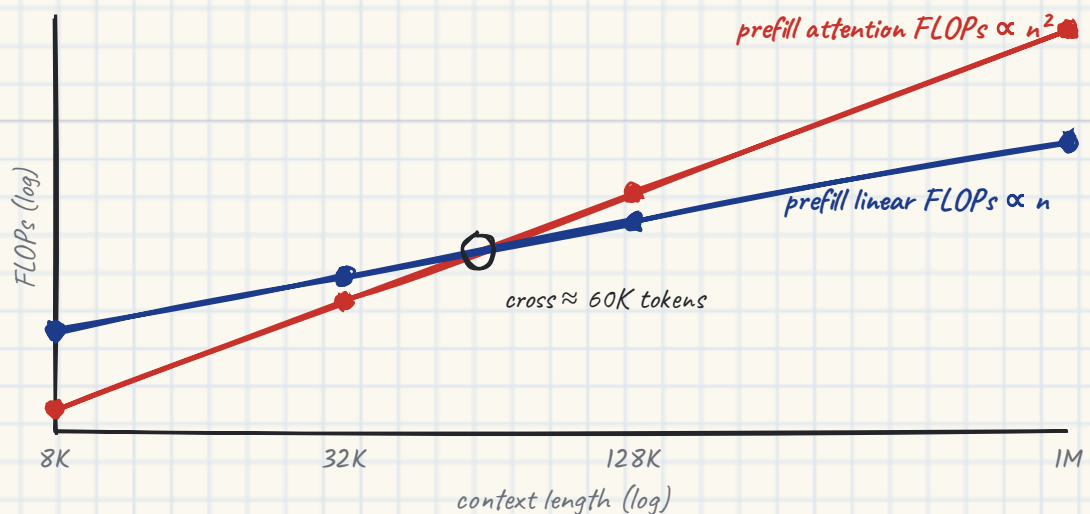
Watch every quantity from Parts 3–11 as context grows from 8K to 1M tokens, and name the problems that appear, in the order they appear.

12.1 The scaling table

Llama-3-8B (128 KiB/token, weights 16.1 GB) and Llama-3-70B (320 KiB/token, weights ~141 GB), BF16 KV, one sequence:

Context	8B KV	70B KV	8B prefill: attention FLOPs	8B prefill: linear FLOPs	8B decode KV read / step (H100)
8K	1 GiB	2.5 GiB	1.8×10^{13}	1.3×10^{14}	0.3 ms
32K	4 GiB	10 GiB	2.8×10^{14}	5.3×10^{14}	1.3 ms
128K	16 GiB	40 GiB	4.5×10^{15}	2.1×10^{15}	5.1 ms ($\approx$ weights' 4.8 ms)
1M	128 GiB	320 GiB	2.9×10^{17}	1.7×10^{16}	41 ms

Prefill attention (causal) $\approx 2 \cdot L \cdot n^2 \cdot d$ FLOPs; linear $\approx 2 \cdot P \cdot n$. KV read time at 3.35 TB/s, batch 1. $K = 1,024$ tokens, $1M = 1,048,576$.



Llama-3-8B prefill, log-log: attention FLOPs ($\propto n^2$) overtake linear-layer FLOPs ($\propto n$) at about 60K tokens

12.2 The five relationships

Context length vs...	Relationship	What it means in practice
KV memory	linear: $n \cdot kv_{tok}$	1M tokens of Llama-3-8B = 128 GiB, more than an 80 GB GPU for ONE user
prefill attention compute	quadratic: $\propto n^2$	at 128K it already exceeds all linear-layer FLOPs; TTFT becomes tens of seconds
decode attention compute	linear per step: $\propto n$	still small in FLOPs, but...
memory bandwidth per step	linear per step: $n \cdot kv_{tok}$	at 128K one sequence's KV read $\approx$ the whole model's weight read; ITL roughly doubles
concurrency	inverse: $N_{max} \propto 1/n$	an A100 that serves ~197 chats at 2K serves ~3 at 128K (Llama-3-8B)

12.3 Problems that appear as context grows

1. **TTFT explodes.** A 128K-token prefill of Llama-3-8B is $\sim 6.6 \times 10^{15}$ FLOPs: ~ 35 s on an A100 at realistic efficiency, ~ 13 s on an H100. Users wait; other requests queue behind it.
2. **Prefill hogs the batch.** One giant prefill in an iteration stalls every running decode (ITL spike). → chunked prefill (Part 13).
3. **Concurrency collapses.** A few long requests consume the KV pool; short chats get queued or preempted. → separate pools/routing for long-context traffic, KV quantization.
4. **Decode slows.** KV reads dominate each step. → GQA/MLA, FP8 KV, sparse or windowed attention.
5. **One GPU isn't enough.** 1M tokens needs the KV split across devices: tensor parallelism splits KV heads; context/sequence parallelism (e.g. ring attention) splits the sequence and passes K/V blocks between GPUs.
6. **Prefix reuse becomes essential.** Re-prefilling a 200K-token document for every question is wasteful. → prefix caching, persistent/offloaded KV (Part 13).
7. **Model-side limits.** Beyond the trained length, quality degrades (position extrapolation). And models may use long contexts unevenly ("lost in the middle"). Caching can't fix modelling limits.

The long-context summary in one sentence

Everything the KV cache "costs" (memory, bandwidth, and prefill compute to fill it) scales with context, while the thing it saves (recomputation) was already gone. So long context is where the cache stops being an optimization and becomes **the system's main constraint**.

Interview phrasing: "what changes at 128K?"

"Per sequence the KV is ~16 GiB for an 8B GQA model, so concurrency drops ~60× versus 2K. Decode ITL roughly doubles because the KV read matches the weight read. Prefill becomes attention-dominated and takes seconds, so I'd use chunked prefill, prefix caching for repeated documents, FP8 KV, probably a separate long-context pool, and I'd watch preemption rates."

Checkpoint · Part 12**Recall**

- 1 How do KV memory, prefill attention FLOPs and per-step KV reads scale with context length n ?
- 2 Roughly where does prefill attention overtake the linear layers for Llama-3-8B?
- 3 Name two ways to split a 1M-token cache across GPUs.

Reasoning

- 4 Why does long context hurt concurrency more than it hurts single-user latency?
- 5 Why can't a KV cache fix "lost in the middle"?

Practical engineering

- 6 You add a "chat with your 300-page PDF" feature to a service that previously handled 2K-token chats. What three metrics do you expect to move, and what would you change in the serving setup before launch?

Calculation

- 7 Llama-3-70B on $8 \times H100$ (tensor parallel, KV split evenly), ~400 GB KV budget in total, BF16 KV. How many concurrent 128K-token sequences fit? With FP8 KV? How many 4K chats does ONE 128K sequence displace?

 Handwritten note

Long context scales every KV cost: memory $\propto n$, reads per step $\propto n$, prefill $\propto n^2$.
Concurrency falls $\propto 1/n$. At long context, KV is the system's constraint.

What do you think happens next?

If most attention mass goes to recent tokens (and a few early ones), must we keep every token's K and V?

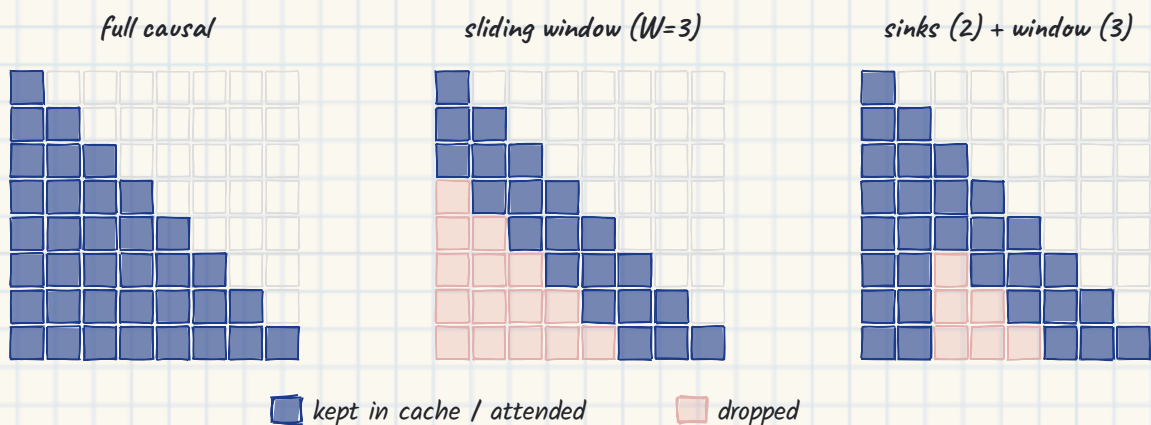
PART 13 · Module D: Optimization

Advanced KV-Cache Techniques

Goal of this part

A field guide. For each technique: the **problem** (which bottleneck), the **idea**, the **trade-off**, and **when it's useful**. By the end, given a symptom, you should be able to pick a technique and name its price.

Bottleneck you see	Techniques that attack it
KV memory (capacity, concurrency)	sliding window · local attention · eviction · compression (MLA, quantization) · offloading · prefix sharing
KV bandwidth (slow decode at long context)	window / local attention · eviction · compression · quantization
prefill compute (TTFT) and interference	prefix sharing · chunked prefill · disaggregated prefill/decode · KV-aware routing
sequential decode latency	speculative decoding



which keys each query may keep: full causal, sliding window, and attention sinks + window (StreamingLLM-style)

① Sliding-window attention

Problem	KV grows without bound with context; so do per-step reads.
Idea	Each token attends only to the last W tokens (the model is trained this way). The cache becomes a rolling buffer of W entries per layer: memory and reads are $O(W)$, not $O(n)$. Stacking layers still lets information travel further than W . Mistral 7B v0.1 used $W = 4,096$.
Trade-off	No direct access to tokens older than W ; long-range recall relies on indirect propagation. Must be designed in at training time.
When useful	Streaming or very long inputs where recent context matters most; fixed memory per sequence.

② Local attention (interleaved local/global layers)

Problem	Pure windows lose long-range recall; full attention everywhere is expensive.
Idea	Mix layer types: most layers use a short local window, a few layers keep full global attention. Only global layers need full-length KV. Gemma 2 alternates local and global layers; Gemma 3 uses 5 local layers per global one with a 1,024-token window.
Trade-off	An architecture choice; the engine must manage two cache types (rolling and full) per model.
When useful	Long-context models where you want most of the memory saving without giving up global recall.

③ KV eviction (keep the important tokens)

Problem	A model trained with full attention, served with too little memory.
Idea	Drop K/V for tokens unlikely to matter. StreamingLLM : keep the first few "attention sink" tokens (which absorb lots of attention mass) plus a recent window. H2O : keep "heavy hitters" with the highest accumulated attention. SnapKV : pick important prompt positions using attention patterns from an observation window at the end of the prompt.
Trade-off	Lossy and irreversible : an evicted token can't be attended to later, even if the question suddenly needs it. Heuristics are task-dependent.
When useful	Very long streaming sessions, summarization-like tasks; risky for precise retrieval.

④ Cache compression

Problem	Each token's KV is large ($L \times H_{kv} \times d_h \times 2$).
Idea	Store a smaller representation. MLA (DeepSeek-V2/V3) caches one low-rank latent vector per token per layer and reconstructs per-head KV from it (trained into the model). Also: quantization (Part 11), merging similar tokens, low-rank projections, cross-layer sharing of KV.
Trade-off	Extra compute to decompress; quality risk for post-hoc methods; architecture-level methods require training for them.
When useful	High-concurrency or long-context serving where KV dominates.

⑤ Cache offloading (tiered KV)

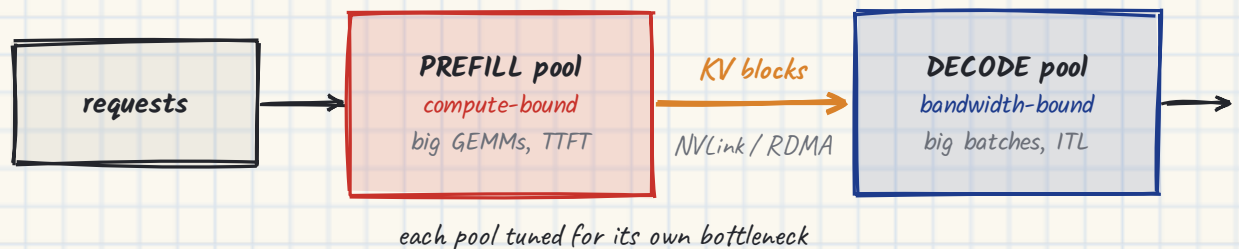
Problem	HBM is small; KV for idle conversations and reusable prefixes is valuable but not needed right now.
Idea	Move cold KV blocks to CPU DRAM or SSD, and bring them back when the conversation resumes. KV-store layers such as LMCache, and engines' CPU swap spaces, do this.
Trade-off	Transfer time over PCIe; extra system complexity; is loading faster than recomputing?
When useful	Multi-turn chat with pauses, long shared documents, agents that revisit context.

⑥ CPU ↔ GPU KV movement: the arithmetic

Problem	Deciding between reload and recompute.
Idea	Compare bytes ÷ link bandwidth with recompute FLOPs ÷ compute throughput. Example: 1 GiB of KV over PCIe Gen4 x16 (~25 GB/s effective) ≈ 43 ms . For Llama-3-8B, 1 GiB = 8K tokens; recomputing an 8K prefill ≈ 1.5×10^{14} FLOPs ≈ 0.3–0.8 s . Reload wins clearly here.
Trade-off	Transfers compete with other traffic; must overlap with compute (prefetch) or they add to latency.
When useful	Whenever KV is large relative to its recompute cost: long prefixes, big models.

⑦ Disaggregated prefill and decode

Problem	Prefill (compute-bound) and decode (bandwidth-bound) interfere on the same GPU: prefills cause ITL spikes, and one configuration can't be optimal for both.
Idea	Separate GPU pools: prefill workers compute the prompt's KV and ship it to decode workers over NVLink/RDMA. Each pool gets its own parallelism and batch policy (DistServe, Splitwise, Mooncake; supported in modern serving stacks).
Trade-off	KV transfer cost and complexity: a 4K-token prompt on Llama-3-70B is ~1.2 GiB of KV, ~26 ms over a 50 GB/s link. Needs capacity balancing between pools.
When useful	Large deployments with strict TTFT and ITL SLOs and long prompts.



⑧ Prefix sharing (beyond one server)

Problem	Prefix caches only help if the request lands where its prefix lives.
Idea	Radix-tree prefix stores (SGLang), plus cache-aware routing across replicas: send a request to the replica most likely to hold its prefix.
Trade-off	Routing for cache hits can fight load balancing (hot replicas).
When useful	Shared system prompts, RAG over the same documents, multi-turn chat, agents.

⑨ Speculative decoding interactions

Problem	Decode is sequential and memory-bound: one token per full weight read.
Idea	A cheap drafter proposes k tokens; the target model verifies all k in one forward pass (a mini-prefill), accepting a prefix of them. KV consequences: the verification step writes k KV entries ; rejected tokens' entries must be rolled back (truncate the sequence length, free blocks); a draft model has its own cache ; tree-based speculation needs tree-shaped attention masks over the cache.
Trade-off	Extra memory (draft cache), wasted work on rejections, cache bookkeeping complexity; gains shrink at large batch where the GPU is already better utilized.
When useful	Latency-sensitive, low-to-moderate batch serving with predictable text (code, structured output).

⑩ Chunked prefill

Problem	A 32K-token prefill in one iteration stalls every running decode for seconds (ITL spikes) and needs a large activation peak.
Idea	Split the prompt into chunks (e.g. 512–2,048 tokens). Each iteration processes one chunk plus the ongoing decodes (Sarathi-Serve's "piggybacking"). Each chunk attends to the KV already written by earlier chunks, so the result is exact.
Trade-off	Slightly higher TTFT for the long request; each chunk re-reads the growing prefix KV; the chunk size tunes TTFT vs ITL.
When useful	Mixed workloads with long prompts and interactive users. It's the default in many modern engines.

⑪ KV-aware scheduling

Problem	Schedulers that ignore KV admit too much (then preempt and thrash) or too little (idle memory).
Idea	Make free blocks the currency: admission control by predicted KV need; preemption choice (swap vs recompute, which victim); priority for short or near-finished requests; routing by prefix-cache locality and KV load (KV-aware routers in stacks like NVIDIA Dynamo and llm-d).
Trade-off	Output lengths are unknown, so predictions are imperfect; fairness vs throughput tension.
When useful	Always in production; critical under memory pressure and bursty load.

Checkpoint · Part 13

Recall

- 1 What's an attention sink, and why does StreamingLLM keep those tokens?
- 2 What must happen to the KV cache when speculative tokens are rejected?
- 3 What problem does chunked prefill solve, and is its output exact?

Reasoning

- 4 Sliding-window attention and KV eviction both bound memory. Why is one "safe" by construction and the other lossy?
- 5 When would you prefer recomputing a prefix over reloading its offloaded KV?

Practical engineering

- 6 Symptom: p99 ITL spikes to 2 s whenever an enterprise customer uploads a long document; median ITL is 30 ms. Pick two techniques and state their trade-offs.

Calculation

- 7 A 70B model (320 KiB/token) runs disaggregated. Prompts average 6K tokens; the link between pools gives 25 GB/s effective. Compute KV transfer time per request. If prefill takes 600 ms, what share of TTFT is transfer, and how could you hide it?

Handwritten note

Name the bottleneck first: memory → window/evict/compress/offload; bandwidth → compress/quantize/window; prefill interference → chunk/disaggregate; sequential latency → speculate. Every technique has a price.

What do you think happens next?

Enough theory. Can you build a tiny Transformer whose cached and uncached outputs match token for token, and measure the difference yourself?

PART 14 · Module E: Hands-on

Implement It From Scratch

Goal of this part

Build a tiny decoder-only Transformer in plain PyTorch, with no hidden library magic. Generate text **without** a cache, then **with** one, prove the outputs are identical, and measure latency, tokens/s and memory. Then replace the concatenating cache with a preallocated one and see why production code never calls `torch.cat` per step.

All code in this part was run and checked: the cached and uncached generators produce identical tokens, and the static cache matches the concatenating cache. The files are `kv_from_scratch.py`, `static_cache.py` and `profile_decode.py` in the companion code folder.

14.1 The model

A GPT-style decoder: token + learned position embeddings, N blocks of (pre-LayerNorm → causal self-attention → residual → pre-LayerNorm → MLP → residual), final LayerNorm, LM head. Default size: $d = 256$, 8 heads ($d_h = 32$), 4 layers, vocab 1,000. Random weights are fine: we're testing mechanics, not language quality.

```

"""Part 14: a tiny decoder-only Transformer, generation with and without a KV cache."""

import math, time, torch, torch.nn as nn

class Cfg:
    vocab, d_model, n_heads, n_layers, d_ff, max_len = 1000, 256, 8, 4, 1024, 4096

class Attention(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.h, self.hd = c.n_heads, c.d_model // c.n_heads
        self.qkv = nn.Linear(c.d_model, 3 * c.d_model, bias=False)
        self.out = nn.Linear(c.d_model, c.d_model, bias=False)

    def forward(self, x, past=None):
        B, T, D = x.shape # T = new tokens this call
        q, k, v = self.qkv(x).split(D, dim=-1) # each [B, T, D]
        # [B, T, D] → [B, heads, T, head_dim]
        q, k, v = (t.view(B, T, self.h, self.hd).transpose(1, 2) for t in (q, k, v))
        if past is not None: # past = (K, V), each [B, h, P, hd]
            k = torch.cat([past[0], k], dim=2) # [B, h, P+T, hd]
            v = torch.cat([past[1], v], dim=2)
        S = k.shape[2]
        P = S - T # S = total keys, P = cached keys
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.hd) # [B, h, T, S]
        qpos = torch.arange(P, P + T, device=x.device)[ :, None] # absolute query positions
        kpos = torch.arange(S, device=x.device)[None, :]
        scores = scores.masked_fill(kpos > qpos, float("-inf")) # causal mask
        y = scores.softmax(-1) @ v # [B, h, T, hd]
        y = y.transpose(1, 2).reshape(B, T, D)

```

```

        return self.out(y), (k, v) # return the grown cache

class Block(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(c.d_model), nn.LayerNorm(c.d_model)
        self.attn = Attention(c)
        self.mlp = nn.Sequential(
            nn.Linear(c.d_model, c.d_ff), nn.GELU(), nn.Linear(c.d_ff, c.d_model)
        )

    def forward(self, x, past=None):
        a, kv = self.attn(self.ln1(x), past)
        x = x + a
        return x + self.mlp(self.ln2(x)), kv

class TinyGPT(nn.Module):
    def __init__(self, c=Cfg):
        super().__init__()
        self.tok = nn.Embedding(c.vocab, c.d_model)
        self.pos = nn.Embedding(c.max_len, c.d_model)
        self.blocks = nn.ModuleList(Block(c) for _ in range(c.n_layers))
        self.ln_f = nn.LayerNorm(c.d_model)
        self.head = nn.Linear(c.d_model, c.vocab, bias=False)

    def forward(self, idx, past_key_values=None):
        B, T = idx.shape
        P = 0 if past_key_values is None else past_key_values[0][0].shape[2]
        pos = torch.arange(P, P + T, device=idx.device) # positions continue after the cache
        x = self.tok(idx) + self.pos(pos)
        new_past = []
        for i, blk in enumerate(self.blocks):
            x, kv = blk(x, None if past_key_values is None else past_key_values[i])
            new_past.append(kv)
        return self.head(self.ln_f(x)), new_past

@torch.no_grad()
def generate_no_cache(model, idx, n_new):
    for _ in range(n_new):
        logits, _ = model(idx) # re-run the WHOLE sequence
        nxt = logits[:, -1].argmax(-1, keepdim=True)
        idx = torch.cat([idx, nxt], dim=1)
    return idx

@torch.no_grad()
def generate_with_cache(model, idx, n_new):
    logits, past = model(idx) # prefill: whole prompt, builds cache
    out = [idx]
    for _ in range(n_new):
        nxt = logits[:, -1].argmax(-1, keepdim=True) # [B, 1]
        out.append(nxt)
        logits, past = model(nxt, past) # decode: ONE token in, cache grows by 1
    return torch.cat(out, dim=1)

def bench(fn, model, idx, n_new, dev):
    if dev == "cuda":
        torch.cuda.synchronize()
        torch.cuda.reset_peak_memory_stats()
    t0 = time.perf_counter()
    out = fn(model, idx, n_new)
    if dev == "cuda":
        torch.cuda.synchronize()
    dt = time.perf_counter() - t0
    peak = torch.cuda.max_memory_allocated() / 2**20 if dev == "cuda" else float("nan")

```

```

return out, dt, peak

if __name__ == "__main__":
    torch.manual_seed(0)
    dev = "cuda" if torch.cuda.is_available() else "cpu"
    model = TinyGPT().to(dev).eval()
    # correctness first: both paths must produce identical tokens
    p = torch.randint(0, Cfg.vocab, (1, 32), device=dev)
    assert torch.equal(generate_no_cache(model, p, 20), generate_with_cache(model, p, 20))
    print("outputs identical: OK")
    for prompt_len in (128, 512):
        idx = torch.randint(0, Cfg.vocab, (1, prompt_len), device=dev)
        for name, fn in (("no cache", generate_no_cache), ("kv cache", generate_with_cache)):
            fn(model, idx, 8) # warm-up
            _, dt, peak = bench(fn, model, idx, 256, dev)
            print(
                f"prompt={prompt_len:4d} {name:9s} {dt*1e3:8.1f} ms "
                f"{256/dt:8.1f} tok/s peak {peak:.1f} MiB"
            )

```

14.2 The important lines, explained

Line	What it does and why it matters
<code>q, k, v = self.qkv(x).split(D, -1)</code>	one fused projection for Q, K and V. During decode x has $T = 1$ row, so this is the "compute one new row" step from Part 2.
<code>.view(B,T,h,hd).transpose(1,2)</code>	$[B, T, D] \rightarrow [B, \text{heads}, T, \text{head_dim}]$: the layout the cache is stored in.
<code>k = torch.cat([past[0], k], dim=2)</code>	the cache write : append the new key rows along the sequence axis (dim 2). Correct but allocates a new tensor and copies the old cache every step (§14.5).
<code>S = k.shape[2]; P = S - T</code>	P = how many tokens were already cached; S = total keys after appending.
<code>qpos = arange(P, P+T), kpos = arange(S), masked_fill(kpos > qpos)</code>	mask handling in one rule for all phases . Prefill ($P = 0$): the usual lower triangle. Decode ($T = 1, P = t-1$): $qpos = t-1$ and every $kpos \leq t-1$, so nothing is masked: the new token may see the whole cache.
<code>return self.out(y), (k, v)</code>	each attention layer returns its grown cache; the model collects one (K, V) pair per layer: that list is <code>past_key_values</code> .
<code>P = past_key_values[0][0].shape[2] pos = arange(P, P+T)</code>	position handling : new tokens get positions that continue after the cache. Forget this and every decoded token thinks it's at position 0: silent garbage.
<code>logits, past = model(nxt, past)</code>	the decode loop feeds one token ($[B, 1]$) plus the cache, and gets back one logit row plus a cache that's one longer.
<code>generate_no_cache: model(idx)</code>	the baseline re-runs the entire growing sequence every step: Part 1's "no notebook" world.
<code>assert torch.equal(...)</code>	correctness first : greedy outputs must match token for token. Always verify an optimization before timing it.
<code>torch.cuda.synchronize()</code>	GPU calls are asynchronous; without syncing, timers measure kernel launch, not execution.

14.3 Run it and read the results

```
$ python kv_from_scratch.py
outputs identical: OK
prompt= 128 no cache      4707.2 ms      54.4 tok/s    peak nan MiB
prompt= 128 kv cache      491.6 ms      520.8 tok/s   peak nan MiB
prompt= 512 no cache     23202.8 ms     11.0 tok/s    peak nan MiB
prompt= 512 kv cache      667.2 ms     383.7 tok/s   peak nan MiB
```

Example output from a CPU-only run (no GPU memory numbers). Your RTX 4050 will be much faster and will report peak MiB. The shape of the results is what to check.

- **The cache wins ~10x at prompt 128 and ~35x at 512 for 256 new tokens.** The no-cache cost grows with the whole sequence length on every step; the cached cost barely depends on it.
- **Cached tok/s drops slightly with a longer prompt (521 → 384):** each step reads a longer cache. That's Part 1's "re-reading remains", measured.
- **On a GPU, peak memory with cache is higher by exactly the cache size:** $2 \times \text{layers} \times \text{heads} \times \text{head_dim} \times \text{tokens} \times \text{bytes}$.

Exercise: predict before you run

For your GPU run in FP16 (`model.half()`), predict the cache size at prompt 512 + 256 new tokens: $2 \times 4 \times 8 \times 32 \times 768 \times 2$ bytes = ? Then compare with the difference in `max_memory_allocated` between the two runs.

14.4 What to measure, and how

Metric	How to measure correctly
latency	wall time around the generate call, with <code>torch.cuda.synchronize()</code> before starting and stopping the timer; warm up first (the first CUDA call pays for context creation and kernel loading)
tokens/s	new tokens ÷ latency (report per-sequence and batch-total separately)
TTFT vs ITL	time the prefill call separately from the decode loop (Part 16's harness does this)
memory	<code>torch.cuda.reset_peak_memory_stats()</code> before, <code>torch.cuda.max_memory_allocated()</code> after
variance	repeat ≥ 5 runs, report the median and p95, fix the seed, keep the laptop on AC power

14.5 Upgrade: a preallocated (static) cache

`torch.cat` allocates a new $[B, h, t+1, d_h]$ tensor and copies t old rows **every step, every layer**. Total bytes copied over a generation $\propto T^2$. In a CPU profile of this model, `aten::cat` took ~19% of runtime. The fix: allocate the full buffer once and write in place.

```

"""Part 14b: a preallocated (static) KV cache: write into a fixed buffer instead of torch.cat."""

import math, torch
from kv_from_scratch import TinyGPT, Cfg, generate_with_cache

class StaticKVCache:
    def __init__(self, n_layers, B, h, max_len, hd, dtype, device):
        shape = (n_layers, B, h, max_len, hd) # one big slab, allocated ONCE
        self.k = torch.zeros(shape, dtype=dtype, device=device)
        self.v = torch.zeros(shape, dtype=dtype, device=device)
        self.len = 0 # tokens written so far

    def write(self, layer, k_new, v_new): # k_new: [B, h, T, hd]
        T = k_new.shape[2]
        self.k[layer, :, :, self.len : self.len + T] = k_new # in-place write, no copy of old K
        self.v[layer, :, :, self.len : self.len + T] = v_new
        return self.k[layer, :, :, : self.len + T], self.v[layer, :, :, : self.len + T] # views

@torch.no_grad()
def forward_static(model, idx, cache):
    B, T = idx.shape
    P = cache.len
    x = model.tok(idx) + model.pos(torch.arange(P, P + T, device=idx.device))
    for li, blk in enumerate(model.blocks):
        a = blk.attn
        h = blk.ln1(x)
        D = h.shape[-1]
        q, k, v = (t.view(B, T, a.h, a.hd).transpose(1, 2) for t in a.qkv(h).split(D, dim=-1))
        K, V = cache.write(li, k, v) # [B, h, P+T, hd]
        s = q @ K.transpose(-2, -1) / math.sqrt(a.hd)
        qpos = torch.arange(P, P + T, device=idx.device)[ :, None]
        s = s.masked_fill(torch.arange(P + T, device=idx.device)[None, :] > qpos, float("-inf"))
        y = (s.softmax(-1) @ V).transpose(1, 2).reshape(B, T, D)
        x = x + a.out(y)
        x = x + blk.mlp(blk.ln2(x))
    cache.len += T
    return model.head(model.ln_f(x))

@torch.no_grad()
def generate_static(model, idx, n_new):
    c = Cfg
    B = idx.shape[0]
    cache = StaticKVCache(
        c.n_layers,
        B,
        c.n_heads,
        idx.shape[1] + n_new,
        c.d_model // c.n_heads,
        next(model.parameters()).dtype,
        idx.device,
    )
    logits = forward_static(model, idx, cache)
    out = [idx]
    for _ in range(n_new):
        nxt = logits[:, -1].argmax(-1, keepdim=True)
        out.append(nxt)
        logits = forward_static(model, nxt, cache)
    return torch.cat(out, 1)

if __name__ == "__main__":
    torch.manual_seed(0)
    m = TinyGPT().eval()

```

```
p = torch.randint(0, Cfg.vocab, (2, 40))
assert torch.equal(generate_static(m, p, 30), generate_with_cache(m, p, 30))
print("static cache matches concat cache: OK")
```

- `self.k[layer, :, :, self.len:self.len+T] = k_new`: **an in-place slice write, with no reallocation and no copy of old rows.**
- `return self.k[layer, :, :, :self.len+T]`: **a view of the filled prefix. Attention reads only valid positions.**
- **Static shapes also make the decode step capturable by CUDA graphs / `torch.compile`, which removes Python and kernel-launch overhead: a big deal for small models on fast GPUs (Part 17).**
- **The cost: memory for T_{max} is reserved up front. For one user that's fine. For a server it's the reservation waste from Part 7, which is why engines page instead.**

14.6 Bugs you will write (everyone does)

Bug	Symptom	Fix
positions restart at 0 on each decode step	first token fine, then repetitive or garbage text	offset positions by the cache length
overwriting instead of appending	cache length stays 1; model "forgets" the prompt	concat or write at index <code>len</code> , then advance <code>len</code>
causal mask applied with wrong offset during decode	new token can't see some of the cache → subtly worse outputs, no crash	build the mask from absolute positions (<code>qpos</code> vs <code>kpos</code>)
padding in batched prompts ignored	left-padded sequences attend to pad tokens	add a padding mask and adjust positions per sequence
cache shared between two generations	the second request continues the first one's story	fresh cache per sequence (or explicit prefix reuse)
timing without synchronize	impossibly fast numbers	synchronize around timers

Checkpoint · Part 14

Recall

- 1 What does `past_key_values` contain in this implementation, and what's its shape per layer?
- 2 How are positions computed for a decode step?
- 3 Why is no mask needed for a single-token decode step, even though the code applies one?

Reasoning

- 4 Why must you verify `torch.equal` on greedy outputs before benchmarking? What would sampling break?

- 5 Why does the concatenating cache cost $O(T^2)$ bytes of copying over a generation, and why doesn't that appear in FLOP counts?

Practical engineering

- 6 Your cached generator is only 1.3× faster than the uncached one on a GPU at prompt length 32. Give two reasons, and say how you'd make the benchmark show the real effect.

Coding exercise

- 7 Add **batched prompts of different lengths** to `kv_from_scratch.py` using left padding: build an attention mask that hides pad tokens, and per-sequence position ids. Verify each sequence's output matches running it alone.

Handwritten note

A working KV cache is three things done right: append KN per layer, offset positions by the cache length, mask by absolute positions. Verify equality first, then measure.

What do you think happens next?

Hugging Face's `use_cache=True` does the same thing. What object does it hand you, and what are its shapes for a real GQA model?

PART 15 · Module E: Hands-on

Inspect a Real Model

Goal of this part

Open up the KV cache of a real Hugging Face model: what `use_cache=True` returns, the tensor shapes for a GQA model, how the cache grows step by step, and how it compares with the way production engines store KV.

15.1 What `use_cache` and `past_key_values` mean

- `model(input_ids, use_cache=True)` returns logits and `past_key_values`: the KV of every layer for every token processed so far.
- Passing that object back (`model(next_ids, past_key_values=cache)`) runs only the new tokens, attends over the cached ones, and **appends** the new KV.
- In current transformers versions the object is a **Cache** class, by default a `DynamicCache` (it grows by concatenation, exactly like Part 14). Older versions returned a tuple of (K, V) tuples, one per layer. The inner attributes have changed across versions, so the script below reads them defensively.
- `model.generate()` manages all of this for you; it uses the cache by default.

15.2 The inspection script

```

"""Part 15: look inside a real Hugging Face KV cache.
python hf_inspect.py                # Qwen/Qwen2.5-0.5B-Instruct (downloads ~1 GB)
python hf_inspect.py --tiny          # random tiny Llama, no download
"""

import sys, torch
from transformers import AutoModelForCausalLM, AutoTokenizer, LlamaConfig, LlamaForCausalLM

def layer_kv(cache, i):
    """Return (K, V) for layer i across transformers versions."""
    if hasattr(cache, "layers"): # recent versions: list of DynamicLayer
        return cache.layers[i].keys, cache.layers[i].values
    if hasattr(cache, "key_cache"): # ~4.36 .. 4.5x
        return cache.key_cache[i], cache.value_cache[i]
    return cache[i] # legacy tuple of (K, V) tuples

def cache_bytes(cache, n_layers):
    return sum(
        k.numel() * k.element_size() + v.numel() * v.element_size()
        for k, v in (layer_kv(cache, i) for i in range(n_layers))
    )

dev = "cuda" if torch.cuda.is_available() else "cpu"
if "--tiny" in sys.argv:

```

```

    cfg = LlamaConfig(
        vocab_size=1000,
        hidden_size=256,
        intermediate_size=512,
        num_hidden_layers=4,
        num_attention_heads=8,
        num_key_value_heads=2,
        max_position_embeddings=512,
    )
    model, ids = LlamaForCausalLM(cfg).to(dev).eval(), torch.randint(
        0, 1000, (1, 10), device=dev
    )
else:
    name = "Qwen/Qwen2.5-0.5B-Instruct"
    tok = AutoTokenizer.from_pretrained(name)
    model = (
        AutoModelForCausalLM.from_pretrained(
            name, dtype=torch.float16 if dev == "cuda" else torch.float32
        )
        .to(dev)
        .eval()
    )
    ids = tok("The cat sat on the mat because", return_tensors="pt").input_ids.to(dev)
    cfg = model.config

L, kvh = cfg.num_hidden_layers, cfg.num_key_value_heads
hd = getattr(cfg, "head_dim", None) or cfg.hidden_size // cfg.num_attention_heads
print(f"layers={L}  q_heads={cfg.num_attention_heads}  kv_heads={kvh}  head_dim={hd}")

with torch.no_grad():
    out = model(ids, use_cache=True)  # PREFILL
    cache = out.past_key_values
    k0, v0 = layer_kv(cache, 0)
    print("cache type:", type(cache).__name__, "| seq len:", cache.get_seq_length())
    print(
        "layer 0 K:",
        tuple(k0.shape),
        " V:",
        tuple(v0.shape),
        " # [batch, kv_heads, seq, head_dim]",
    )

for step in range(3):  # DECODE: one token at a time
    nxt = out.logits[:, -1:].argmax(-1)  # [1, 1]
    with torch.no_grad():
        out = model(nxt, past_key_values=cache, use_cache=True)
        cache = out.past_key_values
        print(
            f"after decode step {step+1}: seq len = {cache.get_seq_length()}, "
            f"K0 shape = {tuple(layer_kv(cache, 0)[0].shape)}"
        )

T = cache.get_seq_length()
measured = cache_bytes(cache, L)
formula = 2 * L * kvh * hd * T * k0.element_size()
print(f"bytes: measured={measured:,}  formula 2*L*kv_heads*head_dim*T*bytes={formula:,}")

```

Tested with a recent transformers release. On older versions, replace `dtype=` with `torch_dtype=` in `from_pretrained`. `--tiny` builds a random 4-layer Llama with 8 query heads and 2 KV heads, so you can check the mechanics offline.

15.3 What you'll see

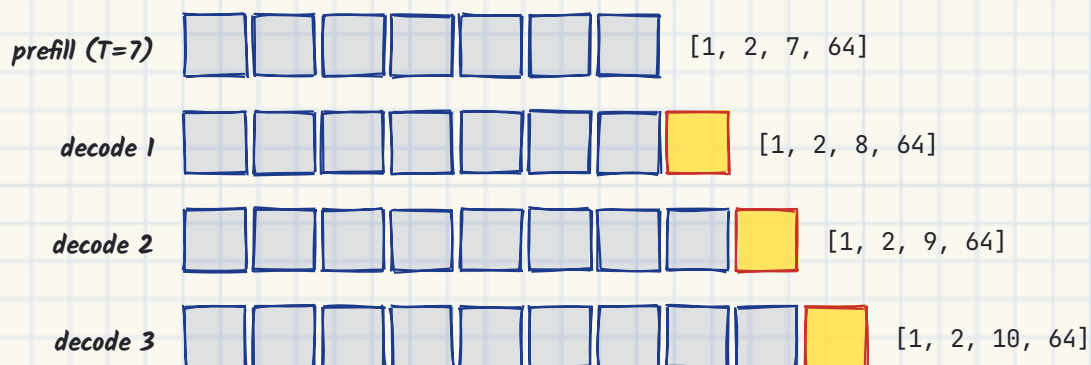
```
$ python hf_inspect.py --tiny
layers=4 q_heads=8 kv_heads=2 head_dim=32
cache type: DynamicCache | seq len: 10
layer 0 K: (1, 2, 10, 32) V: (1, 2, 10, 32) # [batch, kv_heads, seq, head_dim]
after decode step 1: seq len = 11, K0 shape = (1, 2, 11, 32)
after decode step 2: seq len = 12, K0 shape = (1, 2, 12, 32)
after decode step 3: seq len = 13, K0 shape = (1, 2, 13, 32)
bytes: measured=26,624 formula 2*L*kv_heads*head_dim*T*bytes=26,624
```

With the real **Qwen2.5-0.5B-Instruct** you'll see 24 layers, 14 query heads, **2 KV heads**, head_dim 64, so the K tensor per layer is **[1, 2, T, 64]**, not [1, 14, T, 64]. That's GQA in the flesh: the cache stores 2 heads while attention uses 14. In FP16, $2 \times 24 \times 2 \times 64 \times 2 = 12,288$ bytes per token, and the script's measured bytes will match the formula exactly.

A detail that trips people up

Call `generate(ids, max_new_tokens=5)` on a 10-token prompt and inspect the returned cache: its length is **14**, not 15. The last generated token was sampled from logits but **never fed back through the model**, so its KV were never computed. The cache always lags the text by one token after generation. (Verified on the tiny model.)

15.4 Watching the cache grow



layer-0 K tensor for Qwen2.5-0.5B: the "seq" axis grows by one per decode step, every other axis stays fixed

15.5 Other cache classes worth knowing

Cache	Idea	Use when
DynamicCache	grows by concatenation	default; simple, flexible
StaticCache (cache_implementation="static")	preallocated to a max length, in-place writes (Part 14.5)	with torch.compile / CUDA graphs for low-latency decode
sliding-window / hybrid caches	rolling buffers for windowed layers (Part 13)	Mistral-, Gemma-style models

quantized caches	store K/V in low bits via a backend library	fitting longer contexts on small GPUs (Part II)
offloaded caches	keep most layers' KV on CPU, prefetch per layer	long contexts that don't fit in VRAM, when latency is less critical

Names and options evolve; check the "Caching" / "KV cache strategies" page in the transformers docs for your version.

15.6 How production engines represent the cache differently

Aspect	Hugging Face (research/dev)	vLLM-style engine (production)
unit	one tensor per layer per request	one global pool of fixed-size blocks per layer, shared by all requests
layout	$[B, H_{kv}, T, d_h]$, padded across the batch	$[num_blocks, block_size, H_{kv}, d_h]$ (or a kernel-specific variant) + block tables
growth	concatenate (copy) or static max length	allocate a block every <code>block_size</code> tokens; no copies
batching	padded rectangle; everyone in the batch runs together	ragged: sequences of any length in one flattened batch, joining and leaving each step
sharing	manual (copy the object)	ref-counted blocks, prefix hashing, copy-on-write
memory owner	PyTorch caching allocator	the engine's own block manager, pool preallocated at startup
kernels	standard SDPA / FlashAttention on dense tensors	paged attention kernels that read through block tables; fused cache-write kernels

Checkpoint · Part 15

Recall

- 1 What does `past_key_values` hold, and what's the default cache class in current transformers?
- 2 For Qwen2.5-0.5B, what's the per-layer K shape after processing 100 tokens with batch 1?
- 3 Why is the cache one token shorter than the generated text after `generate()`?

Reasoning

- 4 Why does the cache's head dimension show H_{kv} while the attention output uses H_q ? Where does the expansion happen?

5 Why do production engines avoid a padded $[B, H, T, d_h]$ tensor for batched serving?

Practical engineering

6 You pass the same `DynamicCache` object to two different continuations to "reuse the prefix", and the second continuation's output is wrong. What happened, and how do you fix it?

Coding exercise

7 Extend `hf_inspect.py` to print, after each decode step, the measured cache bytes and `torch.cuda.memory_allocated()`. Plot both against sequence length for 200 steps. Are they parallel lines? Explain any gap.

Handwritten note

HF's cache is a per-layer $[B, H_{kv}, T, d_h]$ pair that grows by one row per step: GQA shows up as $H_{kv} < H_q$. Engines replace it with a shared paged pool.

What do you think happens next?

You can now build and inspect caches. Time to turn that into a benchmark project you could show an interviewer.

PART 16 · Module E: Hands-on

KV Cache Lab

Understanding and Optimizing LLM Inference Memory · a portfolio project

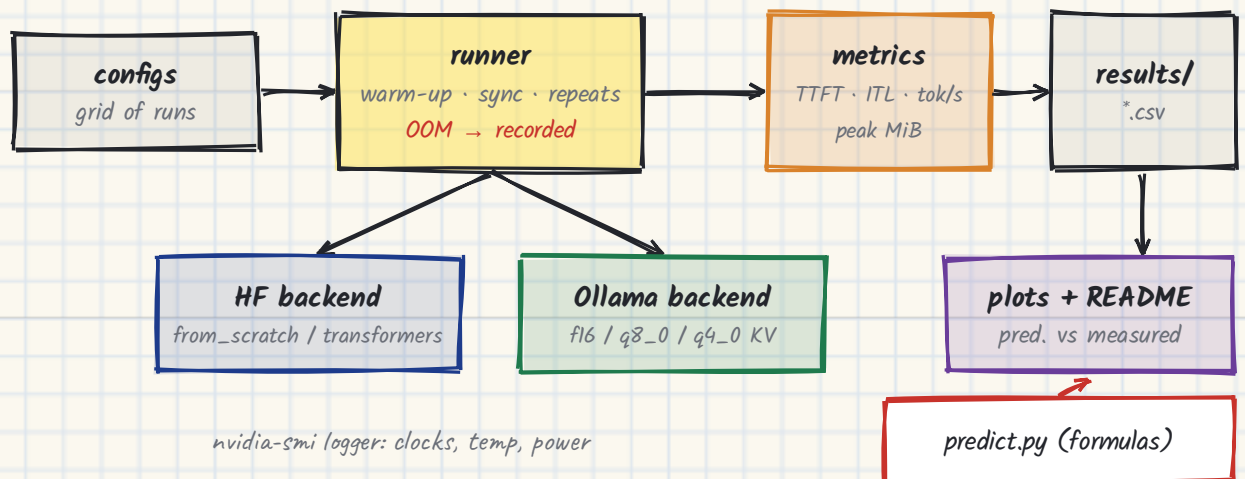
Goal of this part

Design and build a benchmark project that shows real inference-engineering skill: controlled experiments, correct measurement, predictions checked against the formulas from Parts 3–11, and conclusions you can defend in an interview. Sized for an RTX 4050 laptop GPU (6 GB).

16.1 What the project proves about you

- You can **predict** memory and latency from first principles (Part 5 formula, Part 3 bandwidth model) and then **measure** them correctly.
- You understand **TTFT vs ITL vs throughput**, and why they move differently.
- You can explain every graph with a bottleneck: compute, bandwidth, capacity or overhead.
- You know the difference between a model API call and an inference system.

16.2 Architecture



```

kv-cache-lab/
├── README.md           # the story: question → method → results → conclusions
├── configs/            # experiment grids (model, dtype, ctx, batch, cache on/off)
├── lab/
│   ├── from_scratch.py # Part 14 model + static cache (ground truth mechanics)
│   ├── bench_hf.py     # Hugging Face backend: TTFT, ITL, tok/s, peak memory
│   ├── bench_ollama.py # Ollama backend: KV quantization (f16 / q8_0 / q4_0)
│   ├── predict.py      # formula-based predictions (KV bytes, step time)
│   └── plot_lab.py     # graphs
├── results/            # raw CSVs (never edit by hand) + system info
├── figures/            # generated PNGs
└── notebooks/analysis.ipynb # predictions vs measurements

```

16.3 Fitting the experiments into 6 GB

Model	FP16 weights	KV / token	KV budget*	tokens that fit	Role in the lab
Qwen2.5-0.5B-Instruct	~1.0 GB	12 KiB	~3.6 GB	~294K	main workhorse: big batch × context sweeps
Qwen2.5-1.5B-Instruct	~3.1 GB	28 KiB	~1.5 GB	~53K	hits the memory wall early: OOM boundary experiments
TinyLlama-1.1B (GQA-4)	~2.2 GB	22 KiB	~2.4 GB	~107K	a Llama-architecture comparison point
GPT-2 small (MHA)	~0.25 GB	36 KiB	~4.35 GB	~118K	MHA vs GQA: 3× the KV per token of Qwen-0.5B with ¼ the weights (but max context 1,024)

* 90% of 6 GB minus weights minus ~0.8 GB for CUDA context and activations. Measure your own baseline with an empty model loaded; laptop drivers and display output vary.

16.4 The experiment plan

#	Experiment	Sweep	Hypothesis (write it BEFORE running)
1	no cache vs cache	ctx 128-1K, batch 1	no-cache ITL grows ~linearly with ctx; cached ITL nearly flat
2	cache, context sweep	ctx 256 → 16K	$ITL = a + b \cdot ctx$, with $b \approx kv_{tok} \div \text{bandwidth}$
3	batch sweep	$B = 1, 2, 4, 8, 16$	tok/s rises sub-linearly; ITL rises; memory $\propto B \cdot ctx$
4	precision (weights)	FP16 vs BF16 (vs 4-bit weights in Ollama)	smaller weights → faster decode at short ctx
5	precision (KV)	Ollama f16 / q8_0 / q4_0	KV memory halves/quarters; speed gain grows with ctx
6	prefix reuse	shared prefix of 2K-8K + short suffix	warm TTFT $\approx$ suffix-only prefill; ITL unchanged
7	memory vs prediction	all runs	peak – baseline $\approx 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b \cdot B \cdot T$ (+ logits/activation spikes)

8	TTFT	prompt 256 → 16K	~linear, then super-linear (attention n^2) at long prompts
9	ITL distribution	p50 / p95 / p99	tails from thermal throttling, allocator growth, Python overhead
10	throughput vs latency frontier	batch × ctx grid	a Pareto curve: more batch = more tok/s, worse ITL

16.5 Metrics, defined precisely

Metric	Definition used in the lab
TTFT	request start → first token id available (prefill + 1 sample), synchronized
ITL	time between consecutive tokens during decode; report mean, p50, p95
decode tok/s	$B \times \text{generated tokens} \div \text{decode time}$ (per-sequence = that $\div B$)
peak memory	<code>torch.cuda.max_memory_allocated()</code> (and reserved), minus the loaded-model baseline
effective bandwidth	predicted bytes per step (weights + KV) $\div$ measured ITL. Compare with ~192 GB/s

16.6 Benchmark methodology checklist

- ☐ Plugged into AC power, performance mode, fans up; log `nvidia-smi --query-gpu=clocks.sm,clocks.mem,temperature.gpu,power.draw --format=csv -l 1` during runs.
- ☐ Close other GPU apps (browsers use the GPU). Record driver, CUDA, torch and transformers versions in `results/system.json`.
- ☐ Warm up every configuration; repeat ≥ 5 times; report median and p95; keep raw rows.
- ☐ Greedy decoding, fixed seeds, fixed prompt tokens (random ids are fine for systems tests), **fixed number of new tokens** (ignore EOS).
- ☐ `torch.cuda.synchronize()` around every timer; `logits_to_keep=1` during prefill so full-vocab logits don't dominate memory.
- ☐ Catch `torch.OutOfMemoryError` and record "OOM" as a result. The OOM boundary is data.
- ☐ Change one variable at a time; write the hypothesis first.

The logits trap: the prefill of [8, 8192] tokens with Qwen's 151,936-token vocabulary would produce an [8, 8192, 151936] logits tensor, about 20 GB in FP16, far more than the KV cache. Asking for only the last position's logits avoids it. Always check what else scales with $B \times T$.

16.7 Hugging Face harness

```
"""Part 16: KV Cache Lab benchmark harness (Hugging Face backend).
python bench_hf.py --model Qwen/Qwen2.5-0.5B-Instruct --out results.csv
python bench_hf.py --tiny # smoke test, random tiny model, CPU ok
"""
```

```
import argparse, copy, csv, statistics, time, torch
from transformers import AutoModelForCausalLM, LlamaConfig, LlamaForCausalLM
```

```

def sync(dev):
    if dev == "cuda":
        torch.cuda.synchronize()

@torch.no_grad()
def run_once(model, prompt, n_new, use_cache, dev, prefix_cache=None):
    """Greedy-decode n_new tokens. Returns dict of metrics. prompt: [B, T] token ids."""
    if dev == "cuda":
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()
    sync(dev)
    t0 = time.perf_counter()
    if use_cache:
        past = copy.deepcopy(prefix_cache) if prefix_cache is not None else None
        out = model(
            prompt, past_key_values=past, use_cache=True, logits_to_keep=1
        ) # prefill; keep last logits only
        past = out.past_key_values
    else:
        out, seq = model(prompt, use_cache=False, logits_to_keep=1), prompt
    nxt = out.logits[:, -1:].argmax(-1)
    sync(dev)
    ttft = time.perf_counter() - t0 # first token is ready
    itl, t_prev = [], time.perf_counter()
    for _ in range(n_new - 1):
        if use_cache:
            out = model(nxt, past_key_values=past, use_cache=True)
            past = out.past_key_values
        else:
            seq = torch.cat([seq, nxt], 1)
            out = model(seq, use_cache=False, logits_to_keep=1) # re-run all
        nxt = out.logits[:, -1:].argmax(-1)
        sync(dev)
        now = time.perf_counter()
        itl.append(now - t_prev)
        t_prev = now
    total = time.perf_counter() - t0
    B = prompt.shape[0]
    return dict(
        ttft_ms=ttft * 1e3,
        itl_ms=statistics.mean(itl) * 1e3 if itl else 0,
        itl_ms_p95=sorted(itl)[int(0.95 * (len(itl) - 1))] * 1e3 if itl else 0,
        tok_s=B * n_new / total,
        peak_mib=torch.cuda.max_memory_allocated() / 2**20 if dev == "cuda" else float("nan"),
    )

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", default="Qwen/Qwen2.5-0.5B-Instruct")
    ap.add_argument("--tiny", action="store_true")
    ap.add_argument("--out", default="results.csv")
    ap.add_argument("--new", type=int, default=64)
    a = ap.parse_args()
    dev = "cuda" if torch.cuda.is_available() else "cpu"
    if a.tiny:
        cfg = LlamaConfig(
            vocab_size=1000,
            hidden_size=256,
            intermediate_size=512,
            num_hidden_layers=4,
            num_attention_heads=8,
            num_key_value_heads=2,
            max_position_embeddings=4096,
        )
    models = {"fp32": LlamaForCausalLM(cfg).to(dev).eval()}
    V = 1000
    ctxs, batches, a.new = [64, 256], [1, 2], 16

```

```

else:
    dt = (
        {"fp16": torch.float16, "bf16": torch.bfloat16}
        if dev == "cuda"
        else {"fp32": torch.float32}
    )
    models = {
        k: AutoModelForCausalLM.from_pretrained(a.model, dtype=v).to(dev).eval()
        for k, v in dt.items()
    }
    V = next(iter(models.values())).config.vocab_size
    ctxs, batches = [256, 1024, 4096, 8192], [1, 2, 4, 8]
    rows = []
    for dname, model in models.items():
        for T in ctxs:
            for B in batches:
                prompt = torch.randint(0, V, (B, T), device=dev)
                for use_cache in (True, False):
                    if not use_cache and (T > 1024 or B > 1):
                        continue # no-cache is too slow beyond this
                    run_once(model, prompt[:, :32], 4, use_cache, dev) # warm-up
                    try:
                        m = run_once(model, prompt, a.new, use_cache, dev)
                    except torch.OutOfMemoryError:
                        m = dict(ttft_ms="OOM")
                    rows.append(
                        dict(exp="cache", dtype=dname, ctx=T, batch=B, cache=use_cache, **m)
                    )
                print(rows[-1])
            # prefix reuse: shared "system prompt" + short user suffix
            sys_len, user_len = ctxs[-1] // 2, 32
            sys_ids = torch.randint(0, V, (1, sys_len), device=dev)
            user = torch.randint(0, V, (1, user_len), device=dev)
            with torch.no_grad():
                prefix = model(
                    sys_ids, use_cache=True, logits_to_keep=1
                ).past_key_values # computed ONCE
            cold = run_once(model, torch.cat([sys_ids, user], 1), a.new, True, dev)
            warm = run_once(
                model, user, a.new, True, dev, prefix_cache=prefix
            ) # suffix-only prefill
            for tag, m in (("cold", cold), ("warm", warm)):
                rows.append(
                    dict(
                        exp="prefix_" + tag,
                        dtype=dname,
                        ctx=sys_len + user_len,
                        batch=1,
                        cache=True,
                        **m
                    )
                )
            print(rows[-1])
    keys = sorted(
        {k for r in rows for k in r},
        key=lambda k: list(rows[0]).index(k) if k in rows[0] else 99,
    )
    with open(a.out, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=keys)
        w.writeheader()
        w.writerows(rows)
    print("wrote", a.out)

if __name__ == "__main__":
    main()

```

16.8 Ollama harness (KV quantization experiments)

"""Part 16: KV Cache Lab, Ollama backend. Measures TTFT, ITL and tok/s via the streaming API. Start the server with the KV type you want to test, e.g.

```
OLLAMA_FLASH_ATTENTION=1 OLLAMA_KV_CACHE_TYPE=q8_0 ollama serve      (f16 | q8_0 | q4_0)
then: python bench_ollama.py --model qwen2.5:1.5b --kv q8_0 --out ollama.csv
"""
```

```
import argparse, csv, json, statistics, time, requests
```

```
URL = "http://localhost:11434/api/generate"
```

```
FILLER = "The quick brown fox jumps over the lazy dog. " # ~10 tokens per repeat
```

```
def run(model, prompt, num_ctx, n_predict):
    body = {
        "model": model,
        "prompt": prompt,
        "stream": True,
        "keep_alive": "10m",
        "options": {"num_ctx": num_ctx, "num_predict": n_predict, "temperature": 0, "seed": 0},
    }
    t0 = time.perf_counter()
    stamps = []
    final = {}
    with requests.post(URL, json=body, stream=True, timeout=600) as r:
        r.raise_for_status()
        for line in r.iter_lines():
            if not line:
                continue
            msg = json.loads(line)
            if msg.get("response"):
                stamps.append(time.perf_counter())
            if msg.get("done"):
                final = msg
    ttft = (stamps[0] - t0) * 1e3 if stamps else float("nan")
    gaps = [(b - a) * 1e3 for a, b in zip(stamps, stamps[1:])]
    ns = 1e9
    return dict(
        ttft_ms=ttft,
        itl_ms_mean=statistics.mean(gaps) if gaps else float("nan"),
        itl_ms_p95=sorted(gaps)[int(0.95 * (len(gaps) - 1))] if gaps else float("nan"),
        prompt_tokens=final.get("prompt_eval_count", 0), # tokens actually evaluated
        prompt_eval_ms=final.get("prompt_eval_duration", 0) / 1e6,
        gen_tokens=final.get("eval_count", 0),
        decode_tok_s=final.get("eval_count", 0) / (final.get("eval_duration", 1) / ns),
    )
```

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", default="qwen2.5:1.5b")
    ap.add_argument("--kv", default="f16")
    ap.add_argument("--out", default="ollama.csv")
    ap.add_argument("--new", type=int, default=128)
    a = ap.parse_args()
    rows = []
    for ctx_tokens in (512, 2048, 8192, 16384):
        prompt = FILLER * (ctx_tokens // 10) + "\nSummarize the text above in one sentence."
        num_ctx = ctx_tokens + a.new + 256
        run(a.model, "warm up", num_ctx, 4) # load model with this num_ctx
        cold = run(a.model, prompt, num_ctx, a.new) # full prefill
        warm = run(a.model, prompt, num_ctx, a.new) # same prompt again: prefix reuse?
        for tag, m in (("cold", cold), ("repeat", warm)):
            rows.append(dict(kv=a.kv, ctx=ctx_tokens, run=tag, **m))
            print(rows[-1])
    with open(a.out, "w", newline="") as f:
```

```
w = csv.DictWriter(f, fieldnames=list(rows[0]))
w.writeheader()
w.writerows(rows)

if __name__ == "__main__":
    main()
```

Run the server once per KV type (f16, q8_0, q4_0) and label each CSV. HTTP streaming can deliver several tokens in one chunk, so trust Ollama's own eval_count/eval_duration for decode tok/s and treat client-side ITL as indicative. If a repeated prompt shows a much smaller prompt_eval_count, the runner reused its cached prefix: that's experiment 6 in Ollama.

16.9 Plots

```
"""Part 16: turn results.csv into the four KV Cache Lab graphs."""

import sys, pandas as pd, matplotlib.pyplot as plt

df = pd.read_csv(sys.argv[1] if len(sys.argv) > 1 else "results.csv")
df = df[df.ttft_ms != "OOM"].astype({"ttft_ms": float})
c = df[df.exp == "cache"]
fig, ax = plt.subplots(2, 2, figsize=(11, 8))
for flag, g in c[c.batch == 1].groupby("cache"): # 1. ITL vs context
    ax[0, 0].plot(g.ctx, g.itl_ms_mean, "o-", label="KV cache" if flag else "no cache")
ax[0, 0].set(
    xscale="log",
    xlabel="context tokens",
    ylabel="mean ITL (ms)",
    title="Inter-token latency vs context",
)
ax[0, 0].legend()
for T, g in c[c.cache == True].groupby("ctx"): # 2. throughput vs batch
    ax[0, 1].plot(g.batch, g.tok_s, "o-", label=f"ctx={T}")
ax[0, 1].set(xlabel="batch size", ylabel="tokens/s", title="Throughput vs batch")
ax[0, 1].legend()
g = c[c.cache == True] # 3. memory vs tokens in flight
ax[1, 0].scatter(g.ctx * g.batch, g.peak_mib)
ax[1, 0].set(
    xscale="log",
    xlabel="batch x context (tokens in flight)",
    ylabel="peak MiB",
    title="Peak memory vs tokens in flight",
)
p = df[df.exp.str.startswith("prefix")] # 4. prefix reuse
ax[1, 1].bar(p.exp.str.replace("prefix_", ""), p.ttft_ms, color=["#c8322b", "#1f7a4d"])
ax[1, 1].set(ylabel="TTFT (ms)", title="Prefix reuse: cold vs warm TTFT")
fig.tight_layout()
fig.savefig("kv_lab_results.png", dpi=150)
print("saved kv_lab_results.png")
```

16.10 The graphs to produce, and what each should show

Figure	Expected shape, and the sentence it lets you say
ITL vs context (cache vs no cache)	no cache: steep line; cache: gentle slope. "The cache removes recomputation; the remaining slope is KV re-reading."

ITL vs context, fitted line	slope $b \approx kv_{tok}/BW$. "Our measured slope implies $\sim X$ GB/s effective bandwidth, $Y\%$ of the 4050's peak."
tok/s and ITL vs batch	tok/s rises and flattens; ITL rises. "Batching amortizes weight reads until KV reads dominate."
peak memory vs $B \times T$, with the formula line	measured points on (or slightly above) the predicted line. "KV memory matches $2 \cdot L \cdot H_{kv} \cdot d_p \cdot b$ per token."
KV dtype: memory and ITL vs context	q8_0 halves KV memory; speed gain grows with context. "KV quantization pays off where KV dominates traffic."
prefix reuse: cold vs warm TTFT	warm $\ll$ cold, gap $\propto$ prefix length. "Prefix caching removes prefill, not decode."
throughput-latency Pareto frontier	the operating curve a serving engineer tunes along.

16.11 README structure

1. **Title + one-line result** (e.g. "KV caching cut per-token latency $N\times$; KV quantization gave another $Y\%$ at 16K context on a 6 GB laptop GPU").
2. **Why this matters:** 3 sentences on decode being bandwidth-bound and KV being the capacity limit.
3. **Setup:** hardware, software versions, models, methodology checklist.
4. **Predictions:** the formulas and predicted numbers (before results).
5. **Results:** one figure per question, each with a 2-sentence interpretation naming the bottleneck.
6. **Predicted vs measured** table and discussion of gaps (overheads, allocator, thermal).
7. **Limitations:** single GPU, laptop thermals, random-token prompts, HF isn't a serving engine.
8. **How to reproduce:** exact commands.
9. **What I'd do next:** paged cache in `from_scratch.py`, vLLM on Linux/WSL, CUDA graphs, FP8 KV.

16.12 Key conclusions you should expect (verify, don't assume)

1. Without a cache, per-token latency grows with context; with one it is nearly flat. The speedup grows with sequence length.
2. With a cache, ITL still rises linearly with context, and the slope matches KV bytes ÷ bandwidth.
3. Measured KV memory matches the formula; GPT-2 (MHA) uses ~3× the KV per token of Qwen-0.5B (GQA) despite being ~4× smaller.
4. Batching increases total throughput but raises ITL; on 6 GB, capacity (OOM) arrives before compute saturation for the 1.5B model.
5. 8-bit KV halves KV memory with little quality change; its speed benefit appears only at longer contexts.
6. Prefix reuse slashes TTFT for long shared prefixes and leaves ITL unchanged.

Résumé bullet (fill in your numbers)

"Built KV Cache Lab, a benchmark suite on an RTX 4050 (6 GB) quantifying KV-cache effects on TTFT, ITL, throughput and memory across 4 models; validated a first-principles memory/bandwidth model to within X%; showed 8-bit KV cut memory 2× and ITL Y% at 16K context."

Checkpoint · Part 16

Recall

- 1 Which four metrics does the lab report, and how is each measured?
- 2 Why is `logits_to_keep=1` important on a 6 GB GPU?
- 3 Why do we write hypotheses before running experiments?

Reasoning

- 4 Your ITL-vs-context slope implies 90 GB/s effective bandwidth, about half of the 4050's peak. List three reasons measured bandwidth falls below peak.
- 5 Why is GPT-2 a useful comparison model even though it's much smaller than Qwen-0.5B?

Practical engineering

- 6 Your p95 ITL is 3× the median, only in long runs. What would you log to diagnose it, and what are the likely causes on a laptop?

Coding exercise

- 7 Write `predict.py`: given a HF config, dtype, batch and context, print predicted KV bytes, predicted peak memory and predicted ITL ($\text{weights} + \text{KV bytes} \div 192 \text{ GB/s}$). Add a "predicted" column to every row of `results.csv` and plot measured/predicted.

Handwritten note

A benchmark is a controlled experiment: predict from formulas, change one variable, synchronize, repeat, keep raw data, and explain every curve by its bottleneck.

What do you think happens next?

Some of your numbers will surprise you. How do you find out whether a slow step is waiting on memory, compute, or Python?

PART 17 · Module E: Hands-on

Profiling

Goal of this part

Learn the tools and, more importantly, the reasoning an inference engineer uses to decide whether a step is **compute-bound**, **memory-bandwidth-bound**, **capacity-bound** or **overhead-bound**. Then practise on diagnosis drills.

17.1 The four kinds of "slow"

Bound by	Signature	Typical in LLM inference
compute	tensor cores busy; time scales with FLOPs; halving bytes doesn't help	prefill, large-batch GEMMs
memory bandwidth	DRAM throughput near peak; time scales with bytes moved; extra FLOPs are "free"	decode: weights at small batch, KV always
memory capacity	OOMs, preemptions, small max batch while the GPU is idle	long contexts, many users, MHA models
overhead (CPU/launch)	gaps between kernels on the timeline; GPU idle; time barely changes with model size	small models, batch 1, Python loops, per-step allocations

17.2 PyTorch profiler

"""Part 17: profile prefill vs decode with torch.profiler (+ CUDA memory snapshot on GPU)."""

```
import torch
from torch.profiler import profile, ProfilerActivity, record_function
from kv_from_scratch import TinyGPT, Cfg

dev = "cuda" if torch.cuda.is_available() else "cpu"
model = TinyGPT().to(dev).eval()
if dev == "cuda":
    model = model.half()
idx = torch.randint(0, Cfg.vocab, (1, 1024), device=dev)
acts = [ProfilerActivity.CPU] + ([ProfilerActivity.CUDA] if dev == "cuda" else [])

if dev == "cuda":
    torch.cuda.memory._record_memory_history(max_entries=100_000) # allocation timeline
with torch.no_grad(), profile(activities=acts, record_shapes=True, profile_memory=True) as prof:
    with record_function("prefill"):
        logits, past = model(idx)
    with record_function("decode_64_steps"):
        for _ in range(64):
            nxt = logits[:, -1].argmax(-1, keepdim=True)
            logits, past = model(nxt, past)
if dev == "cuda":
    torch.cuda.synchronize()
```

```

sort = "cuda_time_total" if dev == "cuda" else "cpu_time_total"
print(prof.key_averages().table(sort_by=sort, row_limit=12))
prof.export_chrome_trace("trace.json") # open in chrome://tracing or ui.perfetto.dev
if dev == "cuda":
    torch.cuda.memory._dump_snapshot("mem_snapshot.pickle") # drag into pytorch.org/memory_viz
    print(torch.cuda.memory_summary(abbreviated=True))

```

- `record_function("prefill") / ("decode_64_steps")` *label regions, so the table and trace separate the two phases.*
- `key_averages().table(...)`: *which ops dominate. Look for GEMMs (`aten::mm`, `addmm`), attention kernels, and **copies** (`aten::cat`, `copy_`, `clone`).*
- `export_chrome_trace` → *open in [ui.perfetto.dev](#): a timeline of CPU ops and GPU kernels. Gaps on the GPU row = overhead-bound.*

```

-----
Name      Self CPU %   CPU total   ...   # of Calls
decode_64_steps  10.01%    340.440ms  ...      1
  prefill      2.79%    300.966ms  ...      1
  aten::matmul  1.25%    136.250ms  ...    1105
  aten::cat     18.87%    124.385ms  ...     512 ← the concatenating cache
  aten::bmm     13.70%     88.122ms  ...     520

```

Excerpt from an actual CPU run of this script on the Part 14 model. On your GPU, sort by `cuda_time_total` and you'll see kernel names instead. The lesson carries over: per-step cache copies show up clearly.

17.3 CUDA memory statistics

Call	What it tells you
<code>torch.cuda.memory_allocated()</code>	<i>bytes held by live tensors right now</i>
<code>torch.cuda.memory_reserved()</code>	<i>bytes the caching allocator holds from the driver ($\geq$ allocated). The gap is cached free blocks and fragmentation</i>
<code>torch.cuda.max_memory_allocated()</code>	<i>peak since the last <code>reset_peak_memory_stats()</code>; use it per experiment</i>
<code>torch.cuda.memory_summary()</code>	<i>allocator breakdown: large/small pools, number of allocs, OOM retries</i>
<code>torch.cuda.memory._record_memory_history()</code> + <code>_dump_snapshot()</code>	<i>every allocation with a stack trace; drag the pickle into pytorch.org/memory_viz to see a growing cache as a staircase</i>
<code>nvidia-smi</code>	<i>whole-process usage including CUDA context and the reserved pool. It will always read higher than <code>memory_allocated</code></i>

Serving engines preallocate their KV pool, so `nvidia-smi` shows ~90% used from startup regardless of load. Use the engine's own metrics (KV blocks used, e.g. vLLM's KV-cache usage gauge) to see real occupancy.

17.4 GPU utilization: the most misread number

nvidia-smi's "GPU-Util" is the fraction of time **at least one kernel was running**. A decode loop streaming memory with idle tensor cores reads 100%. So can a kernel using one SM. To know what's saturated you need **throughput counters**:

- **Achieved DRAM bandwidth** as % of peak (memory-bound if high).
- **SM / tensor-core throughput** as % of peak (compute-bound if high).
- **MFU** (model FLOPs utilization) = achieved model FLOP/s ÷ peak. Decode MFU is often in the single digits, and that's expected.

17.5 Nsight Systems and Nsight Compute (conceptually)

Nsight Systems (nsys): the timeline

Whole-program view: CPU threads, CUDA API calls, kernel launches, memcpys, NVTX ranges, over time.

Answers: Is the GPU idle between kernels? Is the CPU launch path the bottleneck? Are there unexpected host ↔ device copies?

```
nsys profile -t cuda,nvtx -o decode
python bench.py
```

Nsight Compute (ncu): one kernel, deeply

Per-kernel hardware counters: DRAM throughput, SM throughput, occupancy, cache hit rates, a **roofline** chart.

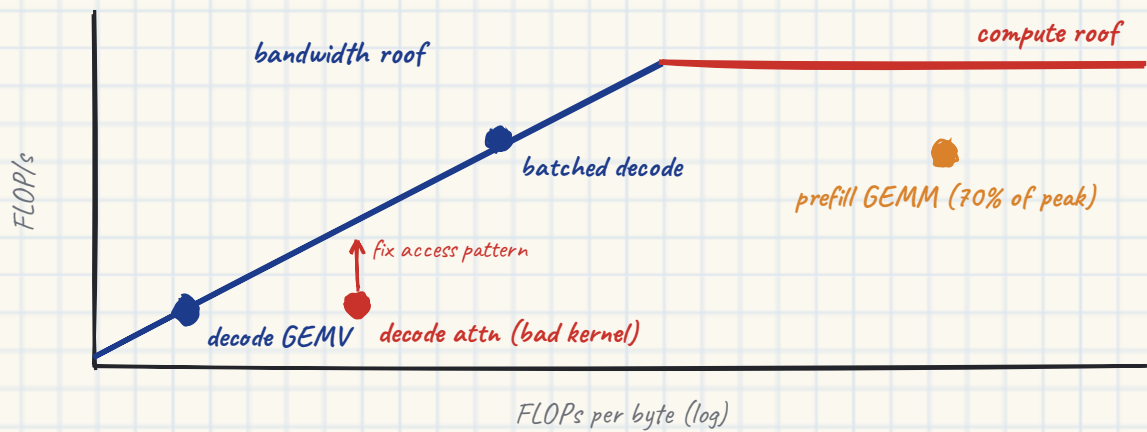
Answers: Is this attention kernel at 85% of memory bandwidth (good, bandwidth-bound) or at 30% (bad access pattern)?

```
ncu --set full -k regex:attn -c 5
python bench.py
```

On a laptop, profiling counters may need admin rights (on Windows, enabling GPU performance counters in the NVIDIA control panel). Profile short runs; ncu replays kernels many times and is slow.

17.6 Five experiments that separate compute from bandwidth

Experiment	If memory-bound...	If compute-bound...
increase batch B (short context)	step time nearly flat → throughput $\propto B$	step time $\propto B$
halve bytes (FP16 → 8-bit weights or KV)	time drops up to $\sim 2\times$	little change (or slower, from dequant)
increase context at fixed batch	time grows with KV bytes (slope = $k_{v_{tok}}/BW$)	(prefill) grows with n^2 attention FLOPs
compute intensity (FLOPs ÷ bytes) vs ridge point	intensity $\ll$ ridge	intensity $\gg$ ridge
ncu counters	DRAM % high, SM % low	SM/tensor % high



place each kernel on the roofline: distance below the roof = wasted potential; which roof = which fix

17.7 Allocation patterns to recognize

- **Staircase** in the memory snapshot: a cache growing by concatenation, each step allocating a slightly bigger tensor. Fix: static or paged cache.
- **Sawtooth**: large temporary tensors (logits, attention scores) allocated and freed every step. Fix: logits_to_keep, fused kernels, FlashAttention.
- **Reserved** >> **allocated**, growing: fragmentation in the caching allocator from variable-size tensors. Fix: fixed-size blocks; `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` can help.
- **OOM with plenty "free"**: fragmentation or a single huge request (prefill activations). Check the snapshot at the moment of failure.

17.8 Diagnosis drills (answers in the answer key)

Scenario A

Batch-1 decode of a 1.5B model on your 4050 takes 17 ms/token. Doubling batch to 2 makes it 17.5 ms/token. GPU-Util 100%.

Scenario B

A 0.5B model decodes at 9 ms/token regardless of whether weights are FP16 or 8-bit. The nsys timeline shows ~60% idle gaps between kernels.

Scenario C

Prefill of 8K tokens takes 1.9 s; converting weights to 8-bit changes it to 1.85 s. Tensor-core utilization ~70%.

Scenario D

ITL is 20 ms at 2K context and 58 ms at 32K context, batch 16, same model.

Scenario E

Qwen2.5-0.5B, HF DynamicCache. The memory snapshot shows allocated memory climbing by $24 \times 2 \times 2 \times 64 \times 2 \times B$ bytes every decode step, plus a 300 MB spike at every request start.

Scenario F

vLLM reports KV cache usage 98% and a rising count of preempted requests; GPU SM utilization is ~35%; ITL is fine; queue time is growing.

Checkpoint · Part 17

Recall

- 1 What does `nvidia-smi's GPU-Util` actually measure?
- 2 Difference between `memory_allocated` and `memory_reserved`?
- 3 Which tool shows gaps between kernels, and which shows DRAM throughput of a single kernel?

Reasoning

- 4 Why can adding FLOPs (e.g. dequantization) leave decode time unchanged?
- 5 Why do small models at batch 1 often become overhead-bound, and what fixes it?

Practical engineering

- 6 Diagnose scenarios A–F above: name the bound and one fix for each.

Calculation

- 7 Your 4050 decodes Qwen2.5-1.5B (3.1 GB FP16 weights, 28 KiB/token) at batch 1, 4K context in 21 ms/token. Compute the effective bandwidth and compare it with 192 GB/s. What does the result suggest?

 Handwritten note

Don't trust GPU-Util. Classify every slow step: compute, bandwidth, capacity or overhead. Then change batch, bytes and context to see which knob moves time.

What do you think happens next?

Production engines package all these ideas together. How do the scheduler, KV manager and kernels talk to each other inside vLLM or TensorRT-LLM?

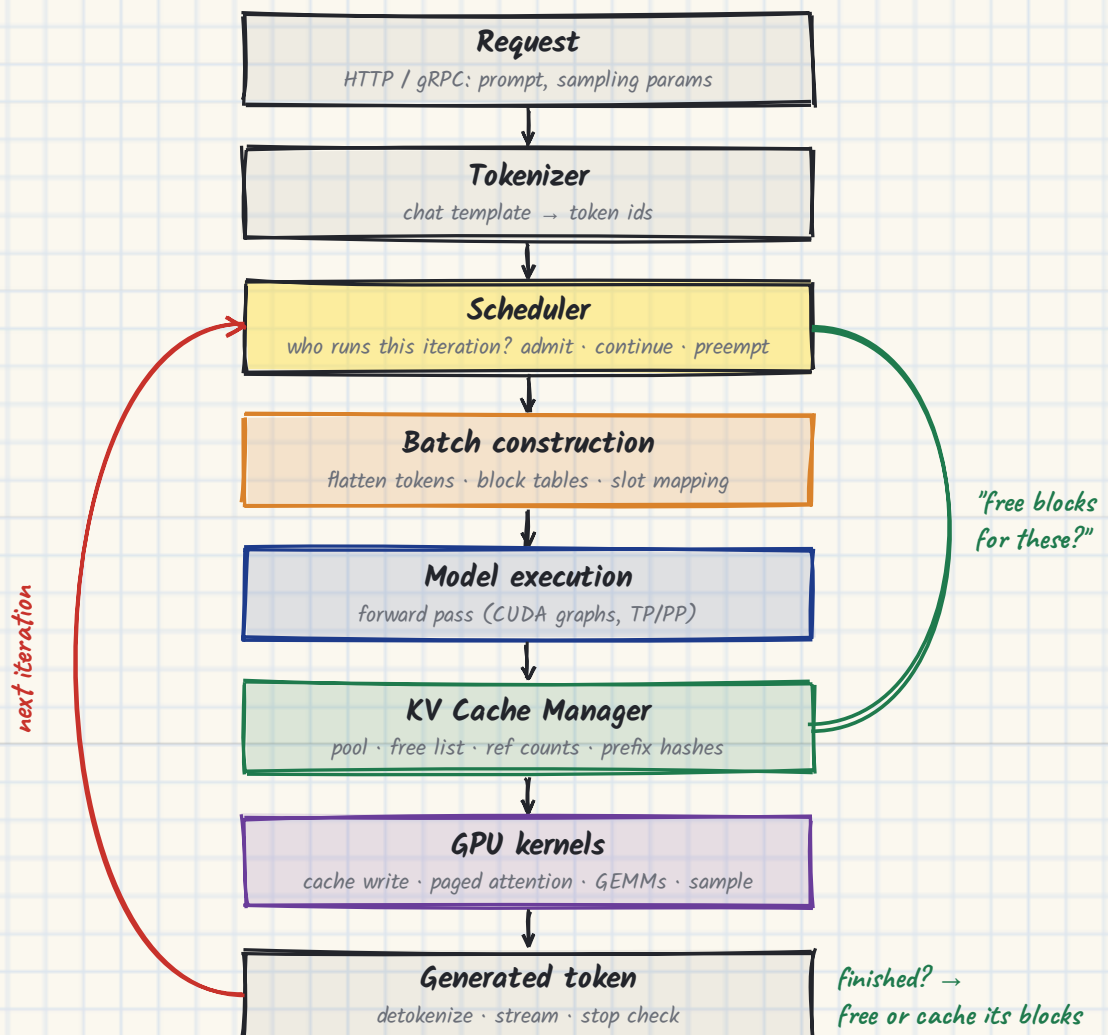
PART 18 · Module F: Production & career

Production Inference Engines

Goal of this part

See a serving engine as a set of cooperating components organized around the KV cache. Follow one request through the whole pipeline. Understand why every serious engine ships its own cache manager and scheduler, and how vLLM, TensorRT-LLM, SGLang and TGI-style servers differ in emphasis.

18.1 The pipeline



one iteration of an engine loop. The scheduler and the KV cache manager negotiate every step.

18.2 What each component does, and how it touches the KV cache

Component	Job	KV interaction
API server / tokenizer	parse request, apply chat template, tokenize, set sampling params; stream tokens back	the exact token ids decide prefix-cache hits (Part 10)
Scheduler	each iteration: pick which waiting requests to admit, which running ones continue, which to preempt; enforce token and sequence budgets	asks the KV manager: "blocks for this prompt? for everyone's next token?" Admission is limited by free blocks
KV cache manager (block manager)	owns the preallocated pool: free list, block tables, ref counts, prefix-hash table, eviction	allocates/frees blocks, matches prefixes, performs CoW, swaps or drops blocks on preemption
Batch construction	flatten all scheduled tokens (prefill chunks + one token per decoding sequence) into one ragged batch; build metadata	produces block tables and slot mappings (where each new token's K/V will be written) for the kernels
Model execution	run the forward pass (often a CUDA graph for decode), with tensor/pipeline parallelism across GPUs	each attention layer: write new K/V to their slots, then attend through the block tables
GPU kernels	GEMMs, fused cache-write kernel, paged/flash attention for prefill and decode, sampling	read K/V blocks via the block table; possibly dequantize FP8
Sampler + output processor	logits → next tokens; detokenize; stop conditions; stream	finished sequences → scheduler → KV manager frees (or caches) their blocks

18.3 Why engines need specialized cache managers and schedulers

- **A general-purpose allocator can't know the future.** Output lengths are unknown; only a KV-aware manager can grow allocations token by token without copying (paging).
- **Admission is a memory decision.** Admitting a request commits future KV; admit too much and you thrash (preempt, recompute); admit too little and GPUs idle.
- **Sharing needs ownership semantics.** Prefixes, parallel samples and beams need ref counts and copy-on-write, which PyTorch tensors don't provide.
- **Kernels and memory layout are co-designed.** The block size and layout exist so attention kernels read memory efficiently.
- **SLOs are set per iteration.** Mixing prefill chunks with decodes, capping tokens per step and prioritizing requests are scheduler policies that directly set TTFT and ITL.

18.4 The engines, by design emphasis

Engine	KV / scheduling design emphasis	Where it tends to shine
--------	---------------------------------	-------------------------

vLLM	PagedAttention block manager; continuous batching; automatic prefix caching via block hashing; chunked prefill; FP8 KV; a scheduler that treats prefill and decode uniformly as "tokens to compute this step"; many model architectures supported	general-purpose open serving; the reference design for paged KV
TensorRT-LLM	compiled, heavily fused kernels for NVIDIA GPUs; in-flight batching; paged KV cache; KV reuse across requests; quantized KV; commonly deployed behind Triton Inference Server or NVIDIA's serving stacks	peak performance on NVIDIA hardware when you can invest in build/tuning
SGLang	RadixAttention: prefix cache organized as a radix tree, with cache-aware scheduling to maximize prefix reuse; fast runtime plus a frontend for structured, multi-call programs	workloads with heavy prefix sharing: agents, few-shot, multi-turn, structured outputs
TGI-style servers	a router (request queue, batching) in front of model shards; continuous batching; paged/flash attention kernels	simple production deployments integrated with a model hub

These projects move fast and borrow each other's ideas; feature lists change release by release. Compare them on the **design questions** (How is KV allocated? How are prefixes shared? How are prefill and decode scheduled? What happens under memory pressure?), not on a feature checklist.

18.5 One request, end to end (vLLM-style)

- Arrive:** a chat request with a 3,000-token prompt; the tokenizer produces ids.
- Prefix match:** the KV manager hashes the 187 full blocks; the first 180 hit (shared system prompt + earlier turns) → ref counts +1.
- Admit:** the scheduler needs blocks for the remaining ~120 tokens plus growth headroom; free blocks exist → admitted this iteration.
- Chunked prefill:** the 120 new tokens are computed alongside 60 other sequences' decode tokens in one flattened batch.
- Decode:** each iteration: 1 token; every 16 tokens a new block; attention reads 3,000+ tokens of KV via the block table.
- Pressure:** a burst arrives and the pool hits 100%. The scheduler preempts the newest low-priority sequence (recompute later).
- Finish:** EOS → blocks' ref counts drop; the private ones become evictable cache; shared ones stay hot for the next request.

Checkpoint · Part 18

Recall

- 1 Name the components between "request" and "generated token" and give each a one-line job.

- 2 What two pieces of metadata does batch construction give the attention kernels?
- 3 What's SGLang's prefix-cache data structure?

Reasoning

- 4 Why is admission control fundamentally a KV-memory decision rather than a compute decision?
- 5 Why do engines run decode as CUDA graphs but often not prefill?

Practical engineering

- 6 You're choosing an engine for an agent product where every call shares a 6K-token tool description and conversation history. Which design properties matter most, and how would you test candidates?

Calculation

- 7 An engine is configured with `max_num_batched_tokens = 8,192` per iteration and 200 decoding sequences. How many prompt tokens can be prefilled per iteration? If a 20K-token prompt arrives, how many iterations until its first token (ignoring other arrivals)?

Handwritten note

An inference engine is a KV-memory manager with a model attached: the scheduler spends free blocks, batch construction turns them into block tables, kernels read through them.

What do you think happens next?

Systems like this fail in characteristic ways. Can you recognize a KV problem from its symptoms alone?

PART 19 · Module F: Production & career

Failure Modes

Goal of this part

Recognize the common KV-cache failures from their symptoms, know how to confirm each one, and know the standard fixes. This is the on-call playbook.

#	Failure	One-line tell
1	GPU out-of-memory	crash or rejected requests at load or with long inputs
2	fragmentation	OOM while "free" memory looks sufficient
3	too many concurrent sequences	ITL climbs steadily with load; preemptions
4	long prompts destroying concurrency	everyone slows when one big request arrives
5	cache thrashing	throughput collapses under load; GPU busy recomputing
6	poor block utilization	fewer sequences fit than the math predicts
7	memory-bandwidth saturation	adding batch stops adding throughput
8	prefix-cache misses	TTFT high although prompts share a prefix
9	incorrect position handling	fluent first token, then degradation
10	cache corruption bugs	outputs leak between requests or depend on batch composition
11	batch scheduling problems	ITL spikes, starvation, unfair latency

1 · GPU out-of-memory

Symptom	CUDA out of memory at startup, on long prompts, or when traffic peaks; the engine refuses to start ("not enough KV cache for max_model_len").
Likely cause	weights + KV budget + activations exceed VRAM; max context too large for the KV pool; a single huge prefill's activations or logits; another process on the GPU.
Diagnose	compute $\text{weights} + kv_{\text{tok}} \times \text{max tokens}$ (Part 5); memory snapshot at failure; check the prefill size at OOM time; <code>nvidia-smi</code> for other processes.
Fixes	lower max model length or max batched tokens; FP8 KV; quantized weights; chunked prefill; last-position-only logits; more GPUs (tensor parallelism splits weights and KV).

2 · Fragmentation

<i>Symptom</i>	<i>OOM or admission failures while total free memory seems adequate; memory_reserved >> memory_allocated.</i>
<i>Likely cause</i>	<i>contiguous, variable-size KV allocations (HF-style concat caches in a custom server), allocator fragmentation.</i>
<i>Diagnose</i>	<i>memory snapshot: many free gaps too small for the failing allocation; compare reserved vs allocated over time.</i>
<i>Fixes</i>	<i>paged/fixed-size block allocation; preallocate a pool; expandable_segments=True; restart-free designs that never reallocate per step.</i>

3 · Too many concurrent sequences

<i>Symptom</i>	<i>ITL grows with load; preemption counter rising; tail latency explodes at peak.</i>
<i>Likely cause</i>	<i>max sequence cap too high for the KV pool, so every step reads more KV (Part 3) and memory runs out mid-generation.</i>
<i>Diagnose</i>	<i>plot ITL vs running sequences; KV usage near 100%; preemption and swap metrics.</i>
<i>Fixes</i>	<i>cap concurrency to meet the ITL SLO; admission control based on predicted KV; scale out; FP8 KV / GQA models.</i>

4 · Long prompts destroying concurrency

<i>Symptom</i>	<i>a few long-context requests make everyone slow or queue; TTFT for short chats jumps.</i>
<i>Likely cause</i>	<i>one 100K-token request consumes as much KV as 50 × 2K chats; its prefill monopolizes iterations.</i>
<i>Diagnose</i>	<i>correlate latency spikes with request lengths; per-request KV usage.</i>
<i>Fixes</i>	<i>chunked prefill; separate long-context pool/replicas; per-tenant token budgets; prefix caching for repeated documents; priorities.</i>

5 · Cache thrashing

<i>Symptom</i>	throughput falls as load rises; GPU busy but output tokens/s low; the same requests are preempted repeatedly.
<i>Likely cause</i>	over-admission: sequences evict each other and are recomputed (or swapped) again and again; prefix cache churn under an LRU with too little room.
<i>Diagnose</i>	preemption counts per request; recomputed-token ratio; prefix-cache eviction rate vs hit rate.
<i>Fixes</i>	stricter admission (reserve headroom); prefer finishing running requests; swap instead of recompute for long sequences; more KV capacity.

6 · Poor block utilization

<i>Symptom</i>	concurrency far below N_{max} computed from the formula.
<i>Likely cause</i>	block size too large for short requests; big per-sequence reservations (lookahead, max_tokens reservation); stale cached blocks not evicted; sliding-window models allocated as full attention.
<i>Diagnose</i>	tokens stored ÷ block slots allocated; distribution of sequence lengths vs block size.
<i>Fixes</i>	smaller blocks; don't reserve max_tokens up front; engine support for window/hybrid caches; tune the prefix-cache share.

7 · Memory-bandwidth saturation

<i>Symptom</i>	adding batch no longer raises throughput; ITL scales linearly with batch × context; SM utilization low, DRAM throughput near peak.
<i>Likely cause</i>	KV reads dominate each step (Part 4's crossover passed).
<i>Diagnose</i>	ncu: attention kernels at high % of DRAM bandwidth; step time $\approx (W + \Sigma KV) \div BW$.
<i>Fixes</i>	FP8/INT8 KV; GQA/MLA models; shorter contexts or windowed attention; more GPUs (split KV across devices); speculative decoding for latency.

8 · Prefix-cache misses

<i>Symptom</i>	low hit rate and high TTFT although prompts "share" a system prompt.
<i>Likely cause</i>	dynamic content early in the prompt (timestamps, user ids); template or tokenization changes; requests load-balanced away from the replica holding the prefix; cache too small (evicted).
<i>Diagnose</i>	diff the token ids of two requests; find the first differing position; check routing; log hit rate per prefix.
<i>Fixes</i>	static-first prompt layout; stable templates; cache-aware routing; larger cache share; pin hot prefixes.

9 · Incorrect position handling

<i>Symptom</i>	first generated token good, then repetition or nonsense; or quality falls only for batched or padded inputs.
<i>Likely cause</i>	decode positions not offset by the cache length; left-padding not reflected in position ids; RoPE applied at the wrong offset after cache truncation (e.g. speculative rollback).
<i>Diagnose</i>	compare cached vs uncached greedy outputs (Part 14's equality test); log position ids per step.
<i>Fixes</i>	$\text{positions} = \text{cache length} + \text{arange}(T)$, per sequence; test padded batches against single-sequence runs.

10 · Cache corruption bugs

<i>Symptom</i>	a response contains fragments of another user's conversation; outputs change depending on which requests share the batch; rare nondeterministic garbage.
<i>Likely cause</i>	block reused before its old owner finished (ref-count bug); wrong slot mapping; missing copy-on-write on a shared partial block; prefix hash collision or a missing key (e.g. ignoring the LoRA id).
<i>Diagnose</i>	reproduce with fixed seeds and batch compositions; assert ref counts; checksum blocks; run with prefix caching off to bisect.
<i>Fixes</i>	fix ownership logic; include every KV-affecting input in hash keys; strong hashes; treat it as a security incident if data crossed tenants.

11 · Batch scheduling problems

<i>Symptom</i>	periodic ITL spikes; some requests starve; good throughput but poor p99.
<i>Likely cause</i>	large unchunked prefills in decode iterations; FCFS with no priority for near-finished requests; token budget per iteration too high (long steps) or too low (idle GPU).
<i>Diagnose</i>	per-iteration timeline: tokens scheduled vs step time; queue time vs execution time per request.
<i>Fixes</i>	chunked prefill with a sensible chunk size; priority/fair scheduling; SLO-aware admission; disaggregated prefill/decode at scale.

Checkpoint · Part 19

Recall

- 1 What's the single best test for position-handling bugs?
- 2 Name three causes of prefix-cache misses.
- 3 What distinguishes thrashing from plain overload?

Reasoning

- 4 Why can raising the maximum number of sequences make throughput worse?
- 5 Why must the LoRA adapter id be part of the prefix-cache key?

Practical engineering

- 6 A customer reports seeing another customer's text in a response. Walk through your first hour: containment, diagnosis, fix.

Calculation

- 7 A pool holds 100K tokens of KV. Traffic: 60 chats of 1.5K tokens and one 40K-token document request. Does it fit? If the document request arrives while the 60 chats are running, how many chats would have to be preempted to admit it?

Handwritten note

Most KV failures are capacity (OOM, thrash, concurrency), bandwidth (saturation), locality (prefix misses) or correctness (positions, ownership). Always verify cached = uncached outputs.

What do you think happens next?

Can you now answer interview questions out loud, from basic to infrastructure level, without notes?

PART 20 · Module F: Production & career

Interview Preparation

How to use this part

Answer each question **out loud** or **in writing** before you look at the model answers in Appendix B. Time yourself: 60–90 seconds for levels 1–3, 3–5 minutes for levels 4–5. Then grade yourself with the rubric below, the way a senior inference engineer would.

What the interviewer is listening for

Signal	Strong answer	Red flag
causality	"because causal masking makes past K/V immutable..."	definitions without "because"
numbers	estimates with units: "~0.5 MiB/token, so 2 GiB at 4K"	"it uses a lot of memory"
precision	per-step vs total; compute vs bandwidth vs capacity	"KV cache makes attention linear"
trade-offs	names what each optimization costs	optimizations presented as free wins
systems view	connects model math to scheduler, memory, SLOs	stays inside the Transformer diagram
calibration	"I'd measure X to confirm"; admits uncertainty	confident wrong specifics

Level 1: Fundamentals

- 1.1** What is a KV cache, and what problem does it solve?
- 1.2** Why cache K and V but not Q?
- 1.3** What's the difference between prefill and decode?
- 1.4** Is the KV cache an approximation?
- 1.5** What does `use_cache=True` do in Hugging Face?
- 1.6** What grows as a conversation gets longer, even with a KV cache?

Level 2: ML engineer

- 2.1 How does KV caching affect computational complexity? Be precise.
- 2.2 Would KV caching reduce TTFT?
- 2.3 Derive the KV-cache memory formula and compute it for Llama-2-7B at 4K tokens.
- 2.4 Why does GQA reduce KV memory? What does it cost?
- 2.5 What goes wrong if position ids aren't offset during cached decoding?
- 2.6 Why can't KV caching be used as-is with a bidirectional encoder generating text?

Level 3: LLM engineer

- 3.1 Why might decode be memory-bandwidth-bound?
- 3.2 Why does batching improve decode throughput, and why does it eventually stop helping?
- 3.3 How does KV cache quantization help, and when would you avoid it?
- 3.4 Explain prefix caching and one way it silently fails.
- 3.5 What changes when serving a 128K-context model?
- 3.6 How does speculative decoding interact with the KV cache?

Level 4: ML systems engineer

- 4.1 Why can the KV cache become the limiting factor in serving?
- 4.2 How does PagedAttention help? What does it cost?
- 4.3 How would you estimate the maximum number of concurrent requests on an 80 GB GPU?
- 4.4 Explain continuous batching and why it needs fine-grained KV allocation.
- 4.5 How do you tell whether a decode step is compute-bound, bandwidth-bound or overhead-bound?
- 4.6 What is chunked prefill and which metric does it protect?

Level 5: Inference infrastructure engineer

- 5.1 How would you design a KV-cache manager?

- 5.2 When would you disaggregate prefill and decode, and what does it cost?
- 5.3 A fleet serves one 70B model. p99 ITL doubled after enabling a long-document feature, while p50 barely moved. Diagnose and fix.
- 5.4 Design cache-aware routing across 20 replicas without creating hot spots.
- 5.5 Swap vs recompute on preemption: derive a decision rule.
- 5.6 How would you serve 1M-token contexts?

Mock-interview protocol

Pick 5 random questions (one per level). Record yourself answering. Then, for each: (1) did you give a causal chain? (2) at least one number with units? (3) a trade-off? (4) a way to measure? Score 0–4. Consistent 3–4 at level 4 is a strong systems signal.

Checkpoint · Part 20

Recall

- 1 List the six signals interviewers listen for.
- 2 What's the 30-second answer to "why cache K and V but not Q"?
- 3 What number should appear in your answer about Llama-2-7B KV memory?

Reasoning

- 4 Why is "KV caching makes attention $O(n)$ " a red flag answer, and how do you correct it gracefully?
- 5 An interviewer asks a question you can't fully answer. How do you use first principles to give a partial but valuable answer?

Practical engineering

- 6 Prepare a 2-minute story about your KV Cache Lab project (Part 16): problem, method, one surprising result, what you'd do next.

Calculation

- 7 Out loud, in under 90 seconds: Llama-3-8B on an A100 80GB, FP16 weights and KV, average 3K tokens per request. Estimate max concurrent requests and batch-1 ITL.

 Handwritten note

Great answers are causal chains with numbers: "because X, therefore Y, which costs Z, and I'd measure W."

What do you think happens next?

The hardest interviews don't ask facts; they hand you an open problem and add constraints until your design breaks.

PART 21 · Module F: Production & career

System Design

Design an LLM inference service serving thousands of concurrent users

How this part works

Six rounds. Each round adds a constraint. **Stop at each "Your move" and sketch your answer first** (numbers, components, trade-offs), then read the reference reasoning. The reference is one good answer, not the only one.

Round 0: clarify before designing

Strong candidates ask questions first. The answers we'll assume:

Question	Assumed answer
Model?	one 70B GQA model (Llama-3-70B-class: 80 layers, 8 KV heads, d_h 128 → 320 KiB/token in BF16)
Traffic?	5,000 concurrent users in chat sessions; ~1,000 sequences actively generating at any moment at peak
Lengths?	average context ~3K tokens (prompt + history + output); outputs ~300 tokens
SLOs?	p95 TTFT < 1 s; p95 ITL < 50 ms (≥ 20 tokens/s per user)
Hardware?	nodes of 8 × H100 80GB (NVLink within a node)
Other?	a shared 5K-token system prompt; streaming responses; multi-tenant

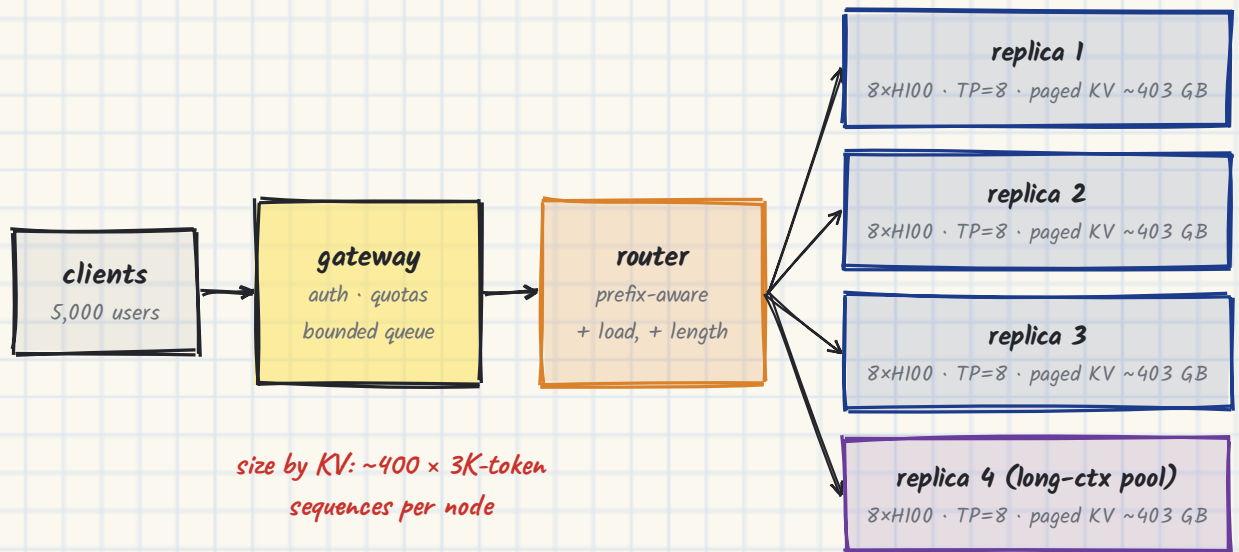
Round 1: baseline sizing

Your move

How many nodes? Start from weights, then KV per sequence, then sequences per node.

- **Weights:** $\sim 70.6B \times 2 B \approx 141 GB$ → doesn't fit on one GPU → tensor parallelism across the 8 GPUs of a node (TP=8 also splits the 8 KV heads: one per GPU).
- **Node memory:** 640 GB. Keep ~10% headroom, plus ~5% for activations/workspace: KV budget $\approx 0.9 \times 640 - 141 - 32 \approx 403 GB$.
- **Tokens per node:** $403 GB \div 327,680 B \approx 1.23M$ tokens → at 3K per sequence ≈ 400 sequences per node.
- **Nodes for 1,000 active sequences:** $1,000 \times 3,072 \times 320 KiB \approx 938 GiB$ of KV → 3 nodes minimum by memory; plan 4 for headroom and failures.

- **Does ITL hold?** Per decode step per GPU: weights $141/8 \approx 17.6$ GB + KV of 400 seqs $\div 8 \approx 50$ GB $\rightarrow \sim 68$ GB at 3.35 TB/s $\approx$ **20 ms ideal** (~ 30 ms real). Under the 50 ms SLO.
Throughput $\approx 400 \div 0.025$ s $\approx$ **16K tokens/s per node**.



baseline: gateway $\rightarrow$ prefix-aware router $\rightarrow$ 4 replicas ($8 \times H100$, $TP=8$), each with its own scheduler and paged KV pool

Round 2: the latency SLOs start failing

New fact: p95 TTFT is 2.4 s and p99 ITL spikes to 400 ms whenever users paste long text.

Your move

Which phase interaction causes both symptoms? What do you change?

- Large prefills run unchunked inside decode iterations $\rightarrow$ every running sequence stalls (ITL spike); prompts queue behind them (TTFT).
- **Chunked prefill** with a per-iteration token budget (e.g. 2K–4K tokens) caps step time. Tune the chunk size to trade TTFT against ITL.
- **Admission by KV + SLO:** cap running sequences (e.g. ~ 300 per node instead of 400) so step time stays under budget.
- **Priority:** short requests first when the queue is deep; near-finished sequences never preempted.
- Measure: TTFT split into queue / prefill; ITL vs tokens-per-iteration.

Round 3: 30% of traffic becomes 32K-token document chats

Your move

Recompute the capacity. What happens to everyone else?

- One 32K sequence = $32,768 \times 320 \text{ KiB} = 10 \text{ GiB}$, the KV of ~11 normal chats.
- If 300 of 1,000 active sequences are 32K: $300 \times 10 \text{ GiB} \approx 3 \text{ TiB}$ of KV → ~8 nodes for those alone. ITL also rises: each step reads 10 GiB per long sequence.
- **Redesign:** a **separate long-context pool** (routing by prompt length) so long jobs can't starve short chats; **FP8 KV** in that pool (halves memory and KV read time); **prefix caching** of the document so follow-up questions skip the 32K prefill; consider disaggregated prefill for this pool.

Round 4: the shared 5K-token system prompt

Your move

How much memory and compute is wasted without sharing? How do you make sharing actually hit?

- Without sharing: $5,000 \times 320 \text{ KiB} \approx 1.5 \text{ GiB per sequence}$ → for 400 sequences per node, 610 GiB, more than the node's KV budget. Sharing is mandatory, not optional.
- **Automatic prefix caching** (block hashes) keeps one copy per replica; **static-first prompt layout** (no timestamps at the top); **prefix-aware routing** with a load-balancing fallback so hot prefixes spread across replicas.
- Effect: TTFT drops by the 5K-token prefill (~hundreds of ms on a 70B); capacity per node rises ~1.5x.

Round 5: a 3x traffic spike, and one node dies

Your move

You can't add GPUs for 20 minutes (model load + warm-up). What degrades, and how do you keep the service up?

- **Admission control and queueing** at the gateway (a bounded queue with timeouts, per-tenant quotas). Better to return 429 quickly than to thrash every replica.
- **Graceful degradation:** lower max_tokens, cap context length, turn off parallel sampling, route the long-context pool's overflow to a smaller model.

- **Preemption policy:** recompute short sequences, swap long ones (Part 13's transfer math); never preempt near-finished ones.
- **Failure:** sessions on the dead node lose their KV. Clients retry, and the router re-prefills on a healthy node (prefix caching makes that cheaper). Keep $N+1$ capacity.
- **Autoscaling** on leading indicators (queue depth, KV usage, TTFT) rather than GPU-Util.

Round 6: multi-GPU and multi-node efficiency

Your move

Is $TP=8$ per replica right? When would you disaggregate?

- **TP within a node** (NVLink) for weights and KV heads; avoid TP across nodes (all-reduce per layer over slower links). Consider $TP=4 \times 2$ replicas per node with FP8 weights (~70 GB) if TTFT allows. That's more replicas, fewer collectives, and each 4-GPU replica still holds 2 KV heads per GPU.
- **Disaggregated prefill/decode** when TTFT and ITL SLOs conflict and prompts are long: prefill nodes (compute-bound) stream KV over RDMA to decode nodes (bandwidth-bound). Cost: the KV transfer (a 3K prompt ≈ 0.94 GiB, ~20 ms at 50 GB/s) and pool balancing.
- **Observability:** TTFT (queue / prefill), ITL p50/p95/p99, KV usage, preemptions, prefix hit rate, tokens per iteration, per-tenant usage.

The design in one paragraph (say this at the end)

"A 70B GQA model at $TP=8$ per node gives ~400 GB of KV per node, about 400 concurrent 3K-token sequences at ~20–30 ms ITL. I size by KV, not FLOPs. Chunked prefill and SLO-aware admission protect ITL; a separate long-context pool with FP8 KV and document prefix caching keeps long jobs from starving chats; automatic prefix caching plus prefix-aware routing makes the shared system prompt nearly free. Under spikes I shed load at the gateway instead of thrashing KV, and I'd disaggregate prefill and decode once prompts are long enough that the two phases fight."

Checkpoint · Part 21

Recall

- 1 What's the KV budget of one $8 \times H100$ node for a 70B BF16 model, and how is it derived?
- 2 Which two scheduler features fixed Round 2's symptoms?
- 3 Why route long-context requests to a separate pool?

Reasoning

- 4 Why is sizing by KV memory more reliable than sizing by FLOPs for this service?
- 5 Why prefer returning 429s during a spike over admitting everyone?

Practical engineering

- 6 Leadership asks to halve GPU cost with the same SLOs. List three levers from this book with their estimated effect and risk.

Calculation

- 7 Redo Round 1 with FP8 weights (~70 GB) and FP8 KV (160 KiB/token) on the same node. Sequences per node at 3K? Ideal ITL at that batch? Nodes needed for 1,000 active sequences?

Handwritten note

Size LLM serving by KV: $\text{budget} = \text{memory} - \text{weights} - \text{headroom}$; $\text{sequences} = \text{budget} \div (\text{kv}_{\text{tok}} \times \text{length})$. Then protect ITL with chunked prefill and admission, and share prefixes.

What do you think happens next?

The final test: can you answer the 13 first-principles questions without looking anything up?

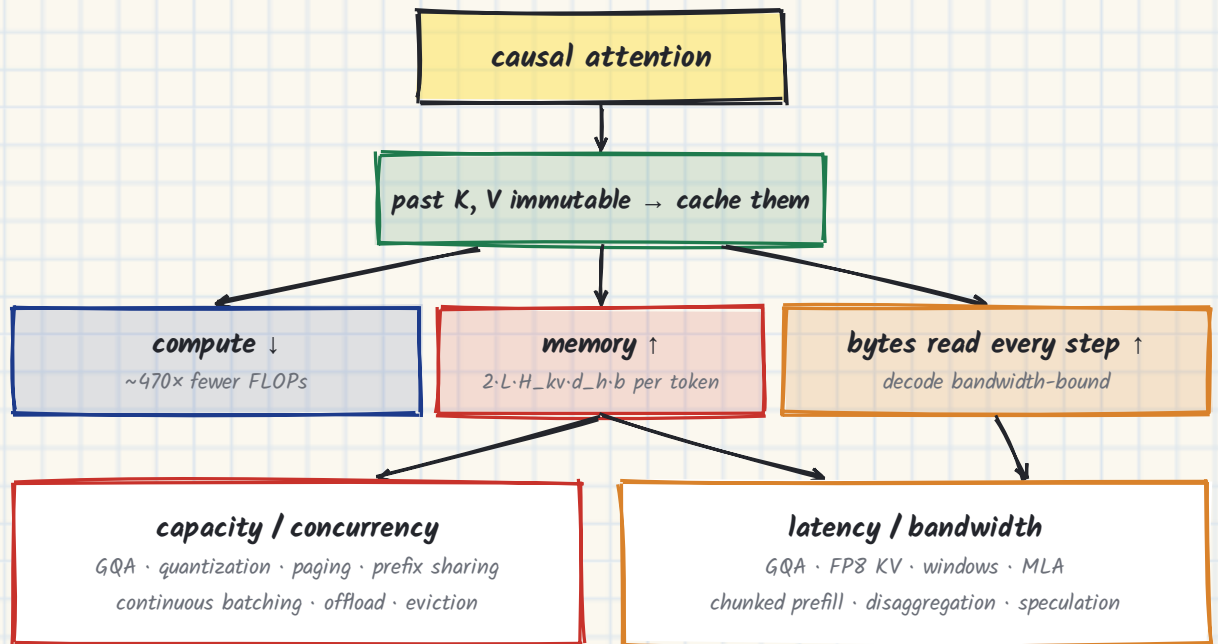
PART 22 · Module F: Production & career

First-Principles Check

The rules

Answer all 13 questions in your own notebook, **without looking anything up**, in 3–5 sentences each, with at least one number where it makes sense. Only then turn the page. For every question you couldn't answer cleanly, the model-answer page names the **conceptual gap** and the section to **reteach** yourself.

- 1 Why does the KV cache exist?
- 2 What exactly is being cached?
- 3 Where is it stored?
- 4 How large is it?
- 5 Why does it grow?
- 6 Why does it improve generation speed?
- 7 Why doesn't it eliminate attention cost?
- 8 Why does long context make KV caching difficult?
- 9 Why do MQA and GQA matter?
- 10 Why was PagedAttention important?
- 11 Why is decode often limited by memory movement?
- 12 How does KV caching affect concurrency?
- 13 What would you optimize first in a real inference server?



the whole book on one page: every idea follows from causality plus "decode reads bytes"

Model answers, gaps and reteach pointers

1 · Why does the KV cache exist?

Generation is autoregressive: each new token needs attention over all previous tokens at every layer. Causal masking means a token's hidden states, and therefore its K and V, depend only on tokens up to itself, so they never change once computed. Caching them turns each step from re-running the whole prefix ($\approx 2P \cdot t$ FLOPs) into processing one token ($\approx 2P$) plus reading the cache. **Gap if stuck:** causality → immutability. **Reteach:** 1.2c, 2.2–2.4.

2 · What exactly is being cached?

The key and value vectors of every past token, for every layer and every KV head: per layer, K and V tensors of shape $[B, H_{kv}, T, d_h]$ (or paged blocks). Not queries, scores, outputs, logits or tokens. **Gap:** confusing K/V with hidden states or scores. **Reteach:** 1.7, 2.5.

3 · Where is it stored?

In GPU HBM/VRAM, next to the weights, usually in a pool the engine preallocates and hands out in blocks. Attention kernels stream tiles of it into on-chip SRAM each step. Cold KV can be offloaded to CPU DRAM or SSD over PCIe, $\sim 100\times$ slower. **Reteach:** 7.1–7.2, 13 ⑤.

4 · How large is it?

$2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$ bytes per token, times all tokens in flight. Llama-2-7B FP16: 512 KiB/token → 2 GiB at 4K. Llama-3-8B (GQA): 128 KiB/token. 70B GQA: 320 KiB/token. **Gap:** using H_q instead of H_{kv} , forgetting the 2 or the layers. **Reteach:** 5.1–5.3.

5 · Why does it grow?

Each processed token appends one K and one V row per layer (per KV head), and old rows are never freed while the sequence lives, because every future token may attend to them. So it grows linearly with context, and with the number of concurrent sequences. **Reteach:** 1.5, 2.2.

6 · Why does it improve generation speed?

It removes recomputation: per decode step the linear layers run on 1 token instead of t , and attention computes one row instead of the full triangle. For 4K prompt + 500 new tokens on a 7B model, total FLOPs drop $\sim 470\times$. **Reteach:** 1.6, 4.3.

7 · Why doesn't it eliminate attention cost?

The new query must still be compared against every cached key and mix every cached value: $O(t)$ work and, more importantly, $O(t)$ bytes read per step. Total attention over a generation is still quadratic. It removes recomputation, not re-reading. **Gap:** "KV cache makes attention linear". **Reteach:** 4.2, 4.5.

8 · Why does long context make KV caching difficult?

Memory $\propto n$ (16 GiB at 128K for an 8B GQA model, 128 GiB at 1M), per-step reads $\propto n$ (they overtake the weight reads near 120K tokens for Llama-3-8B), prefill $\propto n^2$, and concurrency $\propto 1/n$. Long contexts need chunked prefill, quantized/compressed KV, prefix reuse, multi-GPU KV and separate pools. **Reteach:** 12.1–12.3.

9 · Why do MQA and GQA matter?

KV size scales with the number of KV heads. Sharing each K/V head across g query heads cuts KV memory and bandwidth by g (Llama-2-70B: 8x), with near-MHA quality for GQA. It also raises decode attention's arithmetic intensity and splits cleanly under tensor parallelism.

Reteach: 6.1–6.4.

10 · Why was PagedAttention important?

Contiguous max-length allocation wasted most KV memory (often only 20–40% held real tokens) through reservation, internal and external fragmentation. Fixed-size blocks with block tables allocate on demand, free instantly and allow ref-counted sharing, leaving waste under one block per sequence. More sequences fit, so batch and throughput rise (2–4x in the vLLM paper). **Reteach:** 7.5, 8.1–8.4.

11 · Why is decode often limited by memory movement?

Each step must read all the weights and every sequence's cache, but does only ~ 2 FLOPs per weight per sequence and ~ 1 FLOP per KV byte. That's far below a GPU's ~ 150 – 300 FLOPs/byte ridge. Batching amortizes weights but not KV, which is private per sequence.

Reteach: 3.2, 3.6.

12 · How does KV caching affect concurrency?

Each active sequence holds its cache for its whole lifetime, so $N_{\max} \approx (\text{memory} - \text{weights} - \text{activations}) \div (k_{v_{\text{tok}}} \times \text{tokens per sequence})$. E.g. $\sim 52 \times 2\text{K}$ chats for Llama-2-7B on an A100 vs ~ 197 for Llama-3-8B. The KV pool, not compute, usually caps the batch. **Reteach:** 5.5, 9.3.

13 · What would you optimize first in a real inference server?

Measure first: TTFT split, ITL percentiles, KV usage, preemptions, prefix hit rate. Then, typically: (1) a paged KV engine with continuous batching; (2) prefix caching with a static-first prompt layout; (3) chunked prefill tuned to the ITL SLO; (4) FP8 KV (and weights) if capacity- or bandwidth-bound; (5) right-size TP/replicas; then consider speculative decoding and disaggregation. Choose by the bottleneck the metrics show. **Reteach:** 13 intro table, 17.1, 21.

 The final handwritten note

Causality makes K and V immutable, so we cache them. The cache trades compute for memory capacity and bandwidth. Every advanced technique manages that trade: fewer bytes (GQA, quantization, compression), less waste (paging), shared bytes (prefixes), or better scheduling of who gets the bytes (batching, chunking, disaggregation).

APPENDIX A

Checkpoint Answer Key

Only after you've answered

These are model answers, not the only correct ones. Compare your **reasoning**, not your wording. If your answer reached the right conclusion by a different causal path, check that the path holds.

Part 1: Build the Intuition

1. q (what this token is looking for), k (the label other tokens' queries match against), v (the content that gets mixed in). All three come from the token's hidden state h through W_q , W_k , W_v .
2. K and V vectors: per token, per layer, per KV head (and per sequence). Not queries, scores, outputs, logits or tokens.
3. Per step, attention must read and score all t cached K/V (compute and bytes $\propto t$); cache memory $\propto t$; total attention over a generation stays quadratic.
4. **No.** If token 1 could see token 4, token 1's hidden states at layers ≥ 1 would change when token 4 arrives, so its K/V at layers ≥ 2 would be stale. The induction step " h_i uses only positions $\leq i$ " fails. (Layer-1 K/V , computed from embeddings, would still be valid; deeper layers wouldn't.)
5. The final row at layer ℓ needs K/V of all old tokens at layer ℓ , which need their hidden states from layer $\ell-1$, which need full attention + MLP for those tokens below. Without a cache, that recursion forces recomputing every old token through every layer.
6. Any of: every step still reads the whole growing cache (bandwidth $\propto$ context); each new user turn may re-fill the entire history if the server doesn't keep the conversation's KV (no prefix caching), so TTFT grows every turn; long contexts shrink the batch or trigger preemptions.
7. Assume the starting token is processed on forward pass 1 and each pass yields the next token: 200 passes. (a) No cache: inputs of length 1, 2, ..., 200 $\rightarrow \Sigma = 20,100$ token-passes. (b) Cache: **200**. Tokens that went through the model = 200 (the start token + the first 199 generated; the 200th is sampled but never fed back). K/V vectors written = $200 \times 32 \text{ layers} \times 2 = 12,800$ (12,864 if you also feed the last token). The one-token lag is exactly what Part 15 observes in Hugging Face.

Try-it (1.2b): scores $q_{on} \cdot k = 1, 2, 3, -1 \rightarrow \div \sqrt{2} = 0.707, 1.414, 2.121, -0.707 \rightarrow$ weights $\approx 0.135, 0.275, 0.557, 0.033$; $out_{on} = 0.135 \cdot [1, 0] + 0.275 \cdot [0, 2] + 0.557 \cdot [1, 1] + 0.033 \cdot [2, 0] \approx [0.76, 1.11]$.

Part 2: Attention Math

1. Prefill: $[B, H, P, P]$ (masked triangle). Decode: $[B, H, 1, t]$.
2. Projections are row-wise linear maps: $(XW)_i = x_i w$. With causality keeping old rows of X fixed, a new token just appends one row to K and V .
3. Per layer: K and V , each $[B, H_{kv}, T, d_h]$.
4. **No.** Sequence-wise normalization makes every token's hidden state depend on all tokens, including future ones, so old K/V would change as tokens arrive. The cache would go stale (unless the statistic were made causal, e.g. a running mean).

5. Decode feeds one row per sequence: $[B, d] \times [d, d]$. Each weight loaded from memory serves only B rows, so arithmetic intensity $\approx B$ FLOP/byte, far below the GPU ridge point. The math units wait on memory, so bandwidth decides speed.
6. The cache is being **overwritten** with only the newest token's K/V instead of appended. The model sees only one token of context. Fix the append (concat, or write at index len), and assert cache length = prompt + steps.
7. (a) q $[4, 32, 1000, 128]$; K cache $[4, 8, 1000, 128]$; scores $[4, 32, 1000, 1000]$. (b) q $[4, 32, 1, 128]$; K cache $[4, 8, 1010, 128]$; scores $[4, 32, 1, 1010]$. Each of the 8 KV heads serves 4 query heads.

Part 3: Prefill vs Decode

1. TTFT: arrival → first token (queue + prefill), dominated by prefill. ITL: gap between tokens, dominated by the decode step.
2. FLOPs ÷ bytes moved. A decode linear layer with batch B : $\approx B$ FLOP/byte (2 FLOPs per 2-byte weight per row).
3. Token $t+1$ is sampled from token t 's logits: a true data dependency.
4. Weights are shared: one load serves all B sequences. KV is private: each sequence's K/V bytes serve only its own query, so batch doesn't raise reuse.
5. TTFT improves up to $\sim 2\times$ (prefill is compute-bound). ITL barely changes (decode is bandwidth-bound and bandwidth is unchanged).
6. New requests' prefills are scheduled into the same iterations as running decodes. A big prefill makes that iteration long, so every stream stalls (head-of-line blocking). Fix: chunked prefill / per-iteration token budget, or disaggregation.
7. Batch 1: $26 \text{ GB} + 2,000 \times 819,200 \text{ B (1.64 GB)} = 27.6 \text{ GB} \rightarrow \div 3.35 \text{ TB/s} \approx \mathbf{8.3 \text{ ms}}$. Batch 8: $26 + 13.1 = 39.1 \text{ GB} \rightarrow \approx \mathbf{11.7 \text{ ms}}$ ($\approx 85 \text{ tok/s}$ per user, $\approx 680 \text{ tok/s}$ total).

Part 4: Complexity

1. No cache: attention $\propto t^2$, linear $\propto t$. With cache: attention $\propto t$, linear constant.
2. The context at which per-step KV bytes equal weight bytes: $t^* = W \div kv_{\text{tok}}$ (batch 1; t^*/B with batch B).
3. Memory to hold the cache ($\propto$ context, per sequence) and the per-step traffic to read it.
4. Both approaches run one forward pass per generated token, and each pass streams every weight from HBM once, whether it multiplies 1 row or t rows.
5. No: FLOPs stay $O(n^2)$. It avoids materializing the $n \times n$ matrix ($O(n)$ memory) and cuts HBM traffic with tiling, so wall-clock is faster.
6. At 512 tokens the KV read is $\sim 2\%$ of the weight read, which is invisible. At 64K for Llama-2-7B it's 32 GiB per step, $\sim 2.5\times$ the weights, so ITL rises several-fold. Benchmarks must sweep context.
7. $8 \cdot t \cdot 131,072 = 16.1 \times 10^9 \rightarrow t \approx \mathbf{15,350 \text{ tokens}}$.

Part 5: KV Cache Memory

1. $2 \text{ (K and V)} \times L \text{ (layers)} \times H_{kv} \text{ (KV heads)} \times d_h \text{ (values per head)} \times b \text{ (bytes each)} \times T \text{ (tokens)} \times B \text{ (sequences)}$.
2. $2 \times 32 \times 32 \times 128 \times 2 = 524,288 \text{ B} = \mathbf{512 \text{ KiB}}$ per token.
3. Keys and values are separate tensors of the same size.
4. GQA: 8 KV heads. $2 \times 80 \times 8 \times 128 \times 2 = 320 \text{ KiB}$ vs the 13B's $2 \times 40 \times 40 \times 128 \times 2 = 800 \text{ KiB}$.

5. True for one user with short context (13.5 GB weights leave ~1.5–2 GB: a few thousand tokens). False for batching or long contexts: that ~1.5 GB is ~3K tokens in total across all users.
6. Real Monday traffic likely has longer prompts (pasted reports) and more concurrency, so tokens in flight exceed the KV budget. The S12-token load test can't see it. Measure the distribution of prompt + output lengths and tokens in flight at failure time; KV usage; preemptions.
7. Budget $5.4 - 3.1 - 0.8 = 1.5$ GB → $\div 28,672 \approx$ **52,300 tokens** → **12** sequences of 4K. 8-bit KV (14 KiB/token): ~104,600 tokens → **25** sequences.

Part 6: MHA, MQA and GQA

1. MHA: $H_{kv} = H_q$. MQA: $H_{kv} = 1$. GQA: $1 < H_{kv} < H_q$ with query heads grouped.
2. num_attention_heads vs num_key_value_heads (smaller → GQA; 1 → MQA).
3. $g = H_q \div H_{kv}$.
4. Tensor parallelism splits heads across GPUs. A single KV head can't be split, so every GPU needs a full copy (KV replicated TP times). GQA-8 on 8 GPUs gives each GPU its own KV head.
5. Each loaded K/V element serves g query heads → $\sim g \times$ more FLOPs per byte. Decode attention is bandwidth-bound, so fewer bytes per unit of work means a faster step.
6. The attention path is expanding K/V to all query heads before computing (materialized repeat_kv/repeat_interleave), or a kernel without native GQA support, or the cache storing expanded heads. Check cache shapes and which kernel is used.
7. MHA: $2 \times 48 \times 48 \times 128 \times 2 = 1.125$ MiB/token → $\times 8,192 \times 16 =$ **144 GiB**. GQA-8: 192 KiB/token → **24 GiB**. Saved 120 GiB $\div 1.5$ GiB per GQA sequence = **80** more sequences.

Part 7: KV in GPU Memory

1. In HBM/VRAM; ~3.35 TB/s on an H100 SXM.
2. Reservation waste, internal fragmentation, external fragmentation.
3. Every step allocates a new $(t+1)$ -row tensor and copies t old rows: $O(t)$ bytes per step per layer, $O(T^2)$ total.
4. Decode is bandwidth-bound: bytes stored = bytes read every step, so fewer bytes = a shorter step.
5. Lengths are unknown, so each buffer must be sized for the maximum: huge waste, few concurrent requests, no sharing, and memory from finished requests can't be reused flexibly.
6. Probably not a leak: engines preallocate the KV pool at startup, so nvidia-smi shows ~full while 40% of blocks are in use. The rest is CUDA context, graphs and allocator cache. A leak would show block usage not returning to baseline when idle.
7. KV: $32 \times 8,192 \times 131,072$ B = 34.4 GB; + 16.1 GB weights = 50.5 GB → $\div 3.35$ TB/s $\approx$ **15.1 ms**; KV $\approx$ **68%** of step time.

Part 8: PagedAttention

1. Logical block i = tokens $[i \cdot B_s, (i+1) \cdot B_s)$ of a sequence; physical block = a slot group in the pool; block table = the per-sequence logical → physical map.
2. $B_s - 1$ token slots, in the last block.
3. Ref counts drop; blocks at 0 return to the free list, or stay as evictable cached blocks with prefix caching.
4. All blocks are the same size and any free block can serve any sequence, since contiguity is never needed. No hole is ever "too small".

5. Full blocks are never written again (append-only), so only the last, partially filled block can diverge between sharers.
6. Short requests waste up to 127 slots each; 200-token chats would waste ~30% of the pool, so fewer sequences fit and the batch shrinks. Also coarser prefix sharing. The capacity loss outweighs the kernel gain.
7. (a) $A \rightarrow [0, 1, 2]$, $B \rightarrow [3, 4]$, $C \rightarrow [5, 6, 7, 8]$. (b) 11 free (9–19). (c) A 50 tokens $\rightarrow$ 4 blocks (+1); B 27 $\rightarrow$ 2 (+0); C 74 $\rightarrow$ 5 (+1) $\rightarrow$ **2** new blocks.

Part 9: Continuous Batching

1. Static: a fixed batch runs until all finish. Dynamic: a batch is formed from a time/size window, then runs statically. Continuous: the batch is recomposed every iteration.
2. $N_{\max} \approx (M \cdot u - W - A) \div (kv_{\text{tok}} \times \bar{L})$.
3. Preemption: a victim sequence is swapped to CPU or dropped and recomputed later.
4. Sequences join, leave and grow every step. Allocation must happen at token/block granularity with no copies or fragmentation, or admissions stall.
5. Its prefill runs inside the shared iteration, so the step is long and everyone's ITL spikes. Its KV also pushes the pool toward preemptions as all sequences grow.
6. KV capacity (free blocks) or an artificial max-sequences cap. Raise it with: FP8 KV, prefix caching, smaller max_model_len/reservations, a GQA model, more GPUs, higher memory utilization, or a higher cap if it was the limit.
7. (a) $72 - 70 - 2 \approx$ **0 GB**: effectively no room for KV. (b) $144 - 70 - 4 = 70$ GB $\rightarrow$ ~213,600 tokens $\rightarrow$ **52** sequences of 4K. (c) FP8 KV (160 KiB/token) $\rightarrow$ ~427K tokens $\rightarrow$ **~104** sequences on two GPUs (still ~0 on one).

Part 10: Prefix Caching

1. TTFT (prefill skipped). ITL unchanged: decode still reads all KV each step.
2. A block's K/V depend on everything before it (attention context and positions), so its identity must include the whole prefix.
3. Misses: an earlier token differs; different offset; template/tokenization change; different model/LoRA/KV dtype; eviction. Not a miss: different sampling parameters.
4. **No**. The document starts at different positions and after different tokens, so its K/V differ (RoPE positions, attention context) and the chained hashes differ.
5. Later blocks are useful only if all earlier ones are present. Evicting from the tail keeps a usable prefix; evicting an early block orphans everything after it.
6. The template change introduced dynamic content (date, name, id) near the start, or changed tokenization of the system prompt. Confirm by diffing token ids of two consecutive requests to find the first mismatch, and check hit-rate metrics around the deploy.
7. Without: $200 \times 4,000 \times 128$ KiB $\approx$ **97.7 GiB**. With: 187 shared full blocks (2,992 tokens) + $200 \times (8 + 1,000) = 204,592$ tokens $\approx$ **25 GiB**.

Part 11: KV Quantization

1. E4M3: 4 exponent / 3 mantissa bits, range ± 448 , finer precision. E5M2: 5/2, range $\pm 57,344$, coarser.
2. Keys have outlier channels, and key errors change softmax scores (which tokens are attended); value errors only blur the blend.
3. Inside the attention kernel, on tiles as they're loaded into registers/SRAM.

4. Decode's ALUs are idle (intensity ≈ 1), so the extra math hides under memory time. Prefill is compute-bound, so extra ops can land on the critical path.
5. Recent tokens usually receive most attention mass, so errors there hurt most.
6. 4-bit KV noise destroys precise long-range retrieval, while chat mostly uses recent context. Plan: run a long-context retrieval eval; go to FP8/INT8, keep K at higher precision or per-channel, keep recent/sink tokens full precision; route that feature to an FP8-KV pool.
7. $KV = 4 \times 8,192 \times 28 \text{ KiB} = 0.875 \text{ GiB}$ (0.94 GB). Step (FP16 KV): $(3.1 + 0.94) \div 192 \text{ GB/s} \approx 21 \text{ ms}$. 8-bit: $(3.1 + 0.47) \div 192 \approx 18.6 \text{ ms}$ ($\sim 11\%$). Weights dominate here, so the main win is memory.

Part 12: Long Context

1. KV memory $\propto n$; prefill attention FLOPs $\propto n^2$; per-step KV reads $\propto n$.
2. $n \approx P \div (L \cdot d) \approx 8.03e9 \div (32 \times 4,096) \approx 61K$ tokens.
3. Tensor parallelism (split KV heads across GPUs) and context/sequence parallelism (split the sequence, e.g. ring attention). Offloading is a third option.
4. A single user's ITL rises modestly ($\sim 2\times$ at 128K), but each long sequence displaces dozens of others from the shared pool: concurrency $\propto 1/n$ ($\sim 64\times$ fewer at 128K vs 2K).
5. It's a modelling property of how attention learned to use context. The cache reproduces the same computation exactly and can't change what the model attends to.
6. TTFT up (long prefills); KV usage up and concurrency down; ITL and p99 up. Before launch: chunked prefill, prefix caching of the document for follow-ups, FP8 KV, a separate long-context pool/routing, per-tenant limits, a context cap.
7. $128K \times 320 \text{ KiB} = 40 \text{ GiB}$ (42.9 GB) $\rightarrow 400 \div 42.9 \approx 9$ sequences; FP8 $\rightarrow 18$; one 128K sequence = $32 \times 4K$ chats.

Part 13: Advanced Techniques

1. Early tokens that soak up a large share of attention regardless of content (softmax needs somewhere to put weight). Dropping them destabilizes attention, so StreamingLLM keeps them plus a recent window.
2. Roll back: truncate the sequence length/positions to the last accepted token and free now-unused slots/blocks (also in the draft model's cache).
3. Long single-iteration prefills that stall decodes (ITL spikes) and spike activation memory. Yes, exact: each chunk attends to the KV of earlier chunks.
4. A sliding-window model was trained to use only the window, so dropping older KV loses nothing it uses. Eviction removes tokens a full-attention model may still need, so it's lossy.
5. When recompute time (FLOPs $\div$ throughput) is below transfer time (bytes $\div$ link bandwidth): short prefixes, small models, slow or busy links, idle compute.
6. Chunked prefill (bounds step time; the long request's TTFT rises slightly) and a separate long-document pool or disaggregated prefill (isolation; costs KV transfer and complexity). Prefix-cache the document for follow-ups.
7. $6,000 \times 327,680 \text{ B} \approx 1.97 \text{ GB} \div 25 \text{ GB/s} \approx 79 \text{ ms} \approx 12\%$ of a 679 ms TTFT. Hide it by streaming KV layer by layer (send layer ℓ while computing $\ell+1$) or chunk-pipelining; FP8 KV halves it.

Part 14: From Scratch

1. A list with one (K, V) pair per layer, each $[B, \text{heads}, T, \text{head_dim}]$.
2. $\text{pos} = \text{arange}(P, P + T)$, with P = current cache length.

3. The single query sits at the last position and may see every key; the rule "mask kpos > qpos" masks nothing.
4. Exact greedy equality proves the cache logic is correct. Sampling adds randomness (and tiny numeric differences can flip sampled tokens), which hides bugs.
5. Each step copies the whole cache (t rows) into a new tensor: $\sum t \propto T^2$ bytes. Copies are memory traffic, not FLOPs.
6. At prompt 32 recomputation is tiny, and launch/Python overhead dominates both versions; the GPU is underused, so recompute is almost free. Use longer prompts, more new tokens, a length sweep, synchronize and warm up.
7. Sketch: left-pad; keep attention_mask [B, S]; position_ids = cumsum(mask) - 1 (clamped at 0); mask keys where attention_mask = 0 in addition to the causal rule; on each decode step append 1 to the mask and use last position + 1 per sequence. Verify with allclose against single-sequence runs.

Part 15: Inspect a Real Model

1. Per-layer K and V for all processed tokens; DynamicCache.
2. [1, 2, 100, 64].
3. The last sampled token was never fed through the model, so its K/V don't exist yet.
4. The cache stores H_{kv} heads; the attention code repeats/indexes each KV head for its g query heads at compute time.
5. Padding wastes memory and compute up to the longest sequence; sequences join and leave at different times; resizing needs copies. A paged pool with ragged batches avoids all of it.
6. DynamicCache is mutated in place (update appends), so the first continuation extended the shared object and the second saw its tokens. Fix: copy.deepcopy the prefix cache per continuation (as bench_hf.py does) or crop it back to the prefix length.
7. Roughly parallel lines with slope ≈ 12 KiB/token. The offset is weights plus temporaries. Steps and bumps come from allocator rounding and from cat briefly holding old + new tensors.

Part 16: KV Cache Lab

1. TTFT (prefill + first token, synchronized); ITL (per-step gaps, mean/p95); tok/s ($B \times \text{new tokens} \div \text{decode time}$); peak memory (max_memory_allocated minus baseline).
2. Full-prompt logits [B, T, vocab] can dwarf the KV cache (~ 20 GB in the example) and OOM a 6 GB card.
3. To test understanding rather than rationalize afterwards. Gaps between prediction and measurement are the findings.
4. Kernel inefficiency at small batch; Python/launch overhead inside ITL; laptop clocks, thermal and power limits; peak bandwidth is theoretical; extra traffic (activations, logits, cache copies).
5. Its per-token KV is large relative to its tiny weights (MHA), so KV effects stand out, and it contrasts with GQA.
6. Log clocks, temperature and power; per-step timestamps; GC events; reserved memory over time. Likely: thermal or power throttling, background GPU use, allocator growth on new sizes, OS/driver scheduling.
7. $kv_{tok} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$; $W = \text{params} \times b$; predicted ITL = $(W + B \cdot T \cdot kv_{tok}) \div 192e9$; predicted peak = $W + B \cdot T \cdot kv_{tok} + \text{overhead}$. Merge into the CSV and plot measured $\div$ predicted.

Part 17: Profiling

1. The fraction of time at least one kernel was running.
2. allocated = live tensors; reserved = memory the caching allocator holds ($\geq$ allocated).

3. Nsight Systems (timeline gaps); Nsight Compute (per-kernel DRAM throughput).
4. Decode is memory-bound: the ALUs are idle, so extra FLOPs execute while waiting for bytes.
5. Each kernel does microseconds of work, so Python and launch latency (several μs $\times$ hundreds of kernels per step) dominate. Fix with CUDA graphs / torch.compile, fused kernels, a static cache, fewer syncs, bigger batches.
6. **A** weight-bandwidth-bound (batch is almost free) → batch more, quantize weights. **B** overhead-bound (gaps; dtype irrelevant) → CUDA graphs/compile, static cache. **C** compute-bound prefill → FP8 tensor-core GEMMs, FlashAttention, prefix caching to skip prefill. **D** KV-bandwidth growth with context → FP8 KV, GQA model, caps on batch $\times$ context. **E** the staircase is normal cache growth (12 KiB $\times$ B per step); the spike is prefill logits/activations → logits_to_keep, chunked prefill. **F** capacity-bound with idle compute → FP8 KV, prefix caching, smaller reservations, more replicas, route long requests.
7. $(3.1 \text{ GB} + 4,096 \times 28,672 \text{ B} \approx 0.117 \text{ GB}) \div 0.021 \text{ s} \approx \mathbf{153 \text{ GB/s}} \approx 80\%$ of 192 GB/s. Bandwidth-bound and already efficient; only fewer bytes (quantized weights or KV) will help much.

Part 18: Production Engines

1. API/tokenizer (ids) → scheduler (who runs) → KV manager (blocks) → batch construction (flatten + metadata) → model execution (forward) → kernels (cache write, attention, GEMMs) → sampler/output (tokens, stream, stop).
2. Block tables and slot mappings (plus sequence lengths/positions).
3. A radix tree (RadixAttention).
4. Admission commits KV for the sequence's whole lifetime and growth. Compute is shared and elastic per step; memory is a hard ceiling whose violation causes preemption and thrashing.
5. Decode steps have small, repetitive shapes where launch overhead matters, so graphs pay off. Prefill shapes vary with prompt length and its kernels are big, so launch overhead is negligible and capturing every shape is impractical.
6. Prefix-cache design (radix/hash, granularity, eviction), cache-aware scheduling and routing, chunked prefill, long-context support. Test by replaying real agent traces and measuring prefix hit rate, TTFT, throughput and memory under concurrency.
7. $8,192 - 200 = \mathbf{7,992}$ prefill tokens per iteration; $20,000 \div 7,992 \rightarrow \mathbf{3}$ iterations before its first token.

Part 19: Failure Modes

1. Compare cached vs uncached greedy outputs token by token, including padded batches.
2. Dynamic content early in the prompt; template/tokenization changes; routing away from the replica holding the prefix; eviction; model/LoRA mismatch.
3. Thrashing: work is redone (preempt → recompute/swap loops), so goodput falls as load rises. Plain overload: the queue grows but throughput stays at its maximum.
4. More sequences mean more KV reads per step (higher ITL) and pool exhaustion → preemptions → recomputation, so less useful work gets done.
5. The adapter changes the K/V projections, so the same tokens produce different K/V. Without the id, one adapter's cache is reused for another: wrong outputs and possible cross-tenant leakage.
6. Contain: disable the suspect feature (e.g. prefix caching) or roll back, preserve logs, involve security. Diagnose: reproduce with the same batch composition; audit hash keys (tenant/adapter salt), ref counts, slot mappings, hash collisions. Fix with tests and tenant-salted keys; postmortem; customer notification per policy.

7. $90K + 40K = 130K > 100K$: it doesn't fit alongside the chats. Only 10K free, so $\geq 30K$ must be freed: ≥ 20 chats preempted (more for growth headroom). Better to queue it or route it elsewhere.

Part 20: Interview Preparation (checkpoint)

1. Causality, numbers, precision, trade-offs, systems view, calibration.
2. "A query is used only by its own token at its own step. Keys and values of past tokens are read by every future query and never change under causal masking, so K/V are worth storing and Q isn't."
3. 512 KiB per token → 2 GiB at 4K tokens.
4. It conflates per-step with total cost and ignores memory. "Per step, attention is $O(t)$ instead of re-running $O(t^2)$; total attention is still $O(n^2)$, and we pay $O(n)$ memory plus reads."
5. State the principle, reason forward from it, estimate with numbers, say what you'd measure, and be explicit about what you're unsure of.
6. Practice task: no fixed answer.
7. Budget $\approx 52.9 \text{ GB} \div 131,072 \text{ B} \approx 404K \text{ tokens} \div 3K \approx \sim 130$ requests. Batch-1 ITL $\approx (16.1 + 0.4) \text{ GB} \div 2.04 \text{ TB/s} \approx 8 \text{ ms}$.

Part 21: System Design

1. $\approx 403 \text{ GB} = 0.9 \times 640 - 141$ (weights) – ~ 32 (activations/workspace).
2. Chunked prefill with a per-iteration token budget, and SLO/KV-aware admission (plus priorities).
3. Long sequences use disproportionate KV and step time; isolating them protects short-chat latency and allows a specialized setup (FP8 KV, disaggregation).
4. Decode is memory-bound: KV capacity and bandwidth decide concurrency and ITL, while FLOPs are plentiful.
5. Over-admission leads to preemption thrash that degrades everyone. Fast rejection protects admitted requests' SLOs and lets clients back off.
6. E.g. FP8 weights + KV ($\sim 2\times$ capacity; validate quality); prefix caching + routing (large win for shared prompts; watch hot spots); right-size TP to run more replicas per node; tune chunked prefill; speculative decoding for latency; route easy traffic to a smaller model.
7. Budget $576 - 70.6 - 32 \approx 473 \text{ GB} \div 163,840 \text{ B} \approx 2.89M \text{ tokens} \rightarrow \sim 940$ sequences at 3K. Per GPU per step: $8.8 \text{ GB weights} + \sim 59 \text{ GB KV} \approx 68 \text{ GB} \rightarrow \sim 20 \text{ ms}$ ideal. 1,000 sequences need $\sim 503 \text{ GB}$ of KV $\rightarrow 2$ nodes (≈ 1.07 by memory; 2 with headroom).

APPENDIX B

Interview Model Answers

Each answer is what a strong candidate says in 60–90 s (levels 1–3) or 3–5 min (levels 4–5, compressed here). The **senior extra** is what separates a good answer from a great one.

Level 1: Fundamentals

1.1 What is a KV cache and what problem does it solve?

During autoregressive decoding every new token attends to all previous tokens at every layer. Because attention is causal, previous tokens' keys and values never change, so we store them per layer and reuse them. Each step then processes only the new token instead of re-running the whole prefix. **Senior extra:** it trades compute for memory capacity and per-step memory bandwidth, which is why decode ends up bandwidth-bound.

1.2 Why cache K and V but not Q?

A query is used only once: by its own token, at its own step. Keys and values are read by every future query. Old queries are never needed, so caching them is pure cost. **Senior extra:** scores and outputs aren't cacheable either, because they depend on the new query.

1.3 Prefill vs decode?

Prefill processes the whole prompt in one parallel pass, fills the cache and produces the first token; it's compute-bound. Decode generates one token per step per sequence, reading the cache; it's sequential and memory-bandwidth-bound. **Senior extra:** they dominate different metrics (TTFT vs ITL) and interfere when scheduled together, hence chunked prefill and disaggregation.

1.4 Is the KV cache an approximation?

No. It's the same computation reorganized; outputs match the uncached run up to floating-point ordering. Approximations only enter with eviction, quantization or windowing on models not trained for it.

1.5 What does `use_cache=True` do?

The forward pass returns `past_key_values` (a Cache object holding per-layer KV , shape $[B, H_{kv}, T, d_v]$). Passing it back with only the new token makes the model append to it and attend over it. `generate()` does this automatically.

1.6 What grows with conversation length even with a cache?

Cache memory, the bytes read each step (ITL rises), attention FLOPs per step, and, if history isn't prefix-cached across turns, the prefill cost of each new turn.

Level 2: ML engineer

2.1 How does KV caching affect complexity?

Per decode step, linear-layer work goes from $O(t)$ to $O(1)$ and attention from $O(t^2)$ to $O(t)$. Over n tokens: linear $O(n^2) \rightarrow O(n)$, attention $O(n^3) \rightarrow O(n^2)$. New costs: $O(n)$ memory per sequence and $O(n)$ bytes read per step. **Senior extra:** e.g. 4K prompt + 500 tokens on 7B: $\sim 470\times$ fewer FLOPs, but the same weight traffic plus ~ 1.1 TB of KV reads.

2.2 Would KV caching reduce TTFT?

Not within one request: prefill creates the cache. TTFT drops only when a cache already exists from earlier work: prefix caching of a shared system prompt or previous conversation turns. **Senior extra:** it can indirectly raise TTFT under load, because KV capacity limits admission and requests queue.

2.3 Derive the memory formula; Llama-2-7B at 4K.

Per token per layer: K and V , each $H_{kv} \times d_h$ values of b bytes $\rightarrow 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$ per token, $\times T \times B$. Llama-2-7B: $2 \times 32 \times 32 \times 128 \times 2 = 512$ KiB/token $\rightarrow 2$ GiB at 4K per sequence.

2.4 Why does GQA reduce KV memory? Cost?

Groups of g query heads share one K/V head, so only $H_{kv} = H_q/g$ heads are stored: $g\times$ less memory and bandwidth. Cost: less attention expressivity (small for GQA, larger for MQA), and it must be trained in. **Senior extra:** it also raises decode attention intensity and works cleanly with tensor parallelism when $H_{kv} \geq \text{TP degree}$.

2.5 Position ids not offset during cached decoding?

Every new token is embedded or rotated as if at position 0 (or the wrong position), so its relationship to cached keys is wrong. The first token looks fine, then output degrades. Fix: positions = cache length + arange(T), per sequence (with padding handled).

2.6 Why not cache with a bidirectional model?

With bidirectional attention, old tokens' hidden states depend on later tokens, so their K/V change whenever a token is added. Cached values would be stale; you must recompute.

Level 3: LLM engineer

3.1 Why might decode be memory-bandwidth-bound?

Each step streams all weights plus every sequence's cache from HBM, but does ~ 2 FLOPs per weight per sequence and ~ 1 FLOP per KV byte, far below the $\sim 150\text{--}300$ FLOP/byte ridge of an A100/H100. Estimate: step $\approx (W + \sum \text{KV bytes}) \div \text{BW}$, e.g. 7B FP16 on an A100 $\approx 7\text{--}8$ ms at batch 1.

3.2 Why does batching help, and why does it stop?

The weight read is shared across the batch, so throughput rises almost linearly while weights dominate. KV reads are per sequence, so as $B \cdot t_{kv_{tok}}$ overtakes W the step time grows linearly with B . Gains flatten, ITL rises, and KV capacity caps B .

3.3 KV quantization: how does it help, when to avoid?

Fewer bytes per element halves (FP8/INT8) memory and KV read time; dequantization is nearly free because decode compute is idle. Avoid or validate carefully with short contexts (little gain), precise long-range retrieval tasks, or missing kernel support. Keys are the sensitive part (outlier channels).

3.4 Prefix caching, and a silent failure?

Reuse KV blocks of an identical token prefix across requests (chained block hashes), skipping prefill and sharing memory. Silent failure: a timestamp or user name at the top of the system prompt changes the first block's hash, every block after it misses, and TTFT quietly doubles.

3.5 What changes at 128K context?

KV per sequence ~16 GiB (8B GQA) or ~40 GiB (70B), so concurrency drops ~60× versus 2K. Decode KV reads match or exceed weight reads (ITL ~2×). Prefill is attention-dominated and takes seconds (TTFT). Fixes: chunked prefill, prefix caching of documents, FP8 KV, a separate long-context pool, multi-GPU KV.

3.6 Speculative decoding and the cache?

The target verifies k draft tokens in one pass, writing k KV entries. Rejected entries must be rolled back (truncate length, free blocks). The draft model has its own cache. Tree speculation needs tree attention masks. It trades extra memory and wasted work for fewer sequential steps; best at low-to-moderate batch.

Level 4: ML systems engineer**4.1 Why can the KV cache become the limiting factor?**

Every active sequence holds KV $\propto$ its length for its lifetime, so tokens in flight $\times kv_{tok}$ must fit beside the weights. Batch size (and therefore throughput, since decode needs batching to amortize weights) is capped by KV capacity, and ITL grows with KV reads. Example: Llama-2-7B on an A100 fits only ~106K tokens (~52 $\times$ 2K chats) while compute sits mostly idle.

4.2 How does PagedAttention help? Cost?

Contiguous max-length reservations waste most memory (reservation + internal + external fragmentation; ~20–40% utilization reported). Paging stores KV in fixed blocks mapped by block tables: on-demand allocation, instant freeing, waste under one block per sequence, ref-counted sharing with copy-on-write. More sequences fit, so throughput rises (2–4× in the paper). Costs: gather indirection in kernels, block-table bookkeeping, block-size tuning.

4.3 Estimate max concurrent requests on an 80 GB GPU.

$N \approx (80 \text{ GB} \times 0.9 - \text{weights} - \text{activations}) \div (kv_{tok} \times \text{avg tokens per request at peak})$.
Llama-3-8B FP16: $(72 - 16.1 - \sim 3) \approx 53 \text{ GB} \div 128 \text{ KiB} \approx 404\text{K tokens} \rightarrow \sim 197 \text{ at } 2\text{K}, \sim 49 \text{ at } 8\text{K}$. Then check the ITL SLO at that batch $((W + KV) \div BW)$ and reduce N if needed.
Units: GB vs GiB.

4.4 Continuous batching and fine-grained KV allocation?

The scheduler rebuilds the batch every iteration: finished sequences leave and waiting ones join, so the GPU never idles on the longest request. This needs KV that can grow by a token and be freed instantly without copies or fragmentation, i.e. paged blocks. The scheduler's real budget is free blocks.

4.5 Compute-, bandwidth- or overhead-bound?

Compute arithmetic intensity vs the ridge point; sweep batch (flat step time $\Rightarrow$ bandwidth), halve bytes (time drops $\Rightarrow$ bandwidth), check ncu DRAM vs SM throughput, and look for nsys timeline gaps (overhead). GPU-Util isn't evidence.

4.6 Chunked prefill, and which metric?

Split long prompts into chunks processed across iterations alongside decodes; each chunk attends to earlier chunks' KV, so it's exact. It protects ITL (tail latency) of running streams at a small TTFT cost for the long request, and it caps activation memory.

Level 5: Inference infrastructure engineer

5.1 How would you design a KV-cache manager?

Pool: preallocate per-layer KV pools of fixed blocks sized from profiled free memory.

Metadata: per-sequence block tables, free list, ref counts, per-block hash for prefix caching (chained, salted with model/adaptor/tenant). **API:** `can_allocate(prompt, headroom)`, `append_slot(seq) → slot mapping`, `fork(seq)` with CoW, `free(seq)`, `match_prefix(tokens)`.

Policies: admission with headroom; preemption (swap vs recompute by length); LRU eviction from chain tails for cached blocks. **Integration:** emits block tables and slot mappings per iteration; kernels read through them; supports hybrid caches (sliding-window layers, quantized KV). **Correctness:** ref-count invariants, tests comparing to the dense reference, fuzzing batch compositions. **Observability:** usage, hit rate, preemptions, fragmentation. **Senior extra:** multi-GPU (TP shards KV heads), CPU/SSD tiers, and cross-replica prefix routing.

5.2 When to disaggregate prefill and decode?

When long prompts make prefill interference break ITL SLOs and co-located tuning can't satisfy both TTFT and ITL; at scale where separate pools can be sized independently. Cost: KV transfer (a 4K-token 70B prompt $\approx$ 1.2 GiB, $\sim$ 26 ms at 50 GB/s; overlap by streaming per layer), RDMA/NVLink fabric, balancing two pools, failure handling mid-handoff.

5.3 p99 ITL doubled after a long-document feature; p50 unchanged.

Tails come from iterations that contain large document prefills (head-of-line blocking) and from preemptions when long sequences exhaust KV. Confirm: correlate spiked iterations with scheduled prefill tokens; check the preemption counter. Fix: chunked prefill with a token budget; route documents to a long-context pool (FP8 KV); prefix-cache documents for follow-ups; SLO-aware admission.

5.4 Cache-aware routing across 20 replicas without hot spots?

Route by a hash of the leading blocks (consistent hashing) so a prefix's traffic concentrates where its KV lives. Use bounded-load consistent hashing or power-of-two choices weighted by queue depth and KV usage, so a hot prefix can spill to a second replica (which then warms). Replicate very hot prefixes proactively. Measure hit rate vs load imbalance.

5.5 Swap vs recompute on preemption?

Swap cost $\approx 2 \times \text{KV bytes} \div \text{link BW (out and back)}$ plus the risk of CPU memory contention. Recompute cost $\approx \text{prefill FLOPs}(t) \div \text{achieved FLOP/s}$, with some prefix reuse. Swap wins for long sequences and big models (KV grows linearly but prefill grows superlinearly at long t); recompute wins for short ones, fast GPUs or busy links. Example: 1 GiB of Llama-3-8B KV (8K tokens): ~ 86 ms round trip over PCIe Gen4 vs $\sim 0.3\text{--}0.8$ s to recompute.

5.6 How would you serve 1M-token contexts?

Memory first: 1M tokens ≈ 128 GiB (8B GQA) or 320 GiB (70B), so shard across GPUs (TP over KV heads plus context parallelism/ring attention). FP8 KV. Prefill $\approx 10^{17}$ FLOPs, so chunked or distributed prefill and aggressive prefix caching of the document (compute once, query many times), perhaps persisted to CPU/SSD tiers. A dedicated pool with low concurrency. Consider models built for it (sliding/local layers, MLA). Validate quality at length.

APPENDIX C

Formula Sheet

Quantity	Formula	Example
KV bytes per token	$kv_{tok} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b$	Llama-2-7B FP16: 512 KiB
KV bytes total	$kv_{tok} \times \Sigma \text{ tokens in flight } (= T \times B \text{ if uniform})$	4K × 8 seqs → 16 GiB
KV budget	$M_{GPU} \cdot u - W - \text{activations}$	A100: 72 - 13.5 - 3 ≈ 55.5 GB
max concurrent sequences	$N_{max} \approx \text{budget} \div (kv_{tok} \cdot \bar{L})$	~52 × 2K (Llama-2-7B)
FLOPs per token (linear layers)	$\approx 2P$	7B: 1.35×10^{10}
attention FLOPs, one token at position t	$\approx 4 \cdot L \cdot d \cdot t$	
prefill FLOPs (n tokens, causal)	$\approx 2P \cdot n + 2 \cdot L \cdot d \cdot n^2$	7B, 4K: 5.8×10^{13}
attention = linear crossover (prefill)	$n \approx P \div (L \cdot d)$	Llama-3-8B: ~61K
decode step time (bandwidth model)	$\approx (W + B \cdot t \cdot kv_{tok}) \div BW$	7B, A100, B=1, 4K: ~7.7 ms
KV vs weights crossover (per step)	$t^* = W \div (B \cdot kv_{tok})$	7B: 25.7K (B=1)
arithmetic intensity, decode linear	$\approx B \text{ FLOP/byte (FP16 weights)}$	ridge ≈ 150–300
arithmetic intensity, decode attention	$\approx 1 \text{ FLOP/byte (MHA)} \cdot \approx g \text{ (GQA)}$	
GQA saving	$g = H_q \div H_{kv}$	Llama-2-70B: 8×
blocks per sequence (paging)	$[\text{tokens} \div B_s]; \text{waste} \leq B_s - 1 \text{ tokens}$	$B_s = 16$
E2E latency	$\approx TTFT + (N_{out} - 1) \cdot ITL$	
system throughput	$\approx B \div ITL$	
transfer time	$\text{bytes} \div \text{link bandwidth}$	1 GiB / PCIe4 ≈ 43 ms
no cache vs cache (T steps)	token passes $T(T+1)/2$ vs T ; scores $\sim T^3/6$ vs $\sim T^2/2$	$T = 1,000$: 500,500 vs 1,000

→ Reference numbers

Hardware	Memory	Bandwidth	Dense BF16
H100 SXM	80 GB HBM3	~3.35 TB/s	~989 TFLOP/s
A100 SXM 80GB	80 GB HBM2e	~2.0 TB/s	312 TFLOP/s
RTX 4050 laptop	6 GB GDDR6	~192 GB/s	tens of TFLOP/s
PCIe Gen4 / Gen5 x16	—	~25–32 / ~50–64 GB/s	—

Glossary

arithmetic intensity	FLOPs per byte moved; compared with the ridge point to decide compute- vs memory-bound.
attention sink	early tokens that absorb large attention mass; kept by StreamingLLM-style eviction.
block / page	fixed-size chunk of KV for B_s tokens in a paged cache.
block table	per-sequence map from logical blocks to physical blocks.
chunked prefill	splitting a long prompt across iterations, mixed with decodes.
continuous batching	iteration-level scheduling; the running batch changes every step.
copy-on-write (CoW)	copy a shared block only when one sharer needs to write into it.
decode	the one-token-per-step generation phase; bandwidth-bound.
disaggregation	separate GPU pools for prefill and decode, with KV transferred between them.
GQA / MQA / MHA	grouped-query / multi-query / multi-head attention: how many K/V heads are stored.
HBM	high-bandwidth memory stacked on the GPU package; where KV lives.
ITL / TPOT	inter-token latency / time per output token.
MFU	model FLOPs utilization: achieved model FLOP/s ÷ peak.
MLA	multi-head latent attention: caches a compressed latent per token (DeepSeek-V2/V3).
PagedAttention	paged KV storage plus kernels that read through block tables (vLLM).
prefill	parallel processing of the prompt; compute-bound; fills the cache.
prefix caching	reusing KV blocks of an identical token prefix across requests.
preemption	evicting a running sequence's KV (swap or recompute) to free blocks.
RadixAttention	SGLang's radix-tree prefix cache with cache-aware scheduling.
ridge point	peak FLOP/s ÷ peak bytes/s of a device.
RoPE	rotary position embedding; rotates q and k by position before the dot product.
slot mapping	where each new token's K/V is written in the paged pool.
speculative decoding	draft several tokens cheaply, verify them in one target forward pass.
TTFT	time to first token.

The papers behind the ideas

Idea (Part)	Paper
MQA (6)	Shazeer, <i>Fast Transformer Decoding: One Write-Head is All You Need</i> , 2019
GQA (6)	Ainslie et al., <i>GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints</i> , 2023
FlashAttention (2, 4)	Dao et al., <i>FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness</i> , 2022
PagedAttention / vLLM (8)	Kwon et al., <i>Efficient Memory Management for Large Language Model Serving with PagedAttention</i> , SOSP 2023
Iteration-level scheduling (9)	Yu et al., <i>Orca: A Distributed Serving System for Transformer-Based Generative Models</i> , OSDI 2022
RadixAttention (10, 18)	Zheng et al., <i>SGLang: Efficient Execution of Structured Language Model Programs</i> , 2024
KV quantization (11)	Liu et al., <i>KIVI</i> , 2024 · Hooper et al., <i>KVQuant</i> , 2024
Attention sinks (13)	Xiao et al., <i>Efficient Streaming Language Models with Attention Sinks</i> , 2023
Heavy hitters / SnapKV (13)	Zhang et al., <i>H2O</i> , 2023 · Li et al., <i>SnapKV</i> , 2024
MLA (6, 13)	DeepSeek-A1, <i>DeepSeek-V2 technical report</i> , 2024
Chunked prefill (13)	Agrawal et al., <i>Sarathi-Serve</i> , OSDI 2024
Disaggregation (13)	Zhong et al., <i>DistServe</i> , OSDI 2024 · Patel et al., <i>Splitwise</i> , 2024 · Qin et al., <i>Mooncake</i>
Speculative decoding (13)	Leviathan et al., <i>Fast Inference from Transformers via Speculative Decoding</i> , 2023
vAttention (8)	Prabhu et al., <i>vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention</i> , 2024

Engines evolve quickly. For current flags and features, read the vLLM, SGLang, TensorRT-LLM and transformers documentation for the version you run.